DuocUC | ONLINE



DESARROLLO BACKEND III (PBY2203)

Informe escrito

Nombre estudiante: Raúl Zúñiga	
Asignatura: Desarrollo Backend III	**Carrera: Desarrollo de Aplicaciones**
Profesor: Gabriel Grobier	**Fecha:11-10-2026**

Descripción de la actividad

La Evaluación Final Transversal llamada “Desarrollo Backend Avanzado: Spring Cloud y Batch”, consiste en generar una Entrega de encargo con presentación donde, de manera individual, deberás desarrollar una solución Backend según un caso práctico de la industria. Este desarrollo implica la implementación de una arquitectura moderna basada en microservicios, utilizando frameworks y desplegando las soluciones solicitadas en un entorno de trabajo en la nube. El proyecto cubre la migración de procesos batch, la implementación de Backend for Frontend (BFF) para distintos canales y la creación de microservicios seguros y resilientes, cumpliendo con los requerimientos de escalabilidad, seguridad y eficiencia operativa.

Instrucciones específicas

Para llevar a cabo la Evaluación Final Transversal de la asignatura Desarrollo Backend III, tendrás que analizar el contexto y realizar los pasos siguientes:

Contexto

El Banco XYZ es una institución financiera con más de 30 años en el mercado. Su sistema de gestión bancaria ha estado funcionando sobre una plataforma legacy basada en COBOL y scripts Shell, ejecutándose en servidores mainframe. Este sistema maneja operaciones críticas como procesamiento de transacciones, generación de estados de cuenta, cálculos de intereses y gestión de clientes.

Debido a limitaciones en escalabilidad, altos costos de mantenimiento y dificultades para integrar nuevas tecnologías, el banco ha decidido modernizar su infraestructura tecnológica. El objetivo es migrar su sistema legacy a una arquitectura moderna basada en microservicios, alojada en la nube, que permita mayor flexibilidad, resiliencia y capacidad de adaptación a las necesidades actuales del mercado.

Ahora que ya tienes el contexto del Banco XYZ, desarrolla cada paso a paso, según las siguientes secciones solicitadas:

Entregable

- **Evidencia de Arquitectura General:**

Para la arquitectura del proyecto se deja el extracto del archivo pom.xml donde se puede observar el multi-módulo hecho en Maven, lo cual permite ejecutar los microservicios.



- **Evidencia de Docker Compose:**

docker-compose.yml

```yaml
services:
  postgres:
    image: postgres:16
    container_name: banco-xyz-postgres
    environment:
      POSTGRES_DB: banco_xyz
      POSTGRES_USER: banco_user
      POSTGRES_PASSWORD: banco_password_local
    ports:
      - "5433:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U banco_user -d banco_xyz"]
      interval: 5s
      timeout: 5s
      retries: 10

  kafka:
    image: apache/kafka:3.9.1
    container_name: banco-xyz-kafka
    ports:
      - "29092:29092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,PLAINTEXT_HOST://:29092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:2909
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLA
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_NUM_PARTITIONS: 3
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9
      interval: 10s
      timeout: 10s
      retries: 20

```

```yaml
  config-server:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: config-server
    ports:
      - "8888:8888"
    volumes:
      - ./config-repo:/app/config-repo:ro
    depends_on:
      postgres:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8888/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 20

  eureka-server:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: eureka-server
    ports:
      - "8761:8761"
    depends_on:
      config-server:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8761/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 30

  batch-init:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: core-banking-service
```

```
    entrypoint: ["/usr/local/bin/batch-init.sh"]
    environment:
      CONFIG_SERVER_URL: http://config-server:8888
      EUREKA_URL: http://eureka-server:8761/eureka/
      KAFKA_BOOTSTRAP_SERVERS: kafka:9092
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/banco_xyz
      SPRING_DATASOURCE_USERNAME: banco_user
      SPRING_DATASOURCE_PASSWORD: banco_password_local
    depends_on:
      postgres:
        condition: service_healthy
      kafka:
        condition: service_healthy
      config-server:
        condition: service_healthy
      eureka-server:
        condition: service_healthy

  core-banking-service:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: core-banking-service
    environment:
      CONFIG_SERVER_URL: http://config-server:8888
      EUREKA_URL: http://eureka-server:8761/eureka/
      EUREKA_INSTANCE_HOSTNAME: core-banking-service
      KAFKA_BOOTSTRAP_SERVERS: kafka:9092
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/banco_xyz
      SPRING_DATASOURCE_USERNAME: banco_user
      SPRING_DATASOURCE_PASSWORD: banco_password_local
      SPRING_BATCH_JOB_ENABLED: "false"
    ports:
      - "8081:8081"
    depends_on:
      kafka:
        condition: service_healthy
      batch-init:
```

```
125         condition: service_completed_successfully
126     healthcheck:
127       test: ["CMD-SHELL", "curl -fsS http://localhost:8081/actuator/health"]
128       interval: 10s
129       timeout: 5s
130       retries: 30
131
132   account-service:
133     build:
134       context: .
135       dockerfile: docker/Dockerfile.service
136       args:
137         MODULE: account-service
138     environment:
139       CONFIG_SERVER_URL: http://config-server:8888
140       EUREKA_URL: http://eureka-server:8761/eureka/
141       EUREKA_INSTANCE_HOSTNAME: account-service
142       OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl
143     ports:
144       - "8085:8085"
145     depends_on:
146       eureka-server:
147         condition: service_healthy
148     healthcheck:
149       test: ["CMD-SHELL", "curl -fsS http://localhost:8085/actuator/health"]
150       interval: 10s
151       timeout: 5s
152       retries: 30
153
154   payment-service:
155     build:
156       context: .
157       dockerfile: docker/Dockerfile.service
158       args:
159         MODULE: payment-service
160     environment:
161       CONFIG_SERVER_URL: http://config-server:8888
162       EUREKA_URL: http://eureka-server:8761/eureka/
163       EUREKA_INSTANCE_HOSTNAME: payment-service
164       KAFKA_BOOTSTRAP_SERVERS: kafka:9092
```

```
      OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl(
    ports:
      - "8086:8086"
    depends_on:
      kafka:
        condition: service_healthy
      eureka-server:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8086/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 30

  customer-service:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
      args:
        MODULE: customer-service
    environment:
      CONFIG_SERVER_URL: http://config-server:8888
      EUREKA_URL: http://eureka-server:8761/eureka/
      EUREKA_INSTANCE_HOSTNAME: customer-service
      OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl(
    ports:
      - "8087:8087"
    depends_on:
      eureka-server:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS http://localhost:8087/actuator/health"]
      interval: 10s
      timeout: 5s
      retries: 30

  bff-web-service:
    build:
      context: .
      dockerfile: docker/Dockerfile.service
```

```
    args:
      MODULE: bff-web-service
  environment:
    CONFIG_SERVER_URL: http://config-server:8888
    EUREKA_URL: http://eureka-server:8761/eureka/
    EUREKA_INSTANCE_HOSTNAME: bff-web-service
    OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl(
    OAUTH2_ISSUER: banco-xyz-auth
  ports:
    - "8082:8082"
  depends_on:
    core-banking-service:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "curl -fsS http://localhost:8082/actuator/health"]
    interval: 10s
    timeout: 5s
    retries: 30

bff-mobile-service:
  build:
    context: .
    dockerfile: docker/Dockerfile.service
    args:
      MODULE: bff-mobile-service
  environment:
    CONFIG_SERVER_URL: http://config-server:8888
    EUREKA_URL: http://eureka-server:8761/eureka/
    EUREKA_INSTANCE_HOSTNAME: bff-mobile-service
    OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl(
    OAUTH2_ISSUER: banco-xyz-auth
  ports:
    - "8083:8083"
  depends_on:
    core-banking-service:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "curl -fsS http://localhost:8083/actuator/health"]
    interval: 10s
    timeout: 5s
```

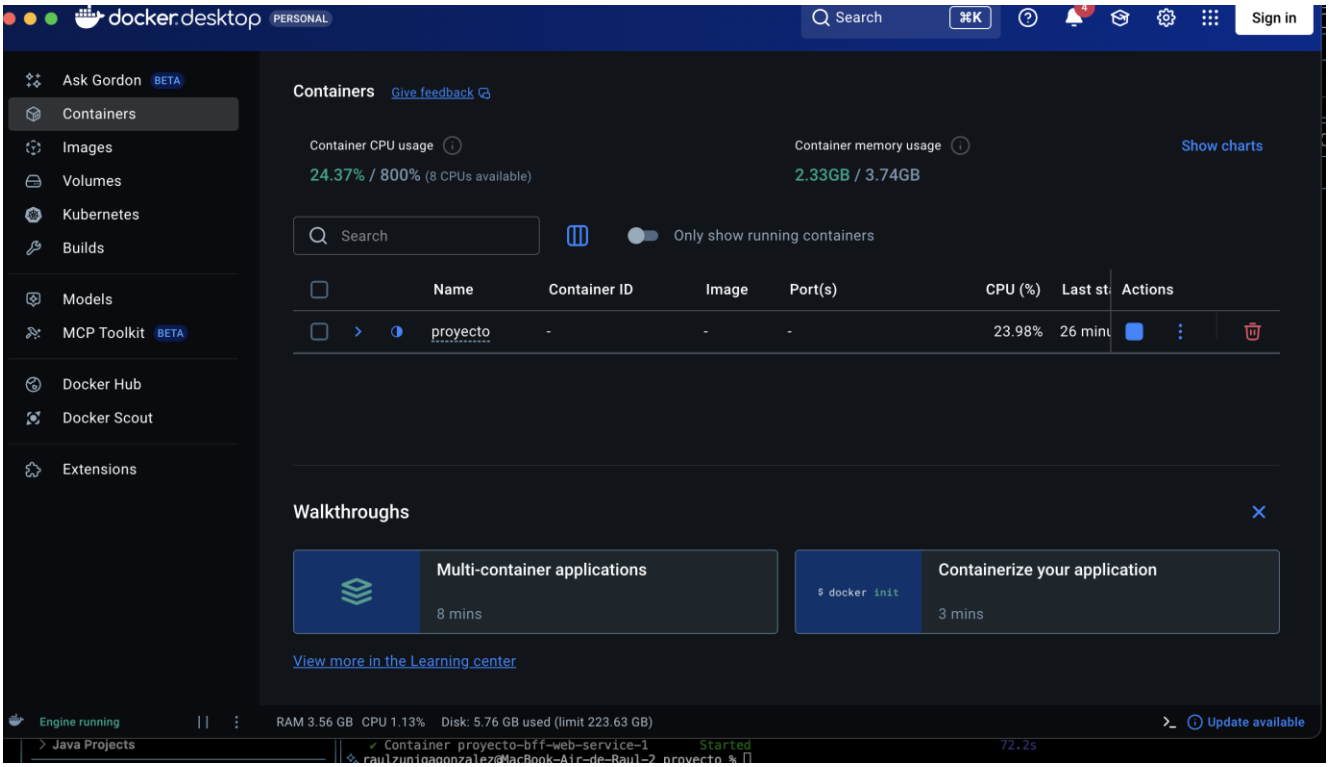
```
245       retries: 30
246
247   bff-atm-service:
248     build:
249       context: .
250       dockerfile: docker/Dockerfile.service
251       args:
252         MODULE: bff-atm-service
253     environment:
254       CONFIG_SERVER_URL: http://config-server:8888
255       EUREKA_URL: http://eureka-server:8761/eureka/
256       EUREKA_INSTANCE_HOSTNAME: bff-atm-service
257       KAFKA_BOOTSTRAP_SERVERS: kafka:9092
258       OAUTH2_JWT_SECRET: QmFuY28tWFlaLVNlbWFuYS04LU9BdXRoMi1jbGF2ZS1zaW1ldHJpY2EtMzItYnl(
259       OAUTH2_ISSUER: banco-xyz-auth
260     ports:
261       - "8084:8084"
262     depends_on:
263       kafka:
264         condition: service_healthy
265       core-banking-service:
266         condition: service_healthy
267     healthcheck:
268       test: ["CMD-SHELL", "curl -fsS http://localhost:8084/actuator/health"]
269       interval: 10s
270       timeout: 5s
271       retries: 30
272
273   volumes:
274     postgres_data:
275
```

Con el archivo de docker compose creado, lo que debemos hacer en la terminal es escribir el siguiente comando “docker compose up -d –build” y comenzará a ejecutarse docker compose.

```
raulzunigagonzalez@MacBook-Air-de-Raul-2 proyecto % docker compose up -d --build
[+] Running 29/29
 ✔ postgres Pulled                                                                14.5s
 ✔ kafka Pulled                                                                   15.7s
[+] Building 123.0s (58/58) FINISHED
 => [internal] load local bake definitions                                         0.0s
 => => reading from stdin 6.94kB                                                   0.0s
 => [batch-init internal] load build definition from Dockerfile.service            0.0s
 => => transferring dockerfile: 853B                                               0.0s
 => [bff-web-service internal] load metadata for docker.io/library/eclipse-temurin:17-  1.9s
 => [core-banking-service internal] load metadata for docker.io/library/eclipse-temuri  1.9s
 => [eureka-server internal] load .dockerignore                                    0.0s
 => => transferring context: 95B                                                   0.0s
 => [bff-web-service build  1/12] FROM docker.io/library/eclipse-temurin:17-jdk@sha256  8.6s
 => => resolve docker.io/library/eclipse-temurin:17-jdk@sha256:5d6042fb8cdc14d614e4e42  0.0s
 => => sha256:97ea7a078a6c8f4aaeb8fa58a0658f6d3631d002252fc264f6be73a8 2.46kB / 2.46kB  0.2s
 => => sha256:70cb2bb6cf6f828366577597e6891d2194b4084dd90082d604d1673f1b6c 158B / 158B  0.2s
 => => sha256:12a4be5f67a4d7dfd447bc299b563bdc9b95b1f453c176a1332e 144.66MB / 144.66MB  6.3s
 => => sha256:787458d0162650daabd0e631a9c7a9b07e87bbe6c06ee05c477799 27.27MB / 27.27MB  2.6s
 => => sha256:06db5ba23986104cd78dcfa853ddf2354c943d2efefc333eca2b44 40.86MB / 40.86MB  6.1s
 => => sha256:5fa77813dc560f299656013d742628128cac51fdfda979349e96a9f09497 392B / 392B  0.4s
 => => extracting sha256:06db5ba23986104cd78dcfa853ddf2354c943d2efefc333eca2b447d4559d  0.6s
 => => extracting sha256:5fa77813dc560f299656013d742628128cac51fdfda979349e96a9f094971  0.0s
 => => extracting sha256:787458d0162650daabd0e631a9c7a9b07e87bbe6c06ee05c4777995a1121f  0.5s
 => => extracting sha256:12a4be5f67a4d7dfd447bc299b563bdc9b95b1f453c176a1332ebb96f75a9  1.1s
 => => extracting sha256:70cb2bb6cf6f828366577597e6891d2194b4084dd90082d604d1673f1b6cb  0.0s
 => => extracting sha256:97ea7a078a6c8f4aaeb8fa58a0658f6d3631d002252fc264f6be73a87eee7  0.0s
 => [bff-atm-service internal] load build context                                  3.5s
 => => transferring context: 360.58MB                                              3.4s
 => [batch-init stage-1 1/6] FROM docker.io/library/eclipse-temurin:17-jre@sha256:207  10.4s
 => => resolve docker.io/library/eclipse-temurin:17-jre@sha256:207ecae0b2b104dfc6dfa76  0.0s
 => => sha256:e4a43cb3a392993c9e2c3adb64bf615daa6b46e847d9cca84a9abd62 2.46kB / 2.46kB  0.2s
 => => sha256:0fc159d94561054acfa203f20f782efd944e6757968392e5e0a67ae6cfda 158B / 158B  0.2s
 => => sha256:b4ff5a44a909cc6d314b2fc8a3d748411e5acea8a6b5eb544c4fbd 47.00MB / 47.00MB  7.5s
 => => sha256:1e5320babd49d050bda58cba3fedbe9f9118f2eab0b9c5af2b548a 23.27MB / 23.27MB  2.5s
 => => extracting sha256:1e5320babd49d050bda58cba3fedbe9f9118f2eab0b9c5af2b548af2b7799  0.5s
 => => extracting sha256:b4ff5a44a909cc6d314b2fc8a3d748411e5acea8a6b5eb544c4fbd885f361  1.3s
 => => extracting sha256:0fc159d94561054acfa203f20f782efd944e6757968392e5e0a67ae6cfda7  0.0s
 => => extracting sha256:e4a43cb3a392993c9e2c3adb64bf615daa6b46e847d9cca84a9abd62fc7b2  0.5s
 => [batch-init build  2/12] WORKDIR /workspace                                    0.9s
 => [core-banking-service build  3/12] COPY .mvn .mvn                              0.9s
```

```
 => => exporting attestation manifest sha256:b1a627c3cd42eae4ed81e7181cb5088c35d118925  0.0s
 => => exporting manifest list sha256:3b8b0c3115dbda424ab8153fa2dfb20108085c478ceda9da  0.0s
 => => naming to docker.io/library/proyecto-config-server:latest                       0.0s
 => => unpacking to docker.io/library/proyecto-config-server:latest                    0.5s
 => [bff-web-service] exporting to image                                               2.6s
 => => exporting layers                                                                2.0s
 => => exporting manifest sha256:687e6f744e05aa73cab7d4e4c00a69e4674e75c748b08efc1ac5a  0.0s
 => => exporting config sha256:a79b52cac7db0b37d5eeb56a5fb246f65bc4f9f44efa9b1e0ae7fa9  0.0s
 => => exporting attestation manifest sha256:4b0169f82429e9cea31ca688c04db628e44b25b8e  0.0s
 => => exporting manifest list sha256:9ad82e5fc607c980456559b49d0b23acf9ace35a817163c0  0.0s
 => => naming to docker.io/library/proyecto-bff-web-service:latest                     0.0s
 => => unpacking to docker.io/library/proyecto-bff-web-service:latest                  0.6s
 => [config-server] resolving provenance for metadata file                             0.1s
 => [bff-web-service] resolving provenance for metadata file                           0.1s
 => [eureka-server] resolving provenance for metadata file                             0.1s
 => [bff-mobile-service] resolving provenance for metadata file                        0.0s
 => [core-banking-service] resolving provenance for metadata file                      0.0s
 => [bff-atm-service] resolving provenance for metadata file                           0.0s
 => [batch-init] resolving provenance for metadata file                                0.0s
[+] Running 18/18
 ✔ proyecto-config-server                    Built                                    0.0s
 ✔ proyecto-eureka-server                    Built                                    0.0s
 ✔ proyecto-batch-init                       Built                                    0.0s
 ✔ proyecto-core-banking-service             Built                                    0.0s
 ✔ proyecto-bff-web-service                  Built                                    0.0s
 ✔ proyecto-bff-mobile-service               Built                                    0.0s
 ✔ proyecto-bff-atm-service                  Built                                    0.0s
 ✔ Network proyecto_default                  Created                                  0.0s
 ✔ Volume proyecto_postgres_data             Create...                                0.0s
 ✔ Container banco-xyz-kafka                 Healthy                                 62.4s
 ✔ Container banco-xyz-postgres              Healthy                                 25.3s
 ✔ Container proyecto-config-server-1        H...                                    25.0s
 ✔ Container proyecto-eureka-server-1        H...                                    35.5s
 ✔ Container proyecto-batch-init-1           Exit...                                 61.4s
 ✔ Container proyecto-core-banking-service-1 Healthy                                 72.0s
 ✔ Container proyecto-bff-atm-service-1      Started                                 72.3s
 ✔ Container proyecto-bff-mobile-service-1   Started                                 72.3s
 ✔ Container proyecto-bff-web-service-1      Started                                 72.2s
raulzunigagonzalez@MacBook-Air-de-Raul-2 proyecto %
```

Si me voy a docker desktop podré ver que se ha creado el contenedor.



- **Evidencia de Spring Batch**

Para evidenciar la implementación de Spring Batch se generaron las siguientes clases:

- core-banking-service/src/main/java/cl/duoc/bancoxyz/config/DailyTransactionsJobConfig.java

DailyTransactionsJobConfig.java

core-banking-service > src > main > java > cl > duoc > bancoxyz > config > DailyTransactionsJobConfig.java

```java
package cl.duoc.bancoxyz.config;

import cl.duoc.bancoxyz.model.DailyTransaction;
import cl.duoc.bancoxyz.processor.DailyTransactionProcessor;
import org.springframework.batch.core.Job;
import org.springframework.batch.core.Step;
import org.springframework.batch.core.configuration.annotation.StepScope;
import org.springframework.batch.core.job.builder.JobBuilder;
import org.springframework.batch.core.repository.JobRepository;
import org.springframework.batch.core.step.builder.StepBuilder;
import org.springframework.batch.item.database.JdbcBatchItemWriter;
import org.springframework.batch.item.database.builder.JdbcBatchItemWriterBuilder;
import org.springframework.batch.item.file.FlatFileItemReader;
import org.springframework.batch.item.file.builder.FlatFileItemReaderBuilder;
import org.springframework.batch.item.support.SynchronizedItemStreamReader;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.io.ClassPathResource;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.retry.backoff.BackOffPolicy;
import org.springframework.retry.RetryPolicy;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

// Configura el Job y sus componentes para las transacciones diarias
@Configuration
public class DailyTransactionsJobConfig {

    // Lee las transacciones desde el archivo CSV legacy
    @Bean
    public SynchronizedItemStreamReader<DailyTransaction> dailyTransactionsReader() {
        FlatFileItemReader<DailyTransaction> reader = new FlatFileItemReaderBuilder<Daily
                .name("dailyTransactionsReader")
                .resource(new ClassPathResource("input/transacciones.csv"))
                .linesToSkip(1)
                .delimited()
                .names("id", "fecha", "monto", "tipo")
                .targetType(DailyTransaction.class)
                .saveState(false)
                .build();
        SynchronizedItemStreamReader<DailyTransaction> synchronizedReader = new Synchron
        synchronizedReader.setDelegate(reader);
        return synchronizedReader;
    }
```

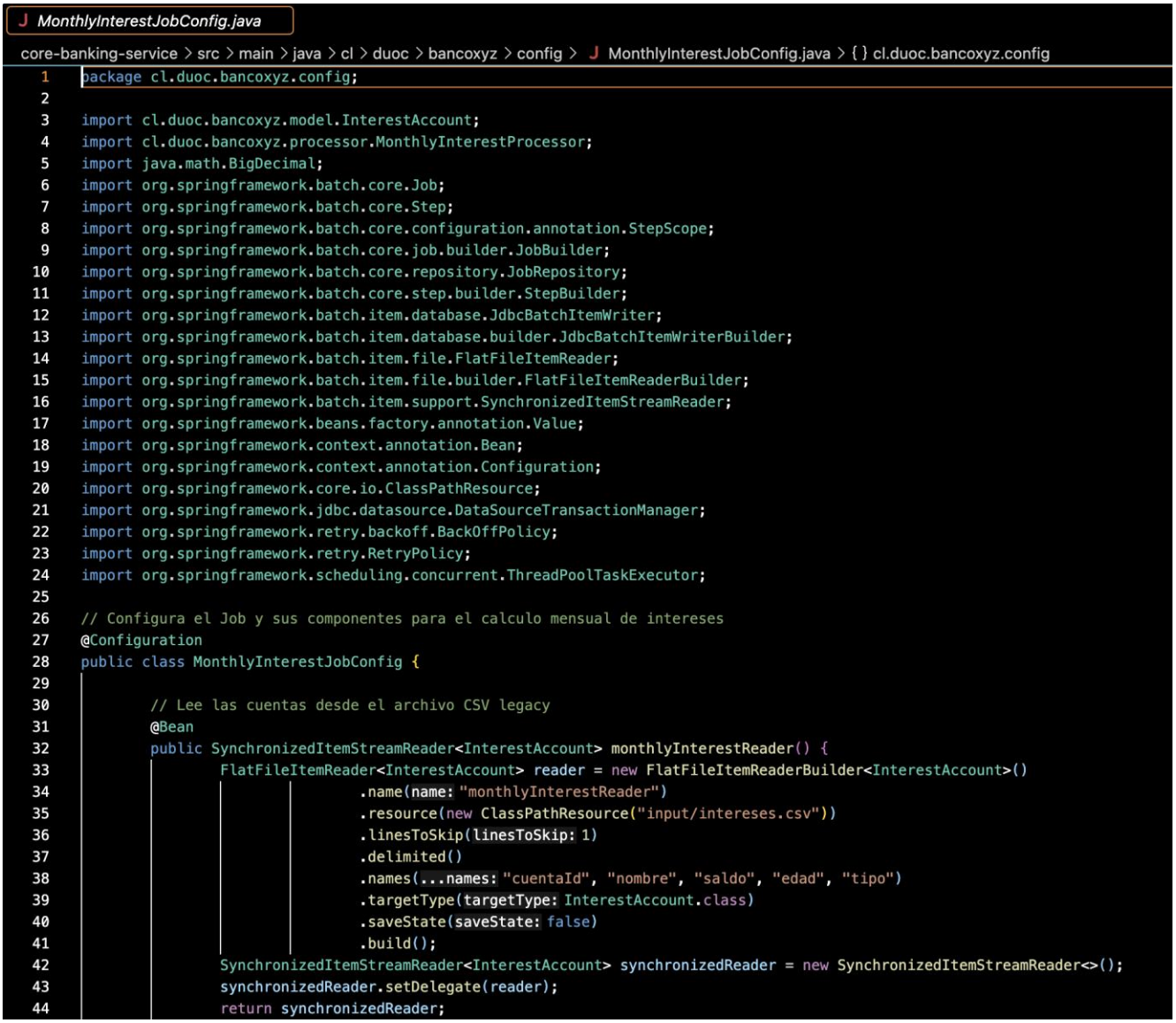
```java
    // Crea el procesador que valida cada transaccion
    @Bean
    @StepScope
    public DailyTransactionProcessor dailyTransactionsProcessor(
            org.springframework.jdbc.core.JdbcTemplate jdbcTemplate) {
        return new DailyTransactionProcessor(jdbcTemplate);
    }

    // Guarda las transacciones validas en PostgreSQL mediante JDBC
    @Bean
    public JdbcBatchItemWriter<DailyTransaction> dailyTransactionsWriter(javax.sql.DataSo
        return new JdbcBatchItemWriterBuilder<DailyTransaction>()
                .dataSource(dataSource)
                .sql("""
                        INSERT INTO transacciones_procesadas (id, fecha, monto, tipo)
                        VALUES (:transactionId, :transactionDate, :amount, :transactionTy
                        ON CONFLICT (id) DO UPDATE SET fecha = EXCLUDED.fecha,
                        monto = EXCLUDED.monto, tipo = EXCLUDED.tipo
                        """)
                .beanMapped()
                .build();
    }

    // Define el flujo paralelo Reader - Processor - Writer con manejo de registros inval
    @Bean
    public Step dailyTransactionsStep(JobRepository jobRepository,
            DataSourceTransactionManager transactionManager,
            SynchronizedItemStreamReader<DailyTransaction> dailyTransactionsReader,
            DailyTransactionProcessor dailyTransactionsProcessor,
            JdbcBatchItemWriter<DailyTransaction> dailyTransactionsWriter,
            ThreadPoolTaskExecutor batchTaskExecutor,
            BankDataSkipPolicy bankDataSkipPolicy,
            BatchSkipListener batchSkipListener,
            StepPerformanceListener stepPerformanceListener,
            RetryPolicy transientDataAccessRetryPolicy,
            BackOffPolicy batchBackOffPolicy,
            @Value("${bank.batch.chunk-size}") int chunkSize) {
        return new StepBuilder("dailyTransactionsStep", jobRepository)
                .<DailyTransaction, DailyTransaction>chunk(chunkSize, transactionManager)
                .reader(dailyTransactionsReader)
                .processor(dailyTransactionsProcessor)
                .writer(dailyTransactionsWriter)
```

```
88                .faultTolerant()
89                .skipPolicy(bankDataSkipPolicy)
90                .retryPolicy(transientDataAccessRetryPolicy)
91                .backOffPolicy(batchBackOffPolicy)
92                .listener(batchSkipListener)
93                .listener(stepPerformanceListener)
94                .taskExecutor(batchTaskExecutor)
95                .build();
96        }
97
98        // Define el Job encargado del reporte diario de transacciones
99        @Bean
100       public Job dailyTransactionsJob(JobRepository jobRepository,
101               Step dailyTransactionsStep,
102               DailyTransactionsSummaryListener dailyTransactionsSummaryListener) {
103           return new JobBuilder("dailyTransactionsJob", jobRepository)
104                   .start(dailyTransactionsStep)
105                   .listener(dailyTransactionsSummaryListener)
106                   .build();
107       }
108   }
109
```

- core-banking-service/src/main/java/cl/duoc/bancoxyz/config/MonthlyInterestJobConfig.java




```java
package cl.duoc.bancoxyz.config;

import cl.duoc.bancoxyz.model.InterestAccount;
import cl.duoc.bancoxyz.processor.MonthlyInterestProcessor;
import java.math.BigDecimal;
import org.springframework.batch.core.Job;
import org.springframework.batch.core.Step;
import org.springframework.batch.core.configuration.annotation.StepScope;
import org.springframework.batch.core.job.builder.JobBuilder;
import org.springframework.batch.core.repository.JobRepository;
import org.springframework.batch.core.step.builder.StepBuilder;
import org.springframework.batch.item.database.JdbcBatchItemWriter;
import org.springframework.batch.item.database.builder.JdbcBatchItemWriterBuilder;
import org.springframework.batch.item.file.FlatFileItemReader;
import org.springframework.batch.item.file.builder.FlatFileItemReaderBuilder;
import org.springframework.batch.item.support.SynchronizedItemStreamReader;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.retry.backoff.BackOffPolicy;
import org.springframework.retry.RetryPolicy;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

// Configura el Job y sus componentes para el calculo mensual de intereses
@Configuration
public class MonthlyInterestJobConfig {

        // Lee las cuentas desde el archivo CSV legacy
        @Bean
        public SynchronizedItemStreamReader<InterestAccount> monthlyInterestReader() {
                FlatFileItemReader<InterestAccount> reader = new FlatFileItemReaderBuilder<InterestAccount>()
                                .name(name: "monthlyInterestReader")
                                .resource(new ClassPathResource("input/intereses.csv"))
                                .linesToSkip(linesToSkip: 1)
                                .delimited()
                                .names(...names: "cuentaId", "nombre", "saldo", "edad", "tipo")
                                .targetType(targetType: InterestAccount.class)
                                .saveState(saveState: false)
                                .build();
                SynchronizedItemStreamReader<InterestAccount> synchronizedReader = new SynchronizedItemStreamReader<>();
                synchronizedReader.setDelegate(reader);
                return synchronizedReader;
```

```java
        }

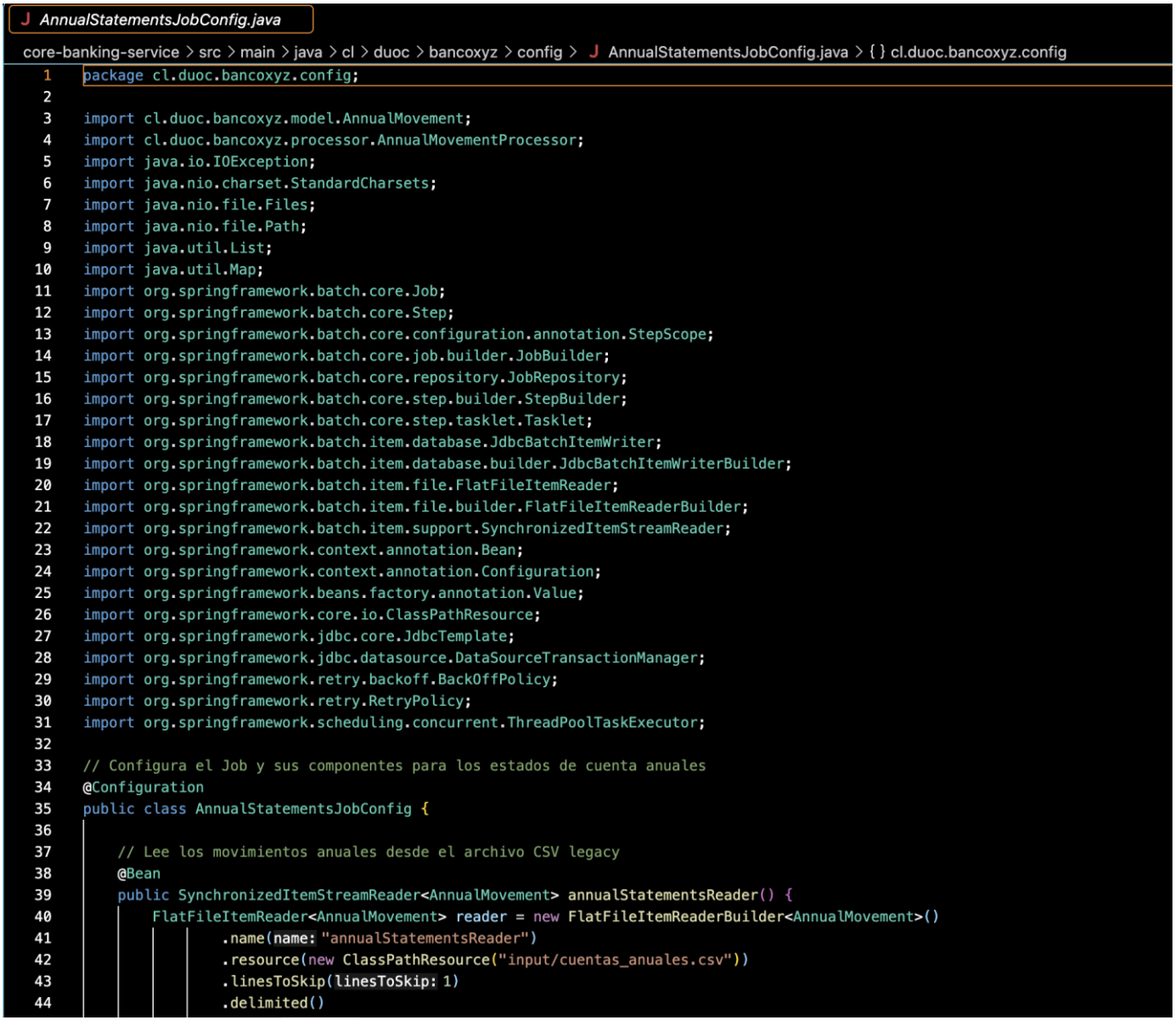
        // Crea el procesador que calcula el saldo final de cada cuenta
        @Bean
        @StepScope
        public MonthlyInterestProcessor monthlyInterestProcessor(
                @Value("${bank.interest.savings-rate}") BigDecimal savingsRate,
                @Value("${bank.interest.loan-rate}") BigDecimal loanRate) {
            return new MonthlyInterestProcessor(savingsRate, loanRate);
        }

        // Guarda los intereses calculados en PostgreSQL mediante JDBC
        @Bean
        public JdbcBatchItemWriter<InterestAccount> monthlyInterestWriter(javax.sql.DataSource dataSource) {
            return new JdbcBatchItemWriterBuilder<InterestAccount>()
                    .dataSource(dataSource)
                    .sql("""
                            INSERT INTO intereses_mensuales
                            (cuenta_id, nombre, tipo, saldo_inicial, tasa, saldo_final)
                            VALUES (:accountId, :accountName, :accountType, :initialBalance, :rate, :finalB
                            ON CONFLICT (cuenta_id) DO UPDATE SET nombre = EXCLUDED.nombre, tipo = EXCLUDE
                            saldo_inicial = EXCLUDED.saldo_inicial, tasa = EXCLUDED.tasa, saldo_final = EX
                            """)
                    .beanMapped()
                    .build();
        }

        // Define el flujo Reader - Processor - Writer para las cuentas validas
        @Bean
        public Step monthlyInterestStep(JobRepository jobRepository,
                DataSourceTransactionManager transactionManager,
                SynchronizedItemStreamReader<InterestAccount> monthlyInterestReader,
                MonthlyInterestProcessor monthlyInterestProcessor,
                JdbcBatchItemWriter<InterestAccount> monthlyInterestWriter,
                ThreadPoolTaskExecutor batchTaskExecutor,
                BankDataSkipPolicy bankDataSkipPolicy,
                BatchSkipListener batchSkipListener,
                StepPerformanceListener stepPerformanceListener,
                RetryPolicy transientDataAccessRetryPolicy,
                BackOffPolicy batchBackOffPolicy,
                @Value("${bank.batch.chunk-size}") int chunkSize) {
            return new StepBuilder("monthlyInterestStep", jobRepository)
                    .<InterestAccount, InterestAccount>chunk(chunkSize, transactionManager)
                    .reader(monthlyInterestReader)
                    .processor(monthlyInterestProcessor)
                    .writer(monthlyInterestWriter)
                    .faultTolerant()
                    .skipPolicy(bankDataSkipPolicy)
                    .retryPolicy(transientDataAccessRetryPolicy)
                    .backOffPolicy(batchBackOffPolicy)
                    .listener(batchSkipListener)
                    .listener(stepPerformanceListener)
                    .taskExecutor(batchTaskExecutor)
                    .build();
        }

        // Define el Job encargado del calculo mensual de intereses
        @Bean
        public Job monthlyInterestJob(JobRepository jobRepository, Step monthlyInterestStep,
                JobSummaryLoggingListener jobSummaryLoggingListener) {
            return new JobBuilder("monthlyInterestJob", jobRepository)
                    .start(monthlyInterestStep)
                    .listener(jobSummaryLoggingListener)
                    .build();
        }
    }
```

- core-banking-service/src/main/java/cl/duoc/bancoxyz/config/AnnualStatementsJobConfig.java



```
            .names(...names: "cuentaId", "fecha", "transaccion", "monto", "descripcion")
            .targetType(targetType: AnnualMovement.class)
            .saveState(saveState: false)
            .build();
        SynchronizedItemStreamReader<AnnualMovement> synchronizedReader = new SynchronizedItemStreamReader<>();
        synchronizedReader.setDelegate(reader);
        return synchronizedReader;
    }

    // Crea el procesador que valida cada movimiento anual
    @Bean
    @StepScope
    public AnnualMovementProcessor annualStatementsProcessor() {
        return new AnnualMovementProcessor();
    }

    // Guarda los movimientos validos en PostgreSQL mediante JDBC
    @Bean
    public JdbcBatchItemWriter<AnnualMovement> annualStatementsWriter(javax.sql.DataSource dataSource) {
        return new JdbcBatchItemWriterBuilder<AnnualMovement>()
            .dataSource(dataSource)
            .sql("""
                INSERT INTO movimientos_anuales (cuenta_id, fecha, transaccion, monto, descripcion)
                VALUES (:accountId, :movementDate, :transaction, :amount, :description)
                ON CONFLICT (cuenta_id, fecha, transaccion, monto, descripcion) DO NOTHING
                """)
            .beanMapped()
            .assertUpdates(assertUpdates: false)
            .build();
    }

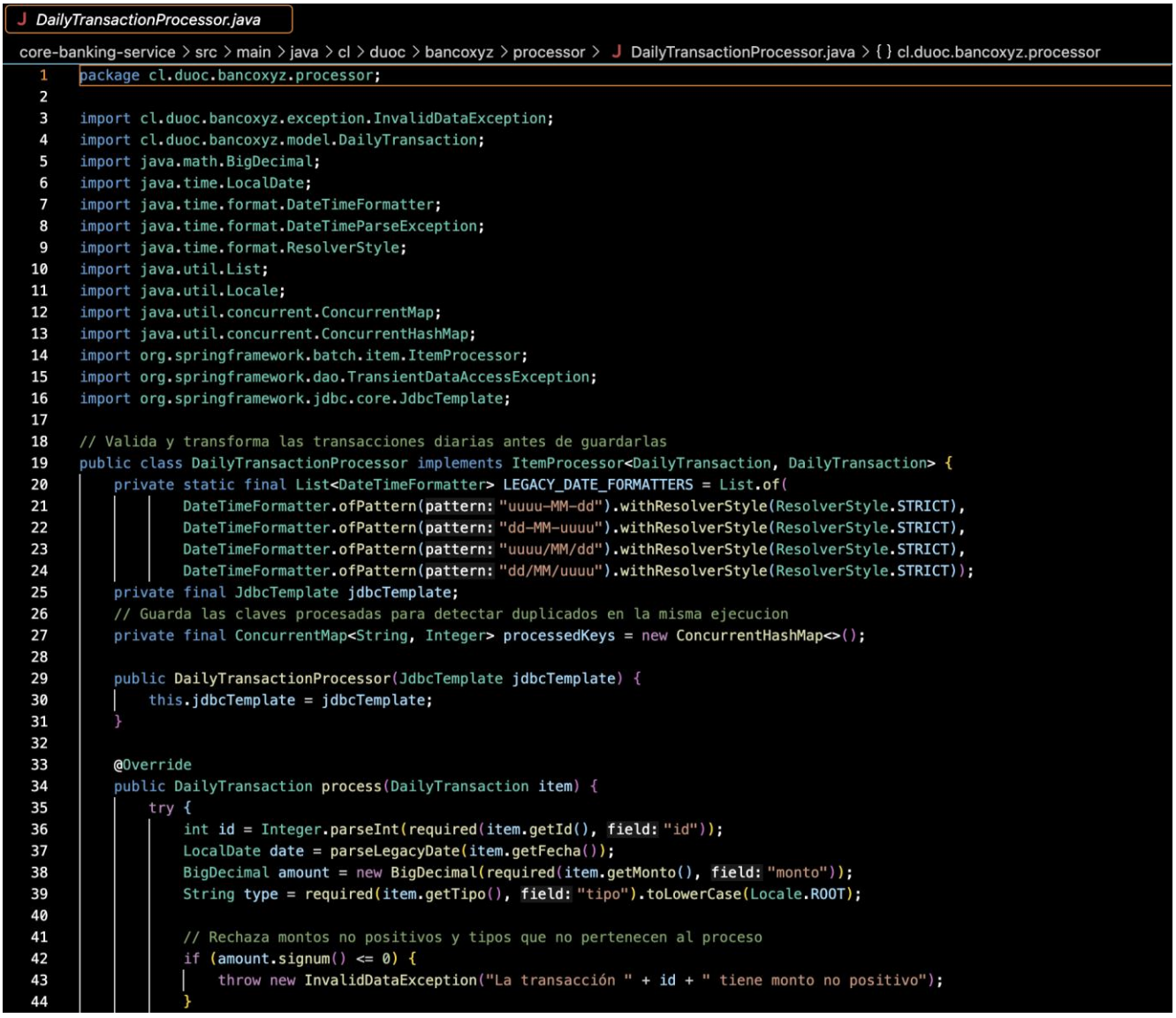
    // Define el flujo Reader - Processor - Writer para los movimientos anuales
    @Bean
    public Step annualMovementsStep(JobRepository jobRepository,
            DataSourceTransactionManager transactionManager,
            SynchronizedItemStreamReader<AnnualMovement> annualStatementsReader,
            AnnualMovementProcessor annualStatementsProcessor,
            JdbcBatchItemWriter<AnnualMovement> annualStatementsWriter,
            ThreadPoolTaskExecutor batchTaskExecutor,
            BankDataSkipPolicy bankDataSkipPolicy,
            BatchSkipListener batchSkipListener,
```

```
            StepPerformanceListener stepPerformanceListener,
            RetryPolicy transientDataAccessRetryPolicy,
            BackOffPolicy batchBackOffPolicy,
            @Value("${bank.batch.chunk-size}") int chunkSize) {
        return new StepBuilder("annualMovementsStep", jobRepository)
                .<AnnualMovement, AnnualMovement>chunk(chunkSize, transactionManager)
                .reader(annualStatementsReader)
                .processor(annualStatementsProcessor)
                .writer(annualStatementsWriter)
                .faultTolerant()
                .skipPolicy(bankDataSkipPolicy)
                .retryPolicy(transientDataAccessRetryPolicy)
                .backOffPolicy(batchBackOffPolicy)
                .listener(batchSkipListener)
                .listener(stepPerformanceListener)
                .taskExecutor(batchTaskExecutor)
                .build();
    }

    // Genera el archivo CSV de auditoria a partir de los movimientos persistidos
    @Bean
    public Tasklet annualStatementFileTasklet(JdbcTemplate jdbcTemplate) {
        return (contribution, chunkContext) -> {
            List<Map<String, Object>> summaries = jdbcTemplate.queryForList("""
                    SELECT cuenta_id,
                           COUNT(*) AS total_movimientos,
                           COALESCE(SUM(CASE WHEN monto > 0 THEN monto ELSE 0 END), 0) AS total_depositos,
                           COALESCE(SUM(CASE WHEN monto < 0 THEN monto ELSE 0 END), 0) AS total_cargos,
                           COALESCE(SUM(monto), 0) AS saldo_neto
                    FROM movimientos_anuales
                    GROUP BY cuenta_id
                    ORDER BY cuenta_id
                    """);
            Path output = Path.of(first: "output", ...more: "estados_cuenta_anuales.csv");
            Files.createDirectories(output.getParent());
            try (var writer = Files.newBufferedWriter(output, StandardCharsets.UTF_8)) {
                writer.write(str: "cuenta_id,total_movimientos,total_depositos,total_cargos,saldo_neto");
                writer.newLine();
                for (Map<String, Object> row : summaries) {
                    writer.write("%s,%s,%s,%s,%s".formatted(
                            row.get(key: "cuenta_id"), row.get(key: "total_movimientos"), row.get(key: "total_depositos"),
                            row.get(key: "total_cargos"), row.get(key: "saldo_neto")));
```

```
128                     writer.newLine();
129                 }
130             } catch (IOException ex) {
131                 throw new IllegalStateException(message: "No fue posible crear el informe anual", ex);
132             }
133             return org.springframework.batch.repeat.RepeatStatus.FINISHED;
134         };
135     }
136
137     // Define el Step que crea el informe anual de salida
138     @Bean
139     public Step annualStatementReportStep(JobRepository jobRepository,
140             DataSourceTransactionManager transactionManager,
141             Tasklet annualStatementFileTasklet) {
142         return new StepBuilder("annualStatementReportStep", jobRepository)
143                 .tasklet(annualStatementFileTasklet, transactionManager)
144                 .build();
145     }
146
147     // Define el Job que procesa y resume los movimientos anuales
148     @Bean
149     public Job annualStatementsJob(JobRepository jobRepository,
150             Step annualMovementsStep,
151             Step annualStatementReportStep,
152             JobSummaryLoggingListener jobSummaryLoggingListener) {
153         return new JobBuilder("annualStatementsJob", jobRepository)
154                 .start(annualMovementsStep)
155                 .next(annualStatementReportStep)
156                 .listener(jobSummaryLoggingListener)
157                 .build();
158     }
159 }
160
```

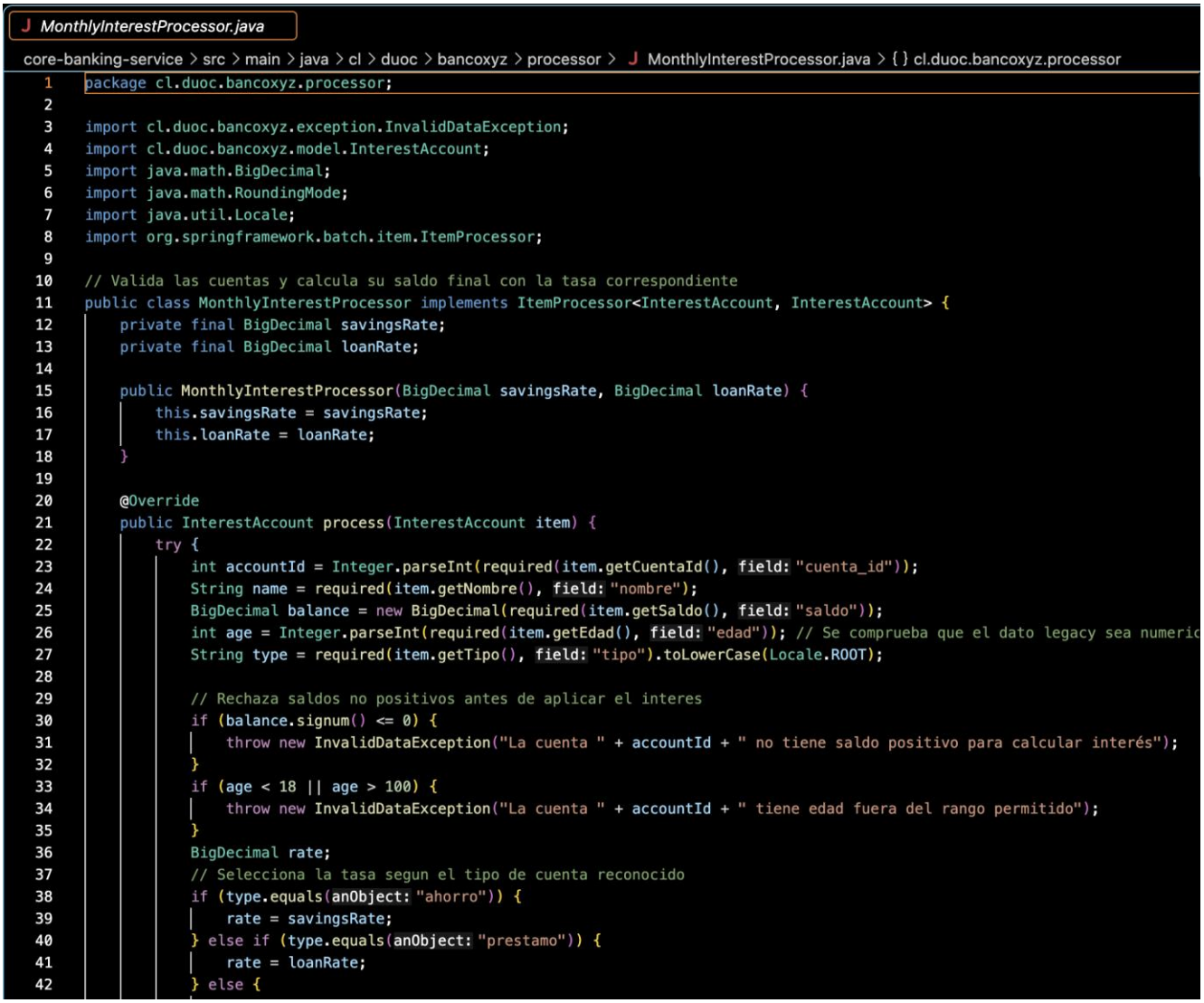
- core-banking-service/src/main/java/cl/duoc/bancoxyz/processor/DailyTransactionProcessor.java



```
45                  if (!type.equals(anObject: "debito") && !type.equals(anObject: "credito")) {
46                      throw new InvalidDataException("La transacción " + id + " tiene tipo no reconocido");
47                  }
48                  String duplicateKey = date + "|" + amount.stripTrailingZeros().toPlainString() + "|" + type;
49                  // Detecta transacciones duplicadas ya persistidas con los mismos datos principales
50                  Integer duplicates = jdbcTemplate.queryForObject("""
51                          SELECT COUNT(*) FROM transacciones_procesadas
52                          WHERE fecha = ? AND monto = ? AND tipo = ? AND id <> ?
53                          """, Integer.class, date, amount, type, id);
54                  if (duplicates != null && duplicates > 0) {
55                      throw new InvalidDataException("La transacción " + id + " está duplicada según fecha, monto y tipo");
56                  }
57                  Integer previousId = processedKeys.putIfAbsent(duplicateKey, id);
58                  if (previousId != null && !previousId.equals(id)) {
59                      throw new InvalidDataException("La transacción " + id + " está duplicada en la misma ejecución");
60                  }
61
62                  item.setTransactionId(id);
63                  item.setTransactionDate(date);
64                  item.setAmount(amount);
65                  item.setTransactionType(type);
66                  return item;
67              } catch (InvalidDataException ex) {
68                  throw ex;
69              // Mantiene el error temporal para que Spring Batch pueda reintentarlo
70              } catch (TransientDataAccessException ex) {
71                  throw ex;
72              } catch (Exception ex) {
73                  throw new InvalidDataException("Transacción con formato inválido: " + ex.getMessage());
74              }
75          }
76
77          private String required(String value, String field) {
78              if (value == null || value.isBlank()) {
79                  throw new InvalidDataException("Campo requerido vacío: " + field);
80              }
81              return value.trim();
82          }
83
```

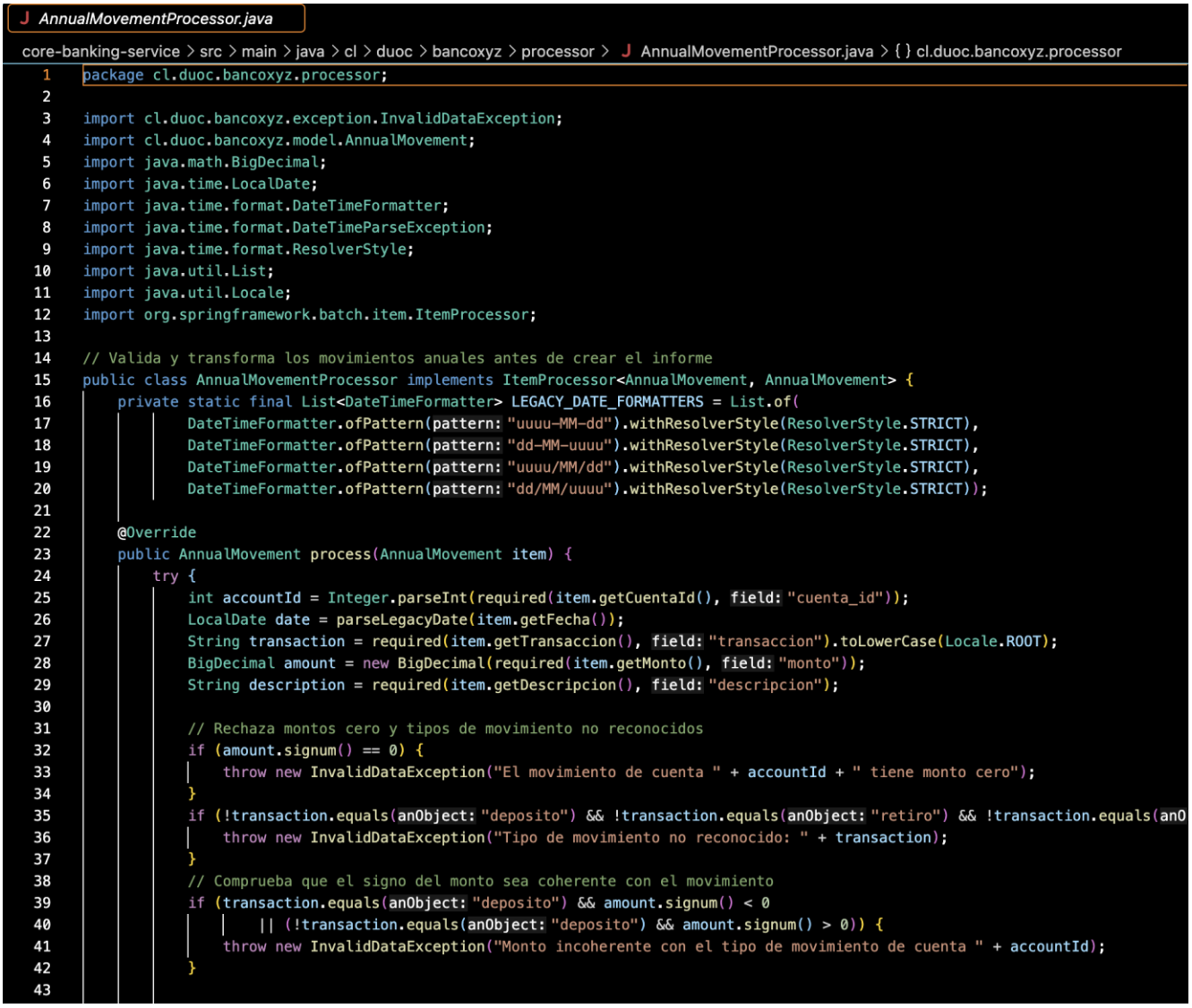
```
84          private LocalDate parseLegacyDate(String value) {
85              String dateValue = required(value, field: "fecha");
86              for (DateTimeFormatter formatter : LEGACY_DATE_FORMATTERS) {
87                  try {
88                      return LocalDate.parse(dateValue, formatter);
89                  } catch (DateTimeParseException ignored) {
90                      // Prueba el siguiente formato legacy permitido.
91                  }
92              }
93              throw new InvalidDataException("Fecha legacy invalida: " + dateValue);
94          }
95      }
96
```

- core-banking-service/src/main/java/cl/duoc/bancoxyz/processor/MonthlyInterestProcessor.java



```
43                     throw new InvalidDataException("La cuenta " + accountId + " tiene tipo no procesable: " + type);
44                 }
45
46                 item.setAccountId(accountId);
47                 item.setAccountName(name);
48                 item.setInitialBalance(balance);
49                 item.setAccountType(type);
50                 item.setRate(rate);
51                 // Calcula el saldo final que sera persistido por el escritor
52                 item.setFinalBalance(balance.multiply(BigDecimal.ONE.add(rate)).setScale(newScale: 2, RoundingMode.HALF_UP));
53                 return item;
54             } catch (InvalidDataException ex) {
55                 throw ex;
56             } catch (Exception ex) {
57                 throw new InvalidDataException("Cuenta con formato inválido: " + ex.getMessage());
58             }
59         }
60
61         private String required(String value, String field) {
62             if (value == null || value.isBlank()) {
63                 throw new InvalidDataException("Campo requerido vacío: " + field);
64             }
65             return value.trim();
66         }
67     }
68
```

- core-banking-service/src/main/java/cl/duoc/bancoxyz/processor/AnnualMovementProcessor.java



AnnualMovementProcessor.java

core-banking-service > src > main > java > cl > duoc > bancoxyz > processor > AnnualMovementProcessor.java > { } cl.duoc.bancoxyz.processor

```java
package cl.duoc.bancoxyz.processor;

import cl.duoc.bancoxyz.exception.InvalidDataException;
import cl.duoc.bancoxyz.model.AnnualMovement;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;
import java.util.List;
import java.util.Locale;
import org.springframework.batch.item.ItemProcessor;

// Valida y transforma los movimientos anuales antes de crear el informe
public class AnnualMovementProcessor implements ItemProcessor<AnnualMovement, AnnualMovement> {
    private static final List<DateTimeFormatter> LEGACY_DATE_FORMATTERS = List.of(
            DateTimeFormatter.ofPattern(pattern: "uuuu-MM-dd").withResolverStyle(ResolverStyle.STRICT),
            DateTimeFormatter.ofPattern(pattern: "dd-MM-uuuu").withResolverStyle(ResolverStyle.STRICT),
            DateTimeFormatter.ofPattern(pattern: "uuuu/MM/dd").withResolverStyle(ResolverStyle.STRICT),
            DateTimeFormatter.ofPattern(pattern: "dd/MM/uuuu").withResolverStyle(ResolverStyle.STRICT));

    @Override
    public AnnualMovement process(AnnualMovement item) {
        try {
            int accountId = Integer.parseInt(required(item.getCuentaId(), field: "cuenta_id"));
            LocalDate date = parseLegacyDate(item.getFecha());
            String transaction = required(item.getTransaccion(), field: "transaccion").toLowerCase(Locale.ROOT);
            BigDecimal amount = new BigDecimal(required(item.getMonto(), field: "monto"));
            String description = required(item.getDescripcion(), field: "descripcion");

            // Rechaza montos cero y tipos de movimiento no reconocidos
            if (amount.signum() == 0) {
                throw new InvalidDataException("El movimiento de cuenta " + accountId + " tiene monto cero");
            }
            if (!transaction.equals(anObject: "deposito") && !transaction.equals(anObject: "retiro") && !transaction.equals(anO
                throw new InvalidDataException("Tipo de movimiento no reconocido: " + transaction);
            }
            // Comprueba que el signo del monto sea coherente con el movimiento
            if (transaction.equals(anObject: "deposito") && amount.signum() < 0
                    || (!transaction.equals(anObject: "deposito") && amount.signum() > 0)) {
                throw new InvalidDataException("Monto incoherente con el tipo de movimiento de cuenta " + accountId);
            }

```

```
            item.setAccountId(accountId);
            item.setMovementDate(date);
            item.setTransaction(transaction);
            item.setAmount(amount);
            item.setDescription(description);
            return item;
        } catch (InvalidDataException ex) {
            throw ex;
        } catch (Exception ex) {
            throw new InvalidDataException("Movimiento anual con formato inválido: " + ex.getMessage());
        }
    }

    private String required(String value, String field) {
        if (value == null || value.isBlank()) {
            throw new InvalidDataException("Campo requerido vacío: " + field);
        }
        return value.trim();
    }

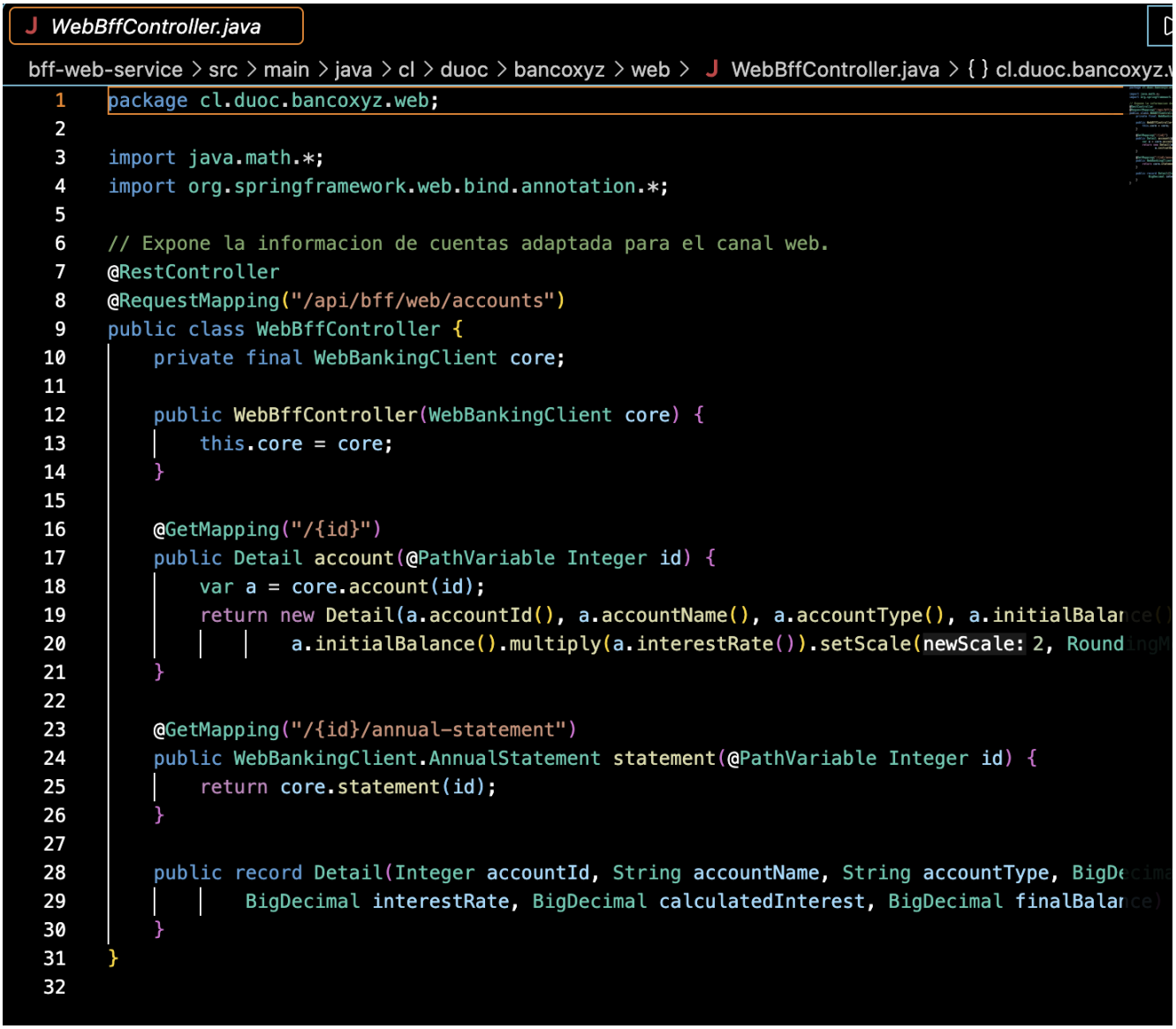
    private LocalDate parseLegacyDate(String value) {
        String dateValue = required(value, field: "fecha");
        for (DateTimeFormatter formatter : LEGACY_DATE_FORMATTERS) {
            try {
                return LocalDate.parse(dateValue, formatter);
            } catch (DateTimeParseException ignored) {
                // Prueba el siguiente formato legacy permitido.
            }
        }
        throw new InvalidDataException("Fecha legacy invalida: " + dateValue);
    }
}

```

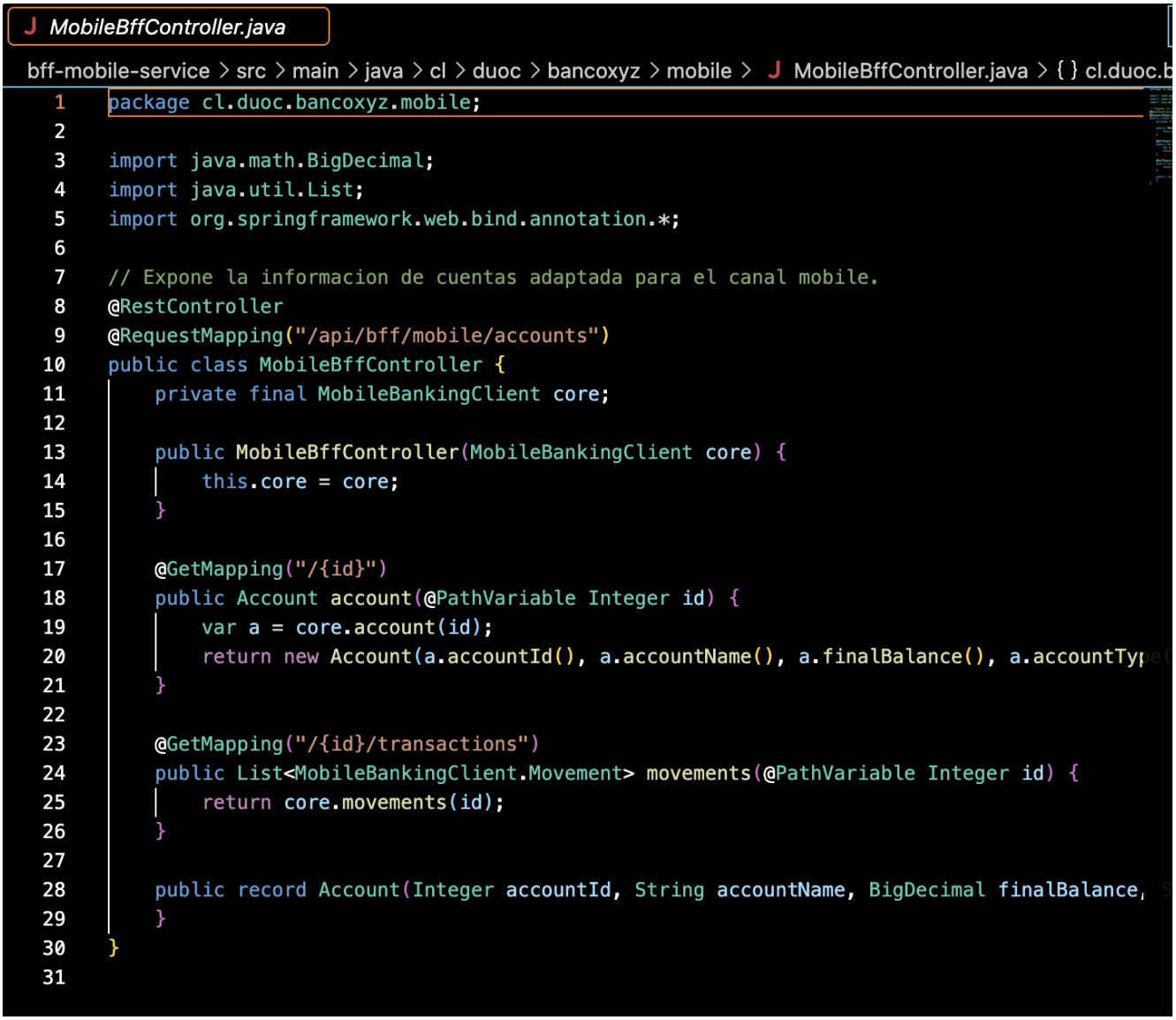
- **Evidencia de BFF**

Los archivos clases del Backend for Frontend (BFF) son los siguientes:

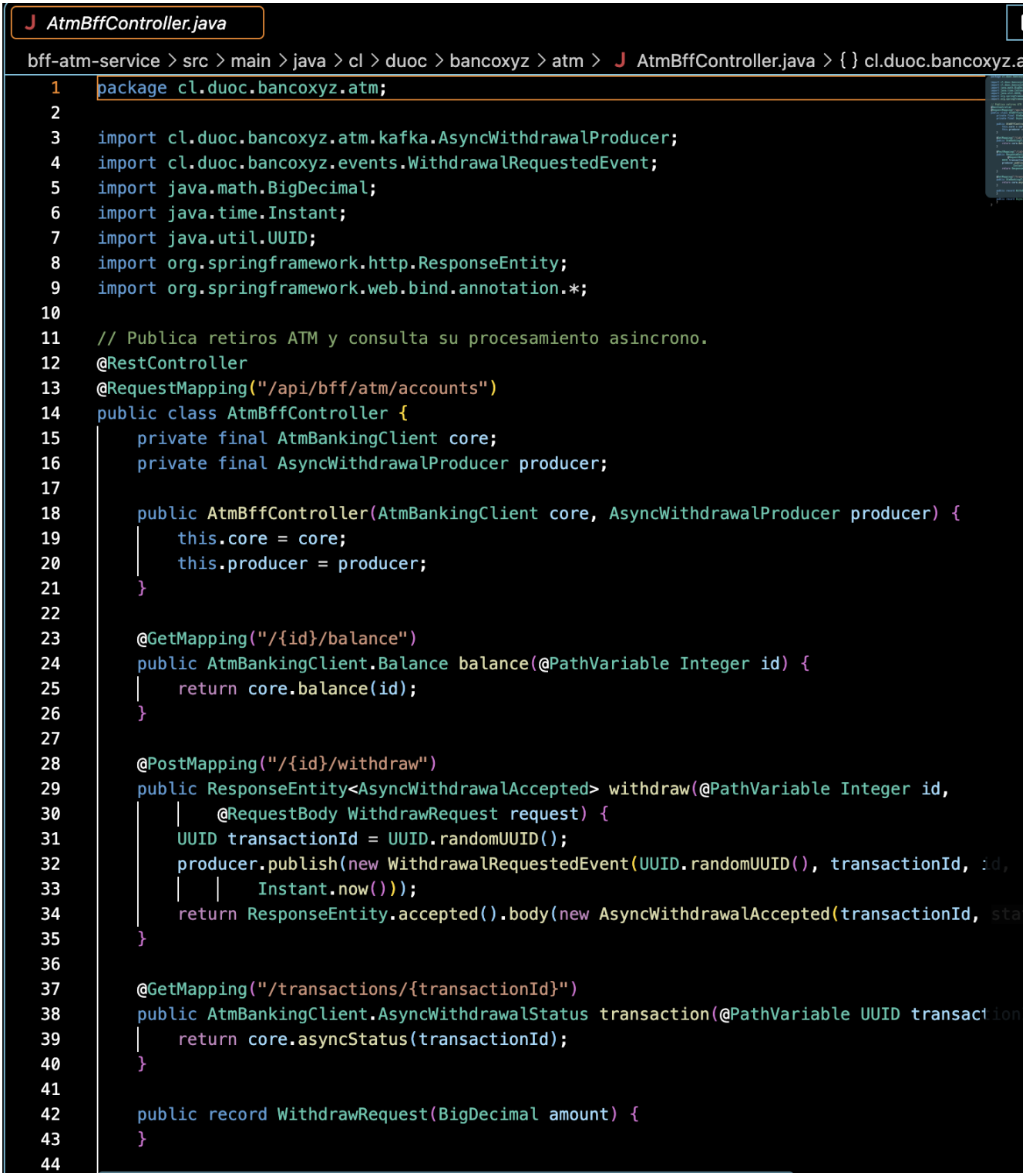
- bff-web-service/src/main/java/cl/duoc/bancoxyz/web/WebBffController.java



- bff-mobile-service/src/main/java/cl/duoc/bancoxyz/mobile/MobileBffController.java



- bff-atm-service/src/main/java/cl/duoc/bancoxyz/atm/AtmBffController.java

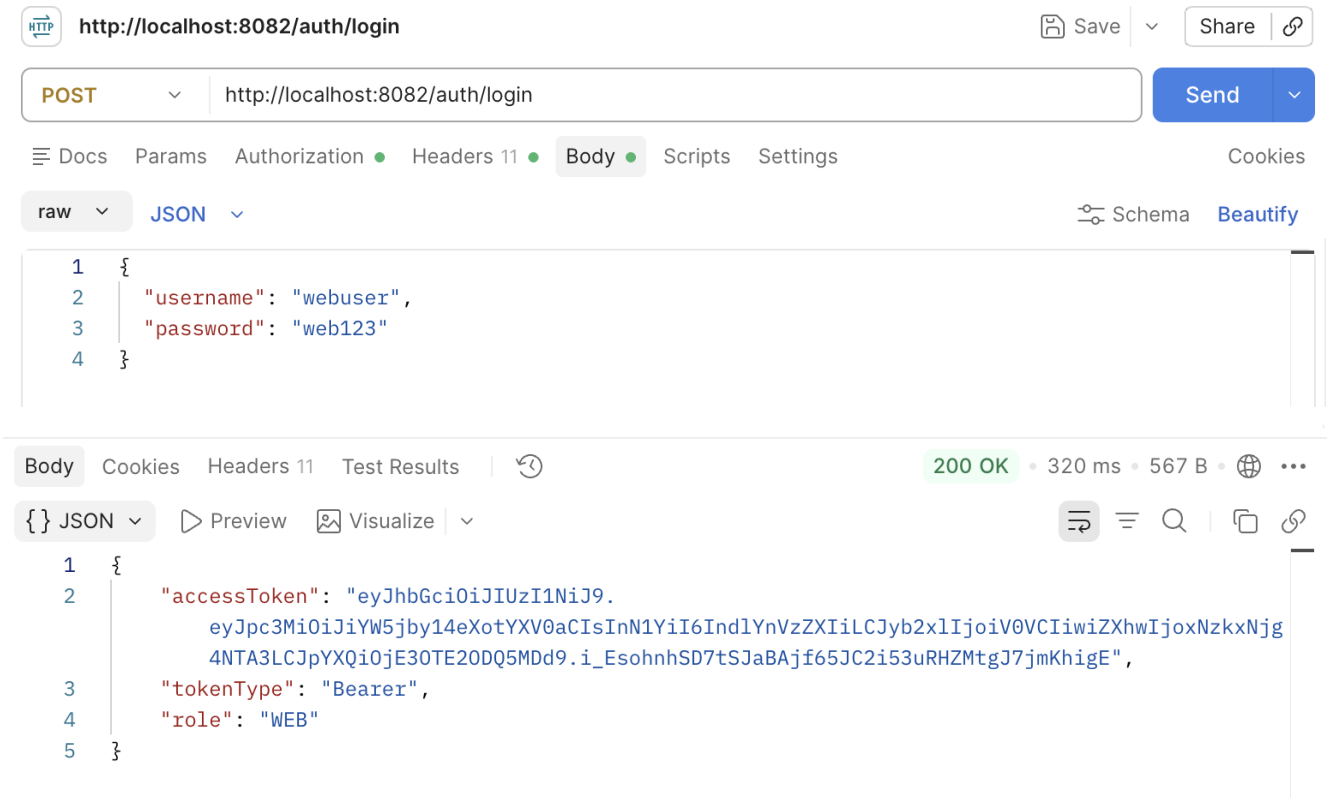


```
45      public record AsyncWithdrawalAccepted(UUID transactionId, String status) {
46      }
47  }
48
```

Ahora para mostrar el funcionamiento, nos vamos a la POSTMAN para mostrar el Login de Web, para ello utilizaremos el método POST y escribiremos la siguiente URL http://localhost:8082/auth/login, después seleccionaremos el Body, con RAW y escibiremos el método en JSON, siendo el código en JSON el siguiente:

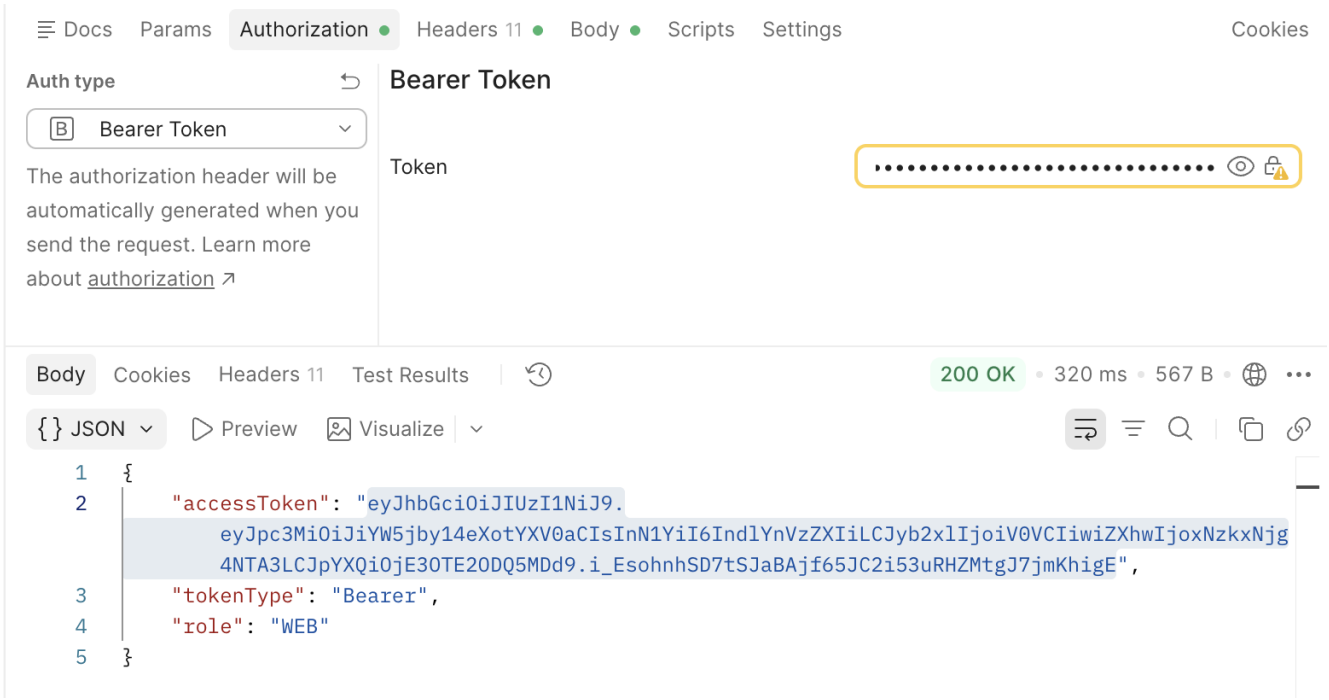
```
{
  "username": "webuser",
  "password": "web123"
}
```

Esto nos deberá devolver un accessToken para el rol de web.

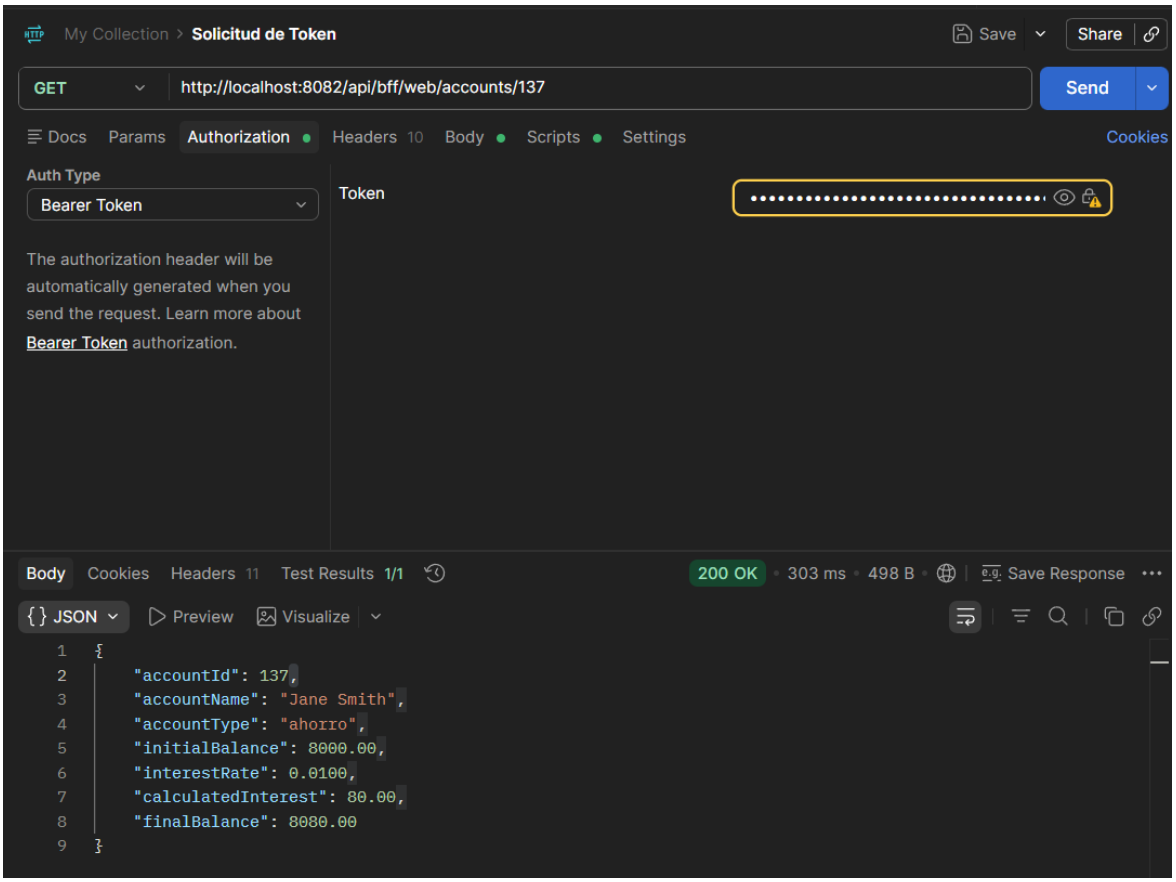


Ahora bien, vamos a hacer el BFF Web para la entrega de datos completos, para ello vamos a seleccionar el método GET y escribiremos la siguiente URL http://localhost:8082/api/bff/web/accounts/137 , pero antes vamos a copiar el token de

acceso que se creó y después nos vamos donde dice Authorization, seleccionamos Barer Token y pegamos el token



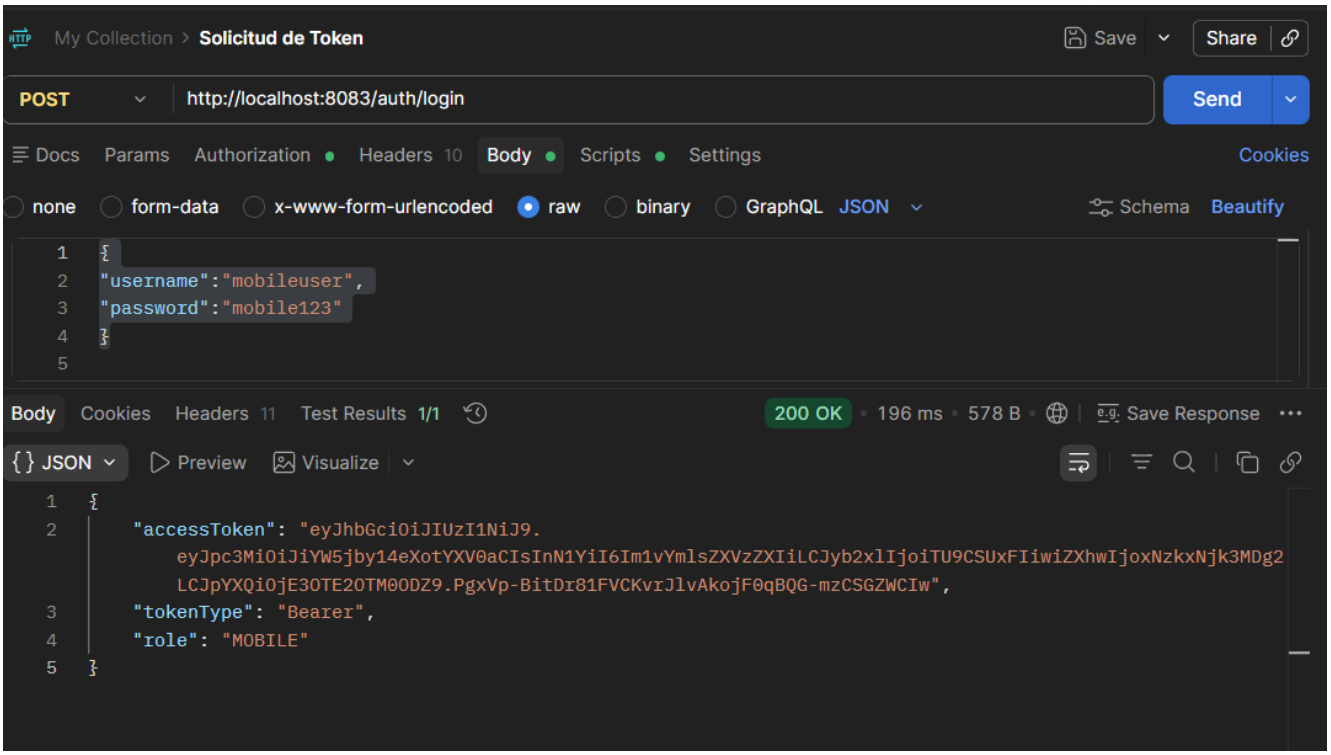
Si le damos al botón Send, podremos ver que nos devuelve el resultado 200 OK



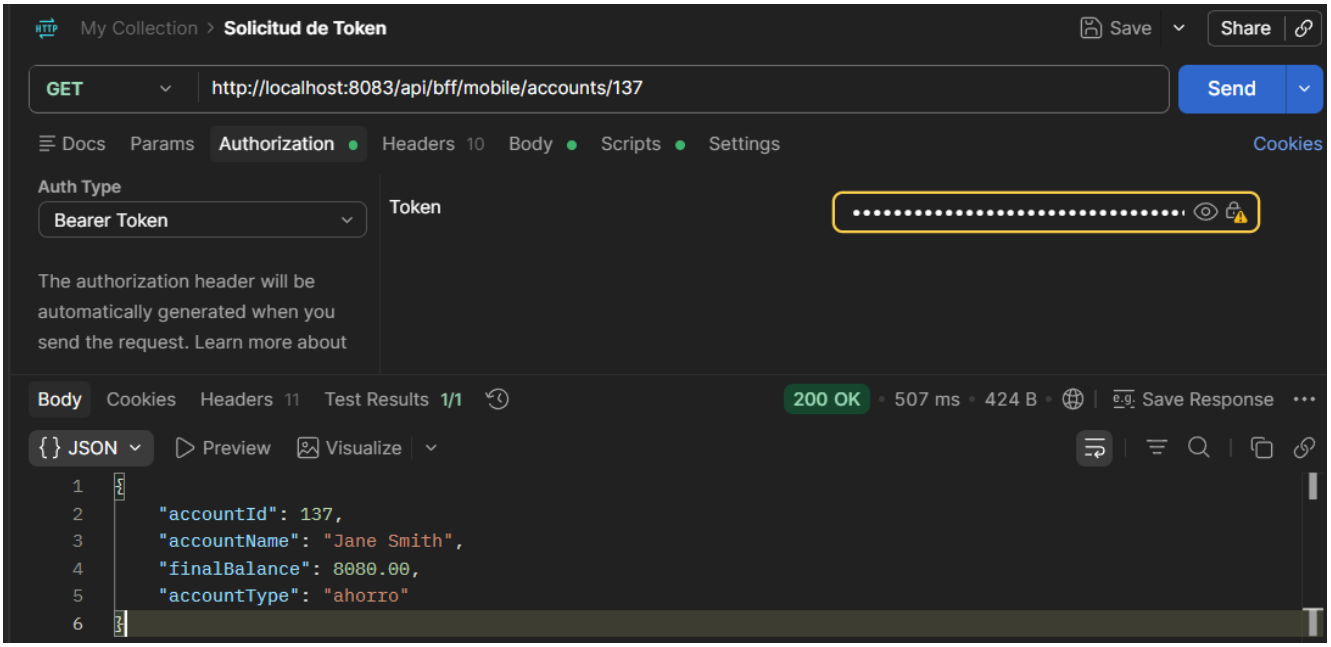
Ahora para mostrar el funcionamiento de móvil, debemos de volver a crear un token de acceso, para ello en POSTMAN debemos de volver a colocar el método POST y escribiremos la siguiente URL http://localhost:8083/auth/login, después seleccionaremos el Body, con RAW y escibiremos el método en JSON, siendo el código en JSON el siguiente:

{
"username":"mobileuser",
"password":"mobile123"
}

Esto nos deberá devolver un accessToken para el rol de web.



Ahora bien, vamos a hacer el BFF Mobil para la entrega de datos completos, para ello vamos a seleccionar el método GET y escribiremos la siguiente URL http://localhost:8083/api/bff/mobile/accounts/137, pero antes vamos a copiar el token de acceso que se creó y después nos vamos donde dice Authorization, seleccionamos Barer Token y pegamos el token. Si le damos al botón Send, podremos ver que nos devuelve el resultado 200 OK



Ahora para mostrar el funcionamiento del cajero o atm, debemos de volver a crear un token de acceso, para ello en POSTMAN debemos de volver a colocar el método POST y escribiremos la siguiente URL http://localhost:8084/auth/login, después seleccionaremos el Body, con RAW y escibiremos el método en JSON, siendo el código en JSON el siguiente:

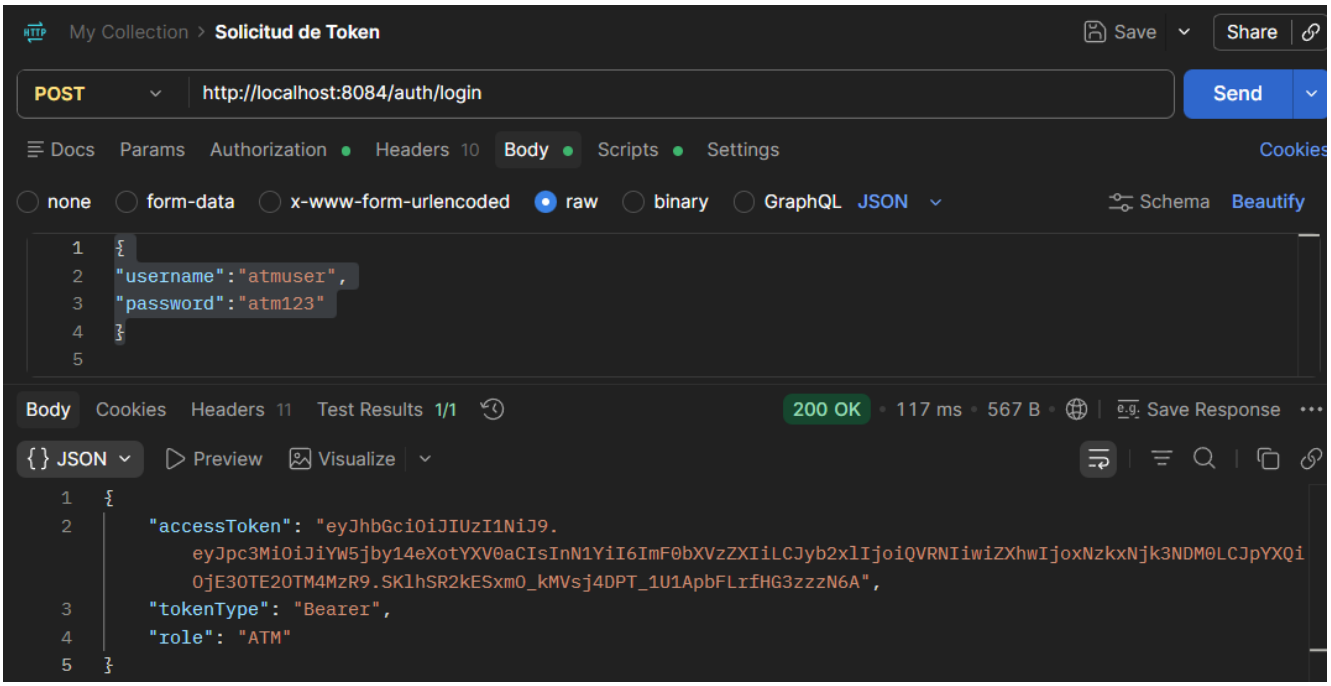
{

"username":"atmuser",

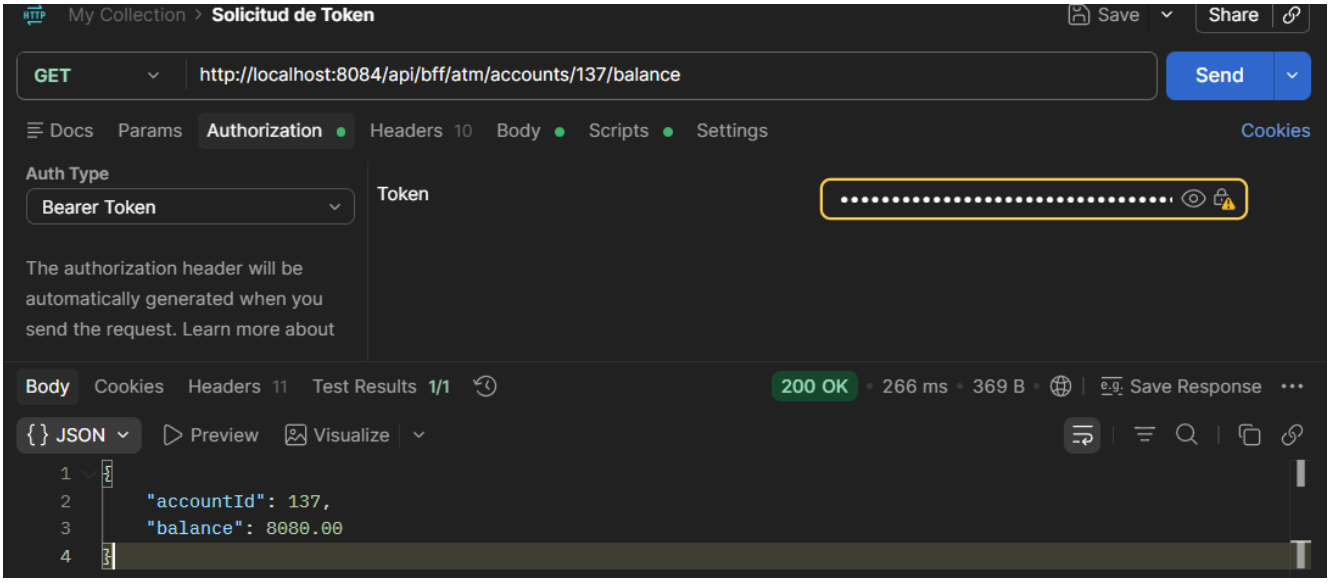
"password":"atm123"

}

Esto nos deberá devolver un accessToken para el rol de web.



Ahora bien, vamos a hacer el BFF ATM para la entrega de datos completos, para ello vamos a seleccionar el método GET y escribiremos la siguiente URL http://localhost:8084/api/bff/atm/accounts/137/balance, pero antes vamos a copiar el token de acceso que se creó y después nos vamos donde dice Authorization, seleccionamos Barer Token y pegamos el token. Si le damos al botón Send, podremos ver que nos devuelve el resultado 200 OK



- Evidencia de Seguridad OAuth2/JWT

Para la seguridad, se utilizó OAuth2/JWT, para ello cree las siguientes clases clave:

- bff-web-service/src/main/java/cl/duoc/bancoxyz/config/SecurityConfig.java

SecurityConfig.java

bff-web-service > src > main > java > cl > duoc > bancoxyz > config > SecurityConfig.java > { } cl.duoc.bancoxyz.config

```java
package cl.duoc.bancoxyz.config;

import jakarta.servlet.DispatcherType;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.nimbusds.jose.jwk.source.ImmutableSecret;
import java.util.Base64;
import java.util.Map;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.security.authentication.*;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;
import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationConverter;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.*;

// Protege los recursos del canal web como OAuth2 Resource Server.
@Configuration
public class SecurityConfig {
    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    UserDetailsService users(PasswordEncoder encoder, @Value("${security.users.web.password}") String web,
            @Value("${security.users.mobile.password}") String mobile,
            @Value("${security.users.atm.password}") String atm) {
        return new InMemoryUserDetailsManager(
                User.withUsername("webuser").password(encoder.encode(web)).roles("WEB").build(),
                User.withUsername("mobileuser").password(encoder.encode(mobile)).roles("MOBILE").build(),
                User.withUsername("atmuser").password(encoder.encode(atm)).roles("ATM").build());
    }

```

```java
    @Bean
    AuthenticationManager authenticationManager(UserDetailsService users, PasswordEncoder encoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(users);
        provider.setPasswordEncoder(encoder);
        return provider::authenticate;
    }

    @Bean
    SecretKey jwtSecretKey(@Value("${security.oauth2.jwt.secret}") String secret) {
        return new SecretKeySpec(Base64.getDecoder().decode(secret), "HmacSHA256");
    }

    @Bean
    JwtEncoder jwtEncoder(SecretKey jwtSecretKey) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecretKey));
    }

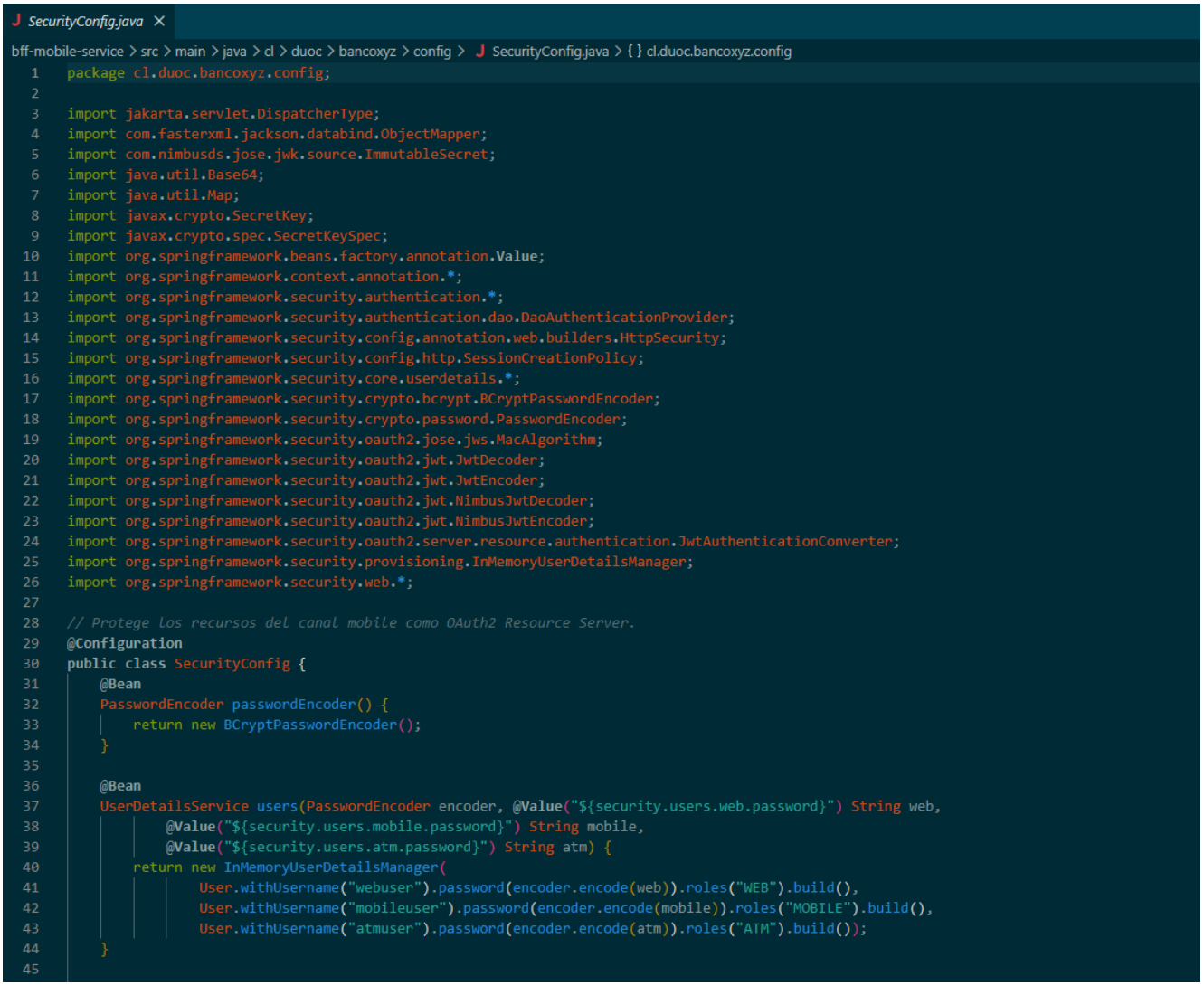
    @Bean
    JwtDecoder jwtDecoder(SecretKey jwtSecretKey) {
        return NimbusJwtDecoder.withSecretKey(jwtSecretKey).macAlgorithm(MacAlgorithm.HS256).build();
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            String role = jwt.getClaimAsString("role");
            if (role == null || role.isBlank()) {
                return java.util.List.of();
            }
            return java.util.List.of(new org.springframework.security.core.authority.SimpleGrantedAuthority("ROLE_" + role));
        });
        return converter;
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationConverter jwtAuthenticationConverter,
            @Value("${security.channel-role}") String role) throws Exception {
        return http.csrf(c -> c.disable())
                .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(a -> a.dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
                        .requestMatchers("/auth/login", "/actuator/health").permitAll().anyRequest().hasRole(role))
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter)))
                .exceptionHandling(e -> e
                        .authenticationEntryPoint(
                                (q, r, x) -> json(r, status: 401, error: "UNAUTHORIZED", message: "Token OAuth2 Bearer requerido o invalido"))
                        .accessDeniedHandler((q, r, x) -> json(r, status: 403, error: "FORBIDDEN",
                                message: "Usuario autenticado sin permisos para este canal")))
                .build();
    }

    private static void json(jakarta.servlet.http.HttpServletResponse r, int status, String error, String message)
            throws java.io.IOException {
        r.setStatus(status);
        r.setContentType("application/json");
        new ObjectMapper().writeValue(r.getOutputStream(),
                Map.of(k1: "status", status, k2: "error", error, k3: "message", message));
    }
}
```

- bff-mobile-service/src/main/java/cl/duoc/bancoxyz/config/SecurityConfig.java



```java
    @Bean
    AuthenticationManager authenticationManager(UserDetailsService users, PasswordEncoder encoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(users);
        provider.setPasswordEncoder(encoder);
        return provider::authenticate;
    }

    @Bean
    SecretKey jwtSecretKey(@Value("${security.oauth2.jwt.secret}") String secret) {
        return new SecretKeySpec(Base64.getDecoder().decode(secret), "HmacSHA256");
    }

    @Bean
    JwtEncoder jwtEncoder(SecretKey jwtSecretKey) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecretKey));
    }

    @Bean
    JwtDecoder jwtDecoder(SecretKey jwtSecretKey) {
        return NimbusJwtDecoder.withSecretKey(jwtSecretKey).macAlgorithm(MacAlgorithm.HS256).build();
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            String role = jwt.getClaimAsString("role");
            if (role == null || role.isBlank()) {
                return java.util.List.of();
            }
            return java.util.List.of(new org.springframework.security.core.authority.SimpleGrantedAuthority("ROLE_" + role));
        });
        return converter;
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationConverter jwtAuthenticationConverter,
            @Value("${security.channel-role}") String role) throws Exception {
        return http.csrf(c -> c.disable())
                .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(a -> a.dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
                        .requestMatchers("/auth/login", "/actuator/health").permitAll().anyRequest().hasRole(role))
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter)))
                .exceptionHandling(e -> e
                        .authenticationEntryPoint(
                                (q, r, x) -> json(r, status: 401, error: "UNAUTHORIZED", message: "Token OAuth2 Bearer requerido o invalido"))
                        .accessDeniedHandler((q, r, x) -> json(r, status: 403, error: "FORBIDDEN",
                                message: "Usuario autenticado sin permisos para este canal")))
                .build();
    }

    private static void json(jakarta.servlet.http.HttpServletResponse r, int status, String error, String message)
            throws java.io.IOException {
        r.setStatus(status);
        r.setContentType("application/json");
        new ObjectMapper().writeValue(r.getOutputStream(),
                Map.of(k1: "status", status, k2: "error", error, k3: "message", message));
    }
}
```

- bff-atm-service/src/main/java/cl/duoc/bancoxyz/config/SecurityConfig.java

SecurityConfig.java

bff-web-service > src > main > java > cl > duoc > bancoxyz > config > SecurityConfig.java > { } cl.duoc.bancoxyz.config

```java
package cl.duoc.bancoxyz.config;

import jakarta.servlet.DispatcherType;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.nimbusds.jose.jwk.source.ImmutableSecret;
import java.util.Base64;
import java.util.Map;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.security.authentication.*;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;
import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationConverter;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.*;

// Protege los recursos del canal web como OAuth2 Resource Server.
@Configuration
public class SecurityConfig {
    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    UserDetailsService users(PasswordEncoder encoder, @Value("${security.users.web.password}") String web,
            @Value("${security.users.mobile.password}") String mobile,
            @Value("${security.users.atm.password}") String atm) {
        return new InMemoryUserDetailsManager(
                User.withUsername("webuser").password(encoder.encode(web)).roles("WEB").build(),
                User.withUsername("mobileuser").password(encoder.encode(mobile)).roles("MOBILE").build(),
                User.withUsername("atmuser").password(encoder.encode(atm)).roles("ATM").build());
    }

```

```java
    @Bean
    AuthenticationManager authenticationManager(UserDetailsService users, PasswordEncoder encoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(users);
        provider.setPasswordEncoder(encoder);
        return provider::authenticate;
    }

    @Bean
    SecretKey jwtSecretKey(@Value("${security.oauth2.jwt.secret}") String secret) {
        return new SecretKeySpec(Base64.getDecoder().decode(secret), "HmacSHA256");
    }

    @Bean
    JwtEncoder jwtEncoder(SecretKey jwtSecretKey) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecretKey));
    }

    @Bean
    JwtDecoder jwtDecoder(SecretKey jwtSecretKey) {
        return NimbusJwtDecoder.withSecretKey(jwtSecretKey).macAlgorithm(MacAlgorithm.HS256).build();
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            String role = jwt.getClaimAsString("role");
            if (role == null || role.isBlank()) {
                return java.util.List.of();
            }
            return java.util.List.of(new org.springframework.security.core.authority.SimpleGrantedAuthority("ROLE_" + role));
        });
        return converter;
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http, JwtAuthenticationConverter jwtAuthenticationConverter,
            @Value("${security.channel-role}") String role) throws Exception {
        return http.csrf(c -> c.disable())
                .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .authorizeHttpRequests(a -> a.dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
                        .requestMatchers("/auth/login", "/actuator/health").permitAll().anyRequest().hasRole(role))
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter)))
                .exceptionHandling(e -> e
                        .authenticationEntryPoint(
                                (q, r, x) -> json(r, status: 401, error: "UNAUTHORIZED", message: "Token OAuth2 Bearer requerido o invalido"))
                        .accessDeniedHandler((q, r, x) -> json(r, status: 403, error: "FORBIDDEN",
                                message: "Usuario autenticado sin permisos para este canal")))
                .build();
    }

    private static void json(jakarta.servlet.http.HttpServletResponse r, int status, String error, String message)
            throws java.io.IOException {
        r.setStatus(status);
        r.setContentType("application/json");
        new ObjectMapper().writeValue(r.getOutputStream(),
                Map.of(k1: "status", status, k2: "error", error, k3: "message", message));
    }
}
```

- account-service/src/main/java/cl/duoc/bancoxyz/config/SecurityConfig.java

SecurityConfig.java

account-service > src > main > java > cl > duoc > bancoxyz > config > SecurityConfig.java > { } cl.duoc.bancoxyz.config

```java
package cl.duoc.bancoxyz.config;

import java.util.Base64;
import javax.crypto.spec.SecretKeySpec;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/actuator/health").permitAll()
                        .anyRequest().authenticated())
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {
                }))
                .build();
    }

    @Bean
    JwtDecoder jwtDecoder(@Value("${security.oauth2.jwt.secret}") String secret) {
        return NimbusJwtDecoder.withSecretKey(new SecretKeySpec(Base64.getDecoder().decode(secret), "HmacSHA256"))
                .macAlgorithm(MacAlgorithm.HS256)
                .build();
    }
}

```

- payment-service/src/main/java/cl/duoc/bancoxyz/config/SecurityConfig.java

SecurityConfig.java

payment-service > src > main > java > cl > duoc > bancoxyz > config > SecurityConfig.java > { } cl.duoc.bancoxyz.config

```java
package cl.duoc.bancoxyz.config;

import java.util.Base64;
import javax.crypto.spec.SecretKeySpec;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/actuator/health").permitAll()
                        .anyRequest().authenticated())
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {
                }))
                .build();
    }

    @Bean
    JwtDecoder jwtDecoder(@Value("${security.oauth2.jwt.secret}") String secret) {
        return NimbusJwtDecoder.withSecretKey(new SecretKeySpec(Base64.getDecoder().decode(secret), "HmacSHA256"))
                .macAlgorithm(MacAlgorithm.HS256)
                .build();
    }
}

```

- customer-service/src/main/java/cl/duoc/bancoxyz/config/SecurityConfig.java

SecurityConfig.java

customer-service > src > main > java > cl > duoc > bancoxyz > config > SecurityConfig.java > { } cl.duoc.bancoxyz.config

```java
package cl.duoc.bancoxyz.config;

import java.util.Base64;
import javax.crypto.spec.SecretKeySpec;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.web.SecurityFilterChain;

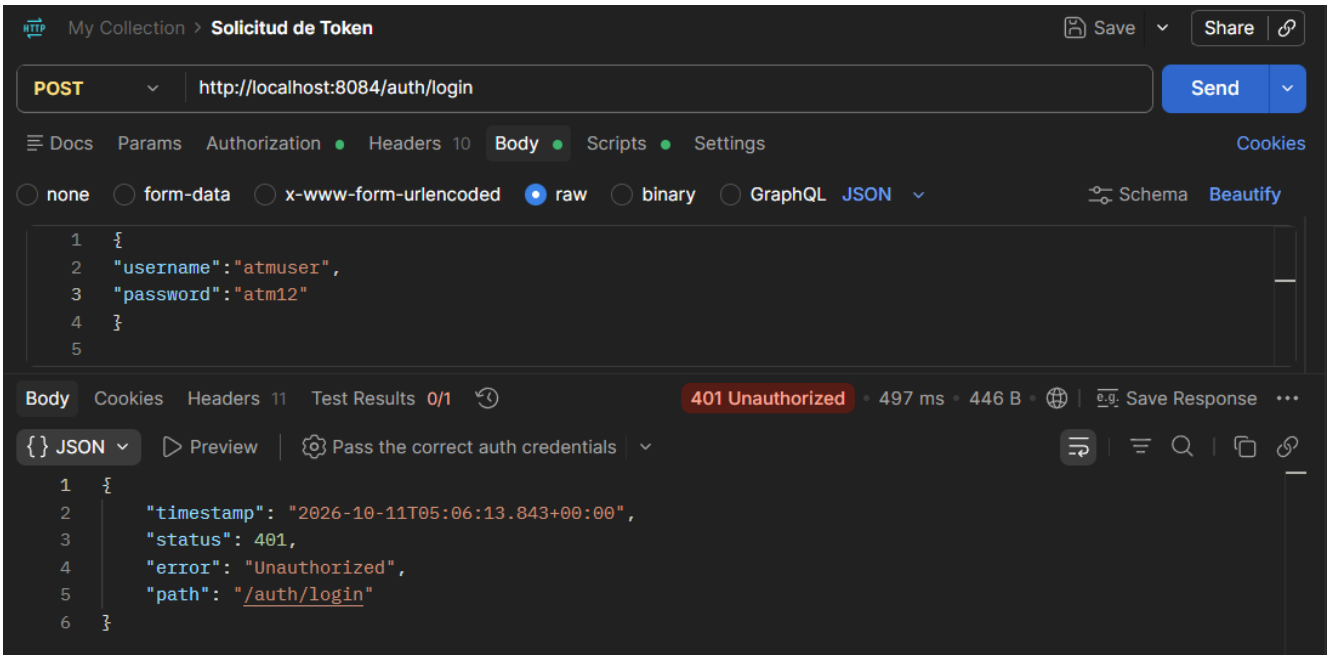
@Configuration
public class SecurityConfig {

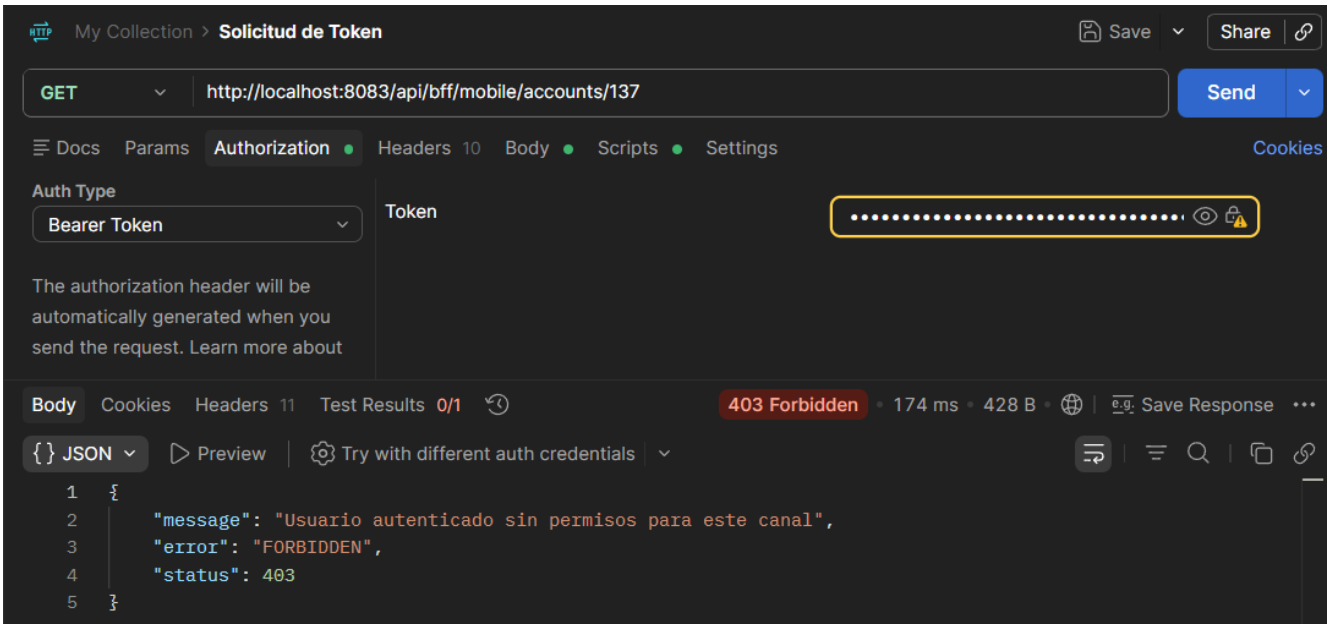
    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/actuator/health").permitAll()
                        .anyRequest().authenticated())
                .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {
                }))
                .build();
    }

    @Bean
    JwtDecoder jwtDecoder(@Value("${security.oauth2.jwt.secret}") String secret) {
        return NimbusJwtDecoder.withSecretKey(new SecretKeySpec(Base64.getDecoder().decode(secret), "HmacSHA256"))
                .macAlgorithm(MacAlgorithm.HS256)
                .build();
    }
}

```

Las clases anteriores nos permitirán poder utilizar la ruta de Login con /auth/login, esto nos permitirá retornar errores 401 si se intenta de ingresar sin un token o un 403 si es que quiero ingresar a otro canal

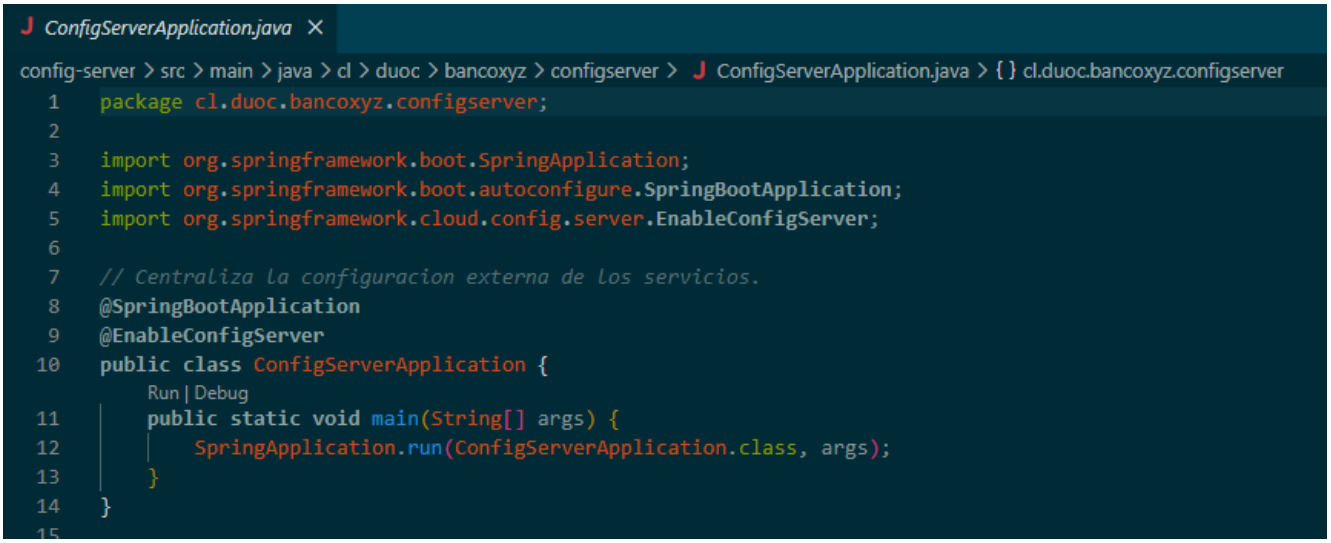

My Collection > Solicitud de Token
Save
Share
POST
http://localhost:8084/auth/login
Send
Docs Params Authorization Headers 10 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"username":"atmuser",
"password":"atm12"
}
Body Cookies Headers 11 Test Results 0/1
401 Unauthorized 497 ms 446 B
Save Response
JSON Preview Pass the correct auth credentials
{
"timestamp": "2026-10-11T05:06:13.843+00:00",
"status": 401,
"error": "Unauthorized",
"path": "/auth/login"
}


My Collection > Solicitud de Token
Save
Share
GET
http://localhost:8083/api/bff/mobile/accounts/137
Send
Docs Params Authorization Headers 10 Body Scripts Settings
Cookies
Auth Type
Bearer Token
Token
The authorization header will be automatically generated when you send the request. Learn more about
Body Cookies Headers 11 Test Results 0/1
403 Forbidden 174 ms 428 B
Save Response
JSON Preview Try with different auth credentials
{
"message": "Usuario autenticado sin permisos para este canal",
"error": "FORBIDDEN",
"status": 403
}

- Evidencia de Spring Cloud Config

Para la utilización de Spring Cloud se creó la clase ConfigServerApplication

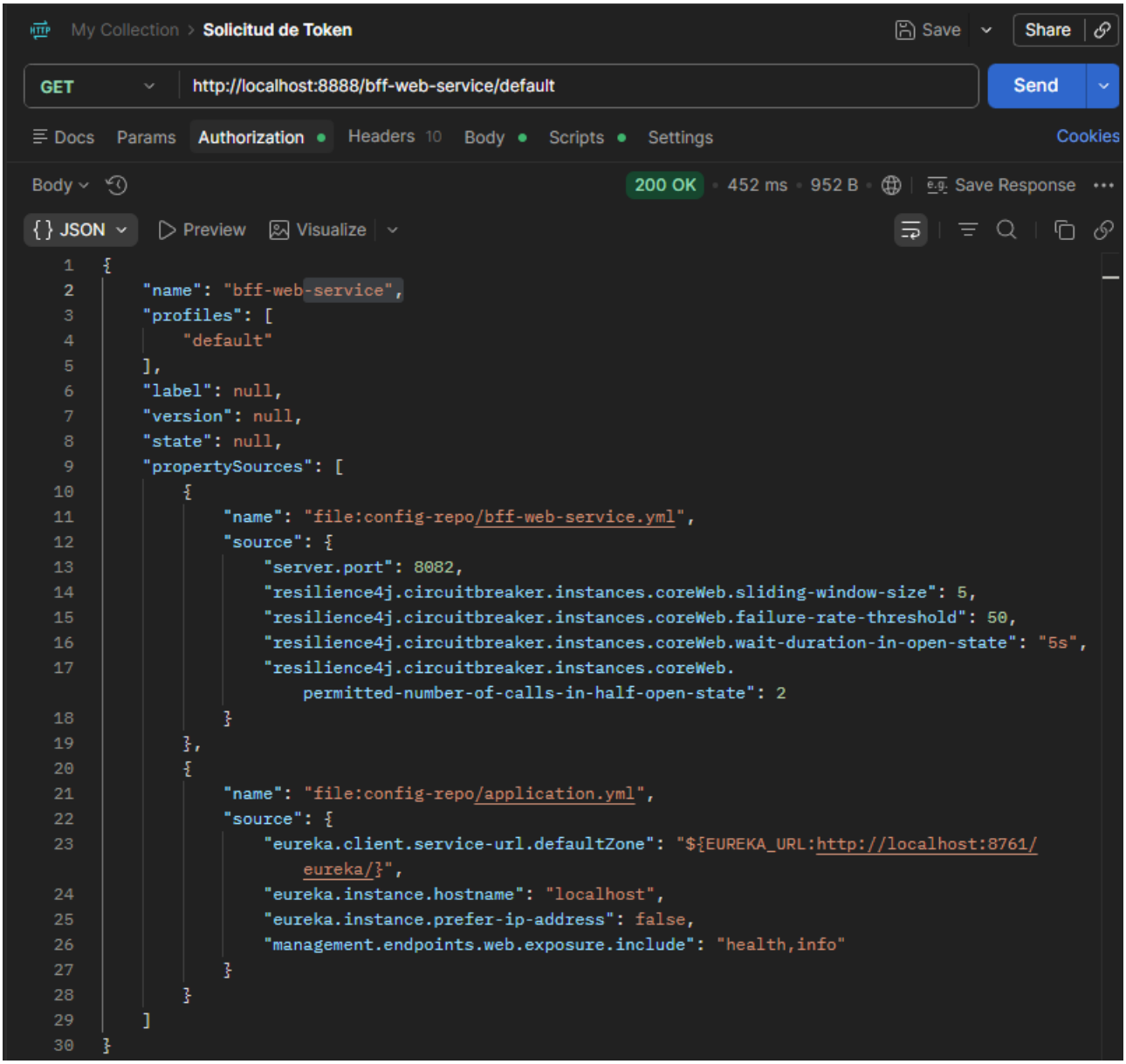
- config-server/src/main/java/cl/duoc/bancoxyz/configserver/ConfigServerApplication.java



La clase anterior permitirá Iniciar el Config Server el cual leerá los siguientes archivos:

config-repo/application.yml	Contiene la configuración común: conexión a **Eureka** para registrar y descubrir servicios, y exposición de los endpoints de monitoreo health e info.
config-repo/bff-web-service.yml	Define el puerto **8082** y configura el **circuit breaker** coreWeb, que controla las llamadas al Core cuando ocurren fallos.
config-repo/core-banking-service.yml	Define el puerto **8081**, la conexión a **PostgreSQL**, el consumidor de **Kafka** para retiros y la inicialización de tablas. También evita que los trabajos Batch se ejecuten automáticamente al iniciar el servicio.
config-repo/account-service.yml	Define el puerto **8085**, el nombre del servicio, la clave para validar **JWT** y los parámetros del circuit breaker accountService.
config-repo/payment-service.yml	Define el puerto **8086**, la validación **JWT**, el circuit breaker paymentService y la configuración para publicar mensajes **JSON en Kafka**.
config-repo/customer-service.yml	Define el puerto **8087**, el nombre del servicio, la validación **JWT** y el circuit breaker customerService.

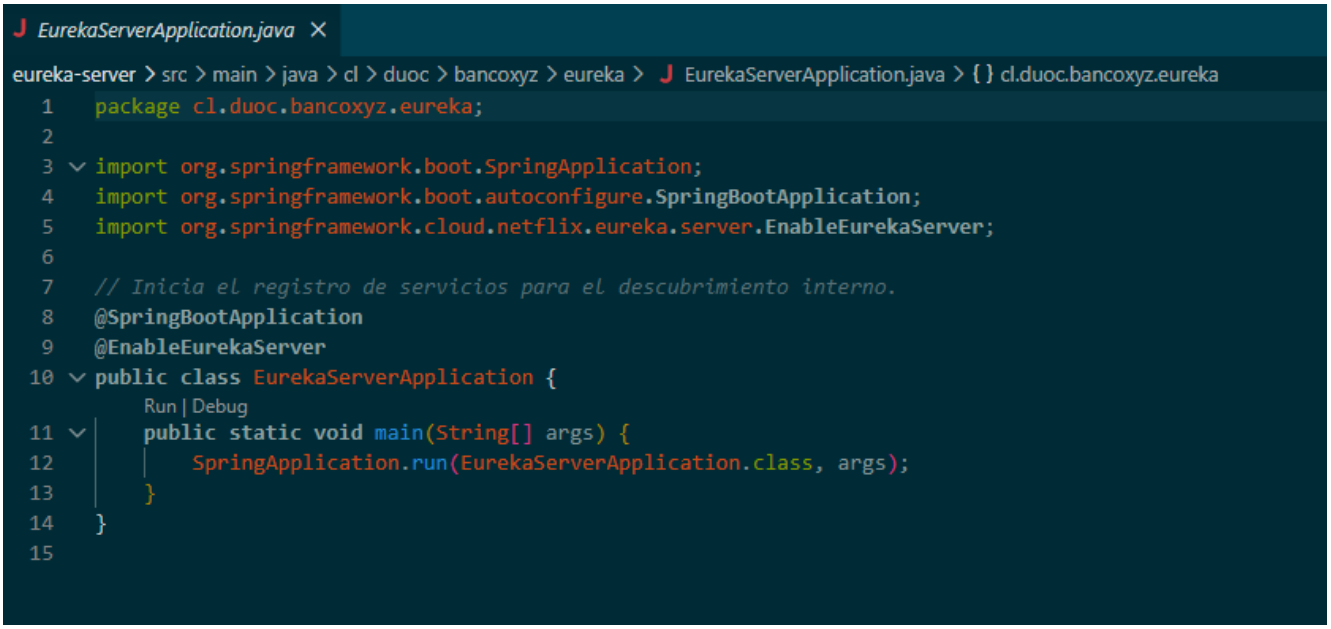
Ahora, para comprobar el funcionamiento, debemos en POSTMAN seleccionar el método GET y escribir la siguiente URL http://localhost:8888/bff-web-service/default esto nos debería devolver un código 200 OK, lo cual demuestra la configuración centralizada.



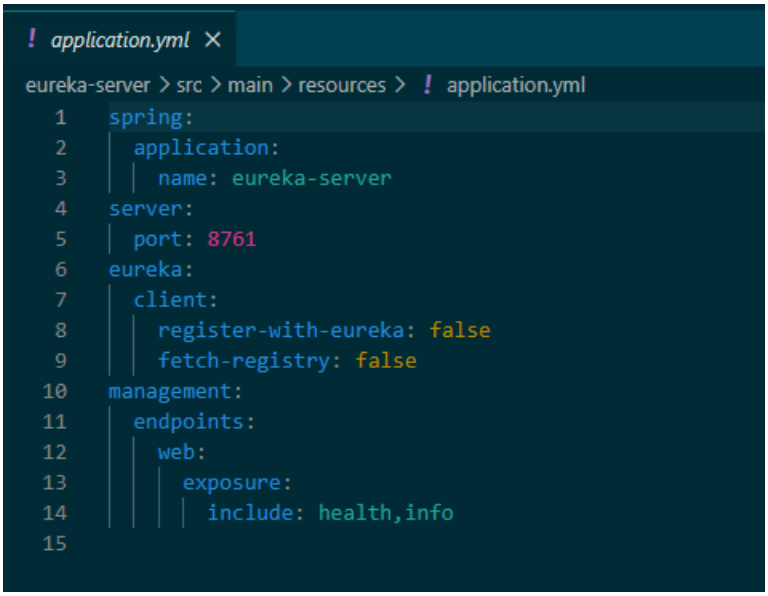
- Evidencia de Eureka

Para la aplicación de Eureka se generó la clase EurekaServerApplication.java en la siguiente ruta

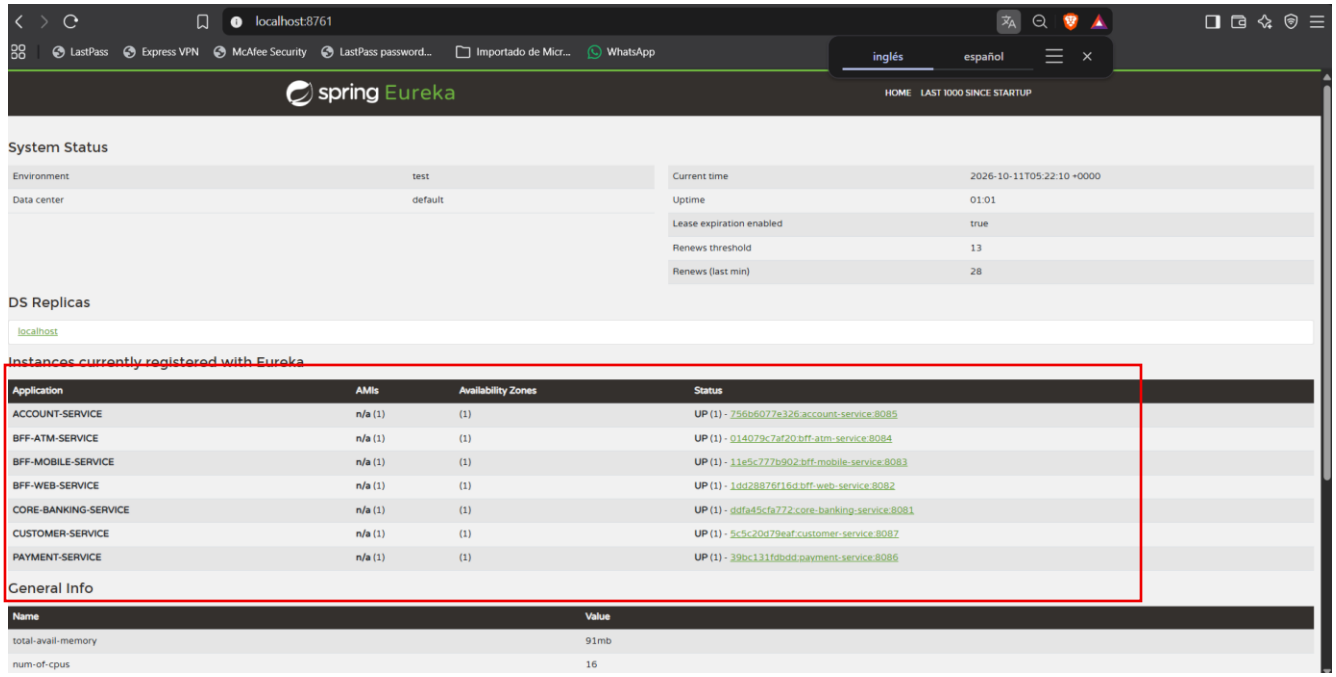
- eureka-server/src/main/java/cl/duoc/bancoxyz/eureka/EurekaServerApplication.java



- eureka-server/src/main/resources/application.yml



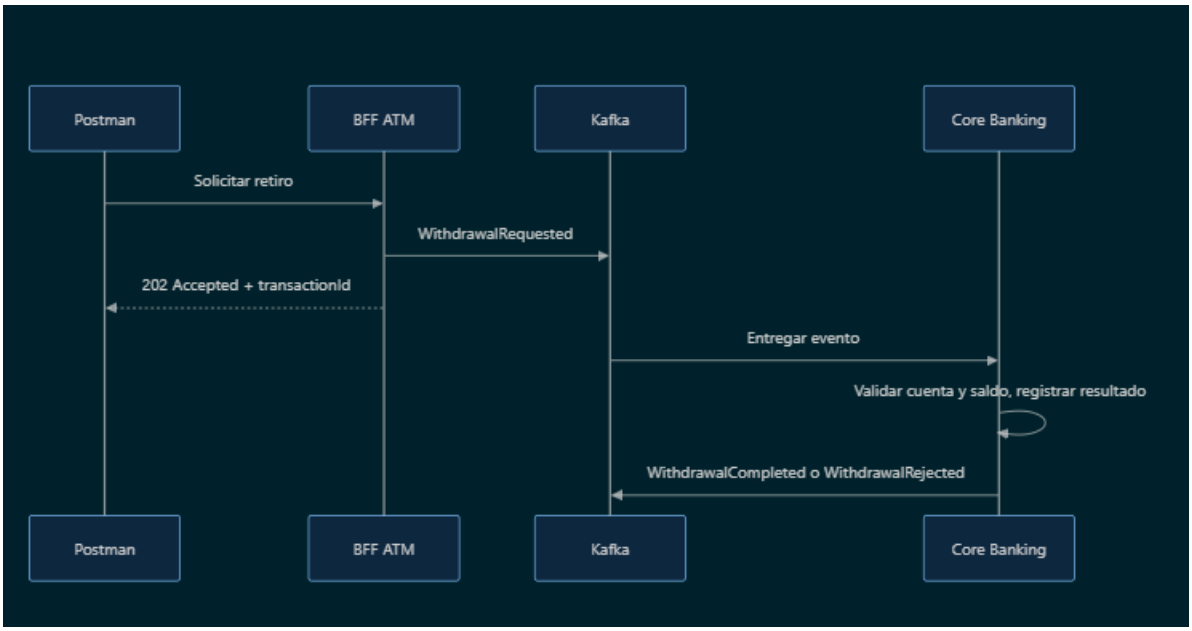
Ahora, lo que debemos hacer para visualizar la implementación, es ingresar a http://localhost:8761 para poder registrar y localizar los microservicios del proyecto, cuando el servicio se inicia, se registra en Eureka indicando su nombre, dirección y puerto.



8. Evidencia de Kafka/Eventos

Kafka permitirá que los microservicios se comuniquen mediante eventos, esto permitirá que un servicio publica un mensaje y otro lo reciba y procese, sin que el primero tenga que esperar el resultado completo. Los mensajes se organizan en canales llamados topics.

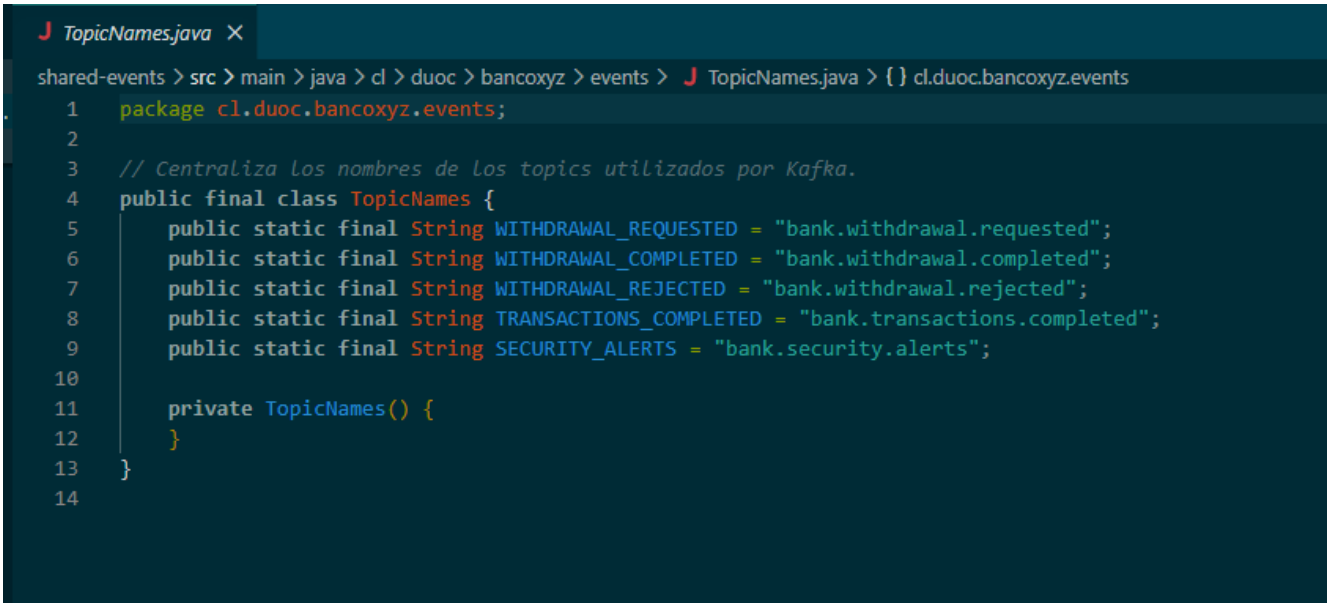
En nuestro proyecto, el retiro sigue el siguiente flujo:



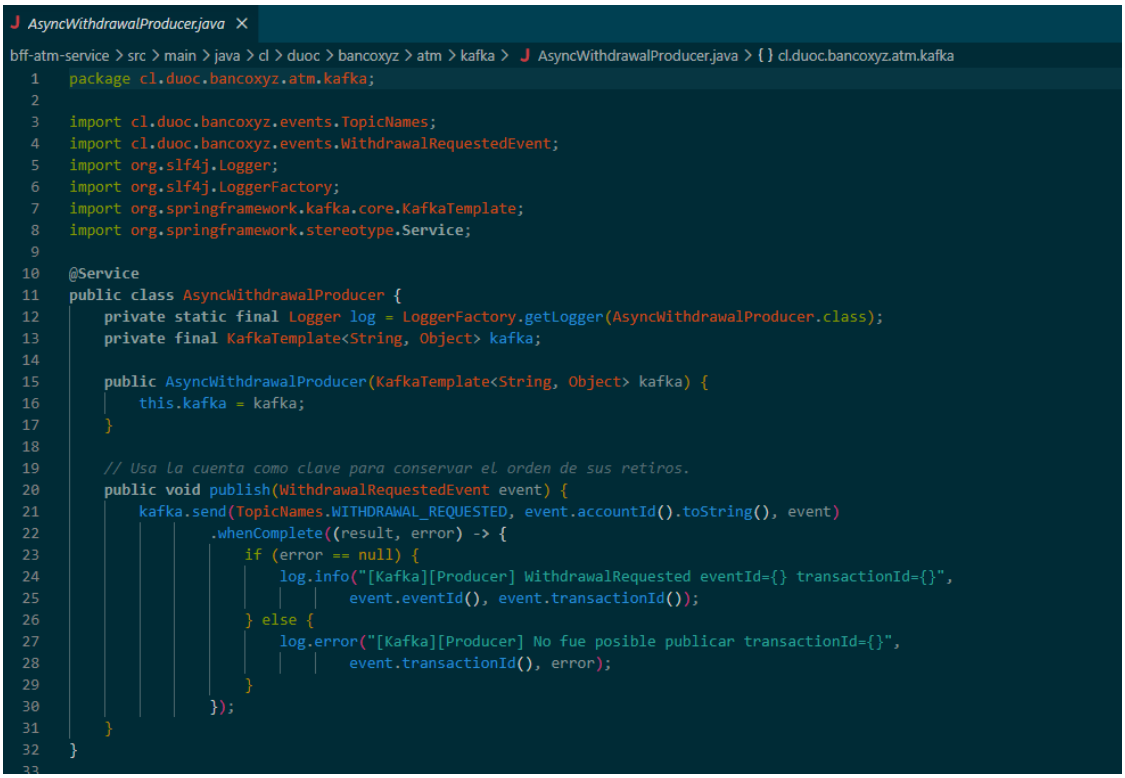
Esto permitirá que se genere un 202 Accepted que indica que la solicitud fue recibida; el resultado final se comprueba después.

Para implementar lo anterior se crearon las siguientes clases:

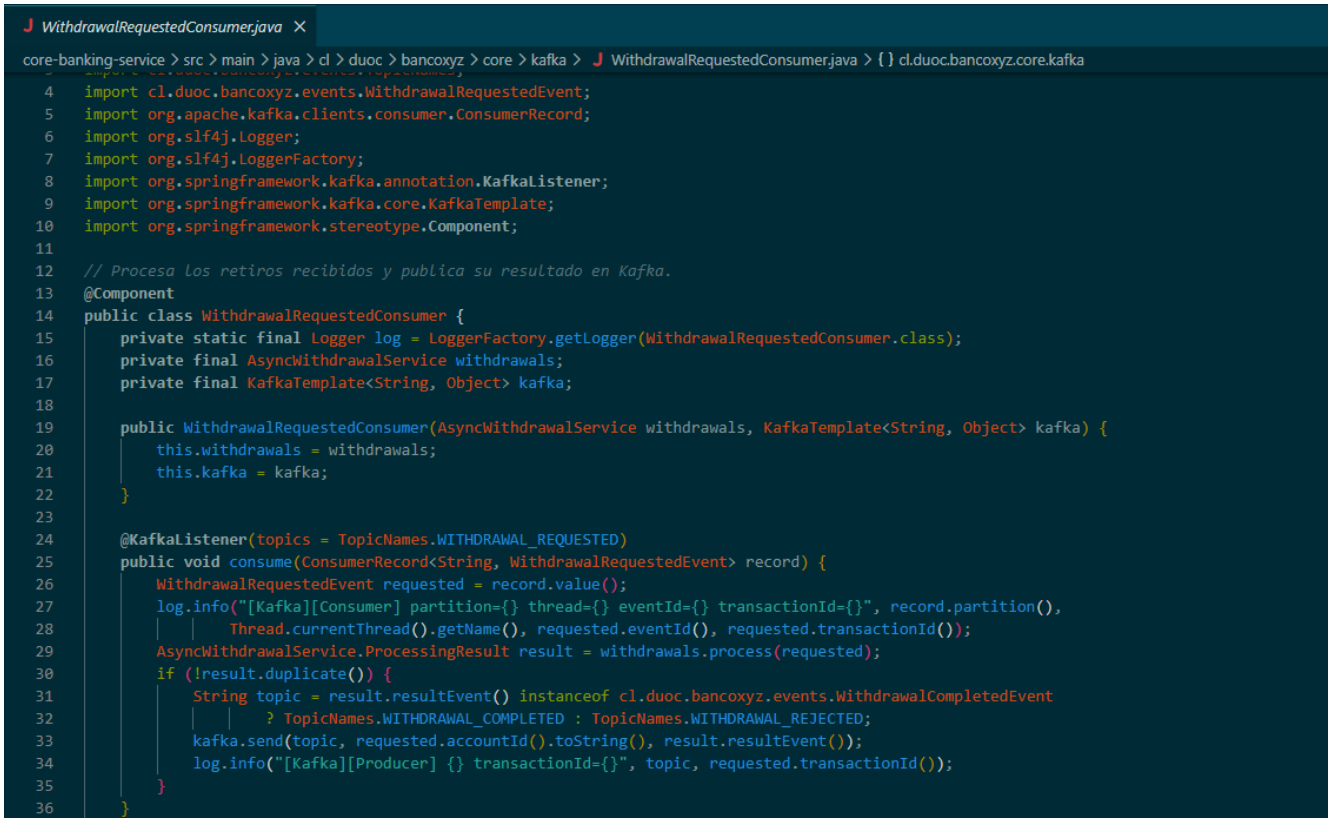
- shared-events/src/main/java/cl/duoc/bancoxyz/events/TopicNames.java



- bff-atm-service/src/main/java/cl/duoc/bancoxyz/atm/kafka/AsyncWithdrawalProducer.java



- core-banking-service/src/main/java/cl/duoc/bancoxyz/core/kafka/WithdrawalRequestedConsumer.java



```java
import cl.duoc.bancoxyz.events.WithdrawalRequestedEvent;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

// Procesa los retiros recibidos y publica su resultado en Kafka.
@Component
public class WithdrawalRequestedConsumer {
    private static final Logger log = LoggerFactory.getLogger(WithdrawalRequestedConsumer.class);
    private final AsyncWithdrawalService withdrawals;
    private final KafkaTemplate<String, Object> kafka;

    public WithdrawalRequestedConsumer(AsyncWithdrawalService withdrawals, KafkaTemplate<String, Object> kafka) {
        this.withdrawals = withdrawals;
        this.kafka = kafka;
    }

    @KafkaListener(topics = TopicNames.WITHDRAWAL_REQUESTED)
    public void consume(ConsumerRecord<String, WithdrawalRequestedEvent> record) {
        WithdrawalRequestedEvent requested = record.value();
        log.info("[Kafka][Consumer] partition={} thread={} eventId={} transactionId={}", record.partition(),
                Thread.currentThread().getName(), requested.eventId(), requested.transactionId());
        AsyncWithdrawalService.ProcessingResult result = withdrawals.process(requested);
        if (!result.duplicate()) {
            String topic = result.resultEvent() instanceof cl.duoc.bancoxyz.events.WithdrawalCompletedEvent
                    ? TopicNames.WITHDRAWAL_COMPLETED : TopicNames.WITHDRAWAL_REJECTED;
            kafka.send(topic, requested.accountId().toString(), result.resultEvent());
            log.info("[Kafka][Producer] {} transactionId={}", topic, requested.transactionId());
        }
    }
```

- core-banking-service/src/main/java/cl/duoc/bancoxyz/core/kafka/AsyncWithdrawalService.java

AsyncWithdrawalService.java

core-banking-service > src > main > java > cl > duoc > bancoxyz > core > kafka > AsyncWithdrawalService.java > { } cl.duoc.bancoxyz.core.kafka

```java
package cl.duoc.bancoxyz.core.kafka;

import cl.duoc.bancoxyz.events.WithdrawalCompletedEvent;
import cl.duoc.bancoxyz.events.WithdrawalRejectedEvent;
import cl.duoc.bancoxyz.events.WithdrawalRequestedEvent;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.math.BigDecimal;
import java.time.Instant;
import java.sql.Timestamp;
import java.util.List;
import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AsyncWithdrawalService {
    private static final Logger log = LoggerFactory.getLogger(AsyncWithdrawalService.class);
    private static final BigDecimal ATM_WITHDRAWAL_MULTIPLE = new BigDecimal(val: "10000");
    private final JdbcTemplate jdbc;
    private final ObjectMapper objectMapper;

    public AsyncWithdrawalService(JdbcTemplate jdbc, ObjectMapper objectMapper) {
        this.jdbc = jdbc;
        this.objectMapper = objectMapper;
    }

    @Transactional
    public ProcessingResult process(WithdrawalRequestedEvent requested) {
        // El registro persistente evita descontar dos veces tras reintentos o reinicios.
        if (jdbc.update("INSERT INTO processed_events (event_id, processed_at) VALUES (?, ?) ON CONFLICT DO NOTHING",
                requested.eventId(), Timestamp.from(Instant.now())) == 0) {
            log.warn("[Idempotency] Evento {} ya procesado, ignorando", requested.eventId());
            return ProcessingResult.ignoredDuplicate();
        }

        saveEvent(requested.eventId(), requested.transactionId(), requested.accountId(), type: "WithdrawalRequested",
                requested.amount(), requested.occurredAt(), requested);
        if (jdbc.update("""
                INSERT INTO withdrawal_transactions (transaction_id, account_id, amount, status, created_at, updated_at)
                VALUES (?, ?, ?, 'PENDING', ?, ?)
                ON CONFLICT (transaction_id) DO NOTHING
                """, requested.transactionId(), requested.accountId(), requested.amount(), Timestamp.from(Instant.now()), Timestamp.from(Ins
            log.warn("[Idempotency] Transaccion {} ya registrada, ignorando", requested.transactionId());
            return ProcessingResult.ignoredDuplicate();
        }

        String rejection = validate(requested);
        if (rejection != null) {
            return reject(requested, rejection);
        }

        BigDecimal current = balance(requested.accountId());
        if (current == null) {
            return reject(requested, reason: "Cuenta no encontrada");
        }
```

```
60          if (current.compareTo(requested.amount()) < 0) {
61              return reject(requested, reason: "Saldo insuficiente");
62          }
63
64          BigDecimal finalBalance = current.subtract(requested.amount());
65          jdbc.update("UPDATE intereses_mensuales SET saldo_final=? WHERE cuenta_id=?", finalBalance, requested.accountId());
66          jdbc.update("UPDATE withdrawal_transactions SET status='COMPLETED', final_balance=?, updated_at=? WHERE transaction_id=?",
67                  finalBalance, Timestamp.from(Instant.now()), requested.transactionId());
68          WithdrawalCompletedEvent completed = new WithdrawalCompletedEvent(UUID.randomUUID(), requested.transactionId(),
69                  requested.accountId(), requested.amount(), finalBalance, Instant.now());
70          saveEvent(completed.eventId(), completed.transactionId(), completed.accountId(), type: "WithdrawalCompleted",
71                  completed.amount(), completed.occurredAt(), completed);
72          log.info("[Banking] Withdrawal completed accountId={} transactionId={}", requested.accountId(),
73                  requested.transactionId());
74          return ProcessingResult.completed(completed);
75      }
76
77      public TransactionStatus asyncWithdrawalStatus(UUID transactionId) {
78          List<TransactionStatus> rows = jdbc.query("""
79                  SELECT transaction_id, account_id, amount, status, final_balance, rejection_reason, updated_at
80                  FROM withdrawal_transactions WHERE transaction_id=?
81                  """, (rs, row) -> new TransactionStatus(UUID.fromString(rs.getString(columnLabel: "transaction_id")),
82                  rs.getInt(columnLabel: "account_id"), rs.getBigDecimal(columnLabel: "amount"), rs.getString(columnLabel: "status"),
83                  rs.getBigDecimal(columnLabel: "final_balance"), rs.getString(columnLabel: "rejection_reason"),
84                  rs.getTimestamp(columnLabel: "updated_at").toInstant()), transactionId);
85          return rows.isEmpty() ? null : rows.get(index: 0);
86      }
87
88      private ProcessingResult reject(WithdrawalRequestedEvent requested, String reason) {
```

```
89          jdbc.update("UPDATE withdrawal_transactions SET status='REJECTED', rejection_reason=?, updated_at=? WHERE transaction_id=?",
90                  reason, Timestamp.from(Instant.now()), requested.transactionId());
91          WithdrawalRejectedEvent rejected = new WithdrawalRejectedEvent(UUID.randomUUID(), requested.transactionId(),
92                  requested.accountId(), requested.amount(), reason, Instant.now());
93          saveEvent(rejected.eventId(), rejected.transactionId(), rejected.accountId(), type: "WithdrawalRejected",
94                  rejected.amount(), rejected.occurredAt(), rejected);
95          log.info("[Banking] Withdrawal rejected accountId={} transactionId={} reason={}", requested.accountId(),
96                  requested.transactionId(), reason);
97          return ProcessingResult.rejected(rejected);
98      }
99
100     private String validate(WithdrawalRequestedEvent requested) {
101         if (requested.amount() == null || requested.amount().signum() <= 0) {
102             return "El monto debe ser mayor a cero";
103         }
104         if (requested.amount().remainder(ATM_WITHDRAWAL_MULTIPLE).signum() != 0) {
105             return "El cajero solo permite retiros en multiplos de 10000";
106         }
107         return null;
108     }
109
110     private BigDecimal balance(Integer accountId) {
111         List<BigDecimal> rows = jdbc.query("SELECT saldo_final FROM intereses_mensuales WHERE cuenta_id=?",
112                 (rs, row) -> rs.getBigDecimal(columnLabel: "saldo_final"), accountId);
113         return rows.isEmpty() ? null : rows.get(index: 0);
114     }
115
116     // Guarda hechos inmutables; el saldo operativo sigue siendo una proyeccion.
117     private void saveEvent(UUID eventId, UUID transactionId, Integer accountId, String type, BigDecimal amount,
```

```
118             Instant occurredAt, Object payload) {
119         try {
120             jdbc.update("""
121                     INSERT INTO banking_events (event_id, transaction_id, account_id, event_type, amount, occurred_at, payload)
122                     VALUES (?, ?, ?, ?, ?, ?, CAST(? AS jsonb))
123                     """, eventId, transactionId, accountId, type, amount, Timestamp.from(occurredAt),
124                     objectMapper.writeValueAsString(payload));
125         } catch (JsonProcessingException exception) {
126             throw new IllegalStateException(message: "No fue posible serializar el evento", exception);
127         }
128     }
129
130     public record ProcessingResult(Object resultEvent, boolean duplicate) {
131         static ProcessingResult completed(WithdrawalCompletedEvent event) {
132             return new ProcessingResult(event, duplicate: false);
133         }
134
135         static ProcessingResult rejected(WithdrawalRejectedEvent event) {
136             return new ProcessingResult(event, duplicate: false);
137         }
138
139         static ProcessingResult ignoredDuplicate() {
140             return new ProcessingResult(resultEvent: null, duplicate: true);
141         }
142     }
143
144     public record TransactionStatus(UUID transactionId, Integer accountId, BigDecimal amount, String status,
145             BigDecimal finalBalance, String rejectionReason, Instant updatedAt) {
146     }
```

- payment-service/src/main/java/cl/duoc/bancoxyz/payment/PaymentController.java

PaymentController.java

payment-service > src > main > java > cl > duoc > bancoxyz > payment > PaymentController.java > { } cl.duoc.bancoxyz.payment

```java
package cl.duoc.bancoxyz.payment;

import cl.duoc.bancoxyz.events.TopicNames;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/payments")
public class PaymentController {

  private final KafkaTemplate<String, Object> kafkaTemplate;

  public PaymentController(KafkaTemplate<String, Object> kafkaTemplate) {
    this.kafkaTemplate = kafkaTemplate;
  }

  @PostMapping("/transfer")
  public PaymentResponse transfer(@RequestBody PaymentRequest request) {
    PaymentResponse response = new PaymentResponse(
        UUID.randomUUID().toString(),
        request.sourceAccountId(),
        request.targetAccountId(),
        request.amount(),
        status: "COMPLETED",
        Instant.now());
    kafkaTemplate.send(TopicNames.TRANSACTIONS_COMPLETED, response.transactionId(), response);
    return response;
```

```java
    }

    @PostMapping("/deposit")
    public PaymentResponse deposit(@RequestBody DepositRequest request) {
        PaymentResponse response = new PaymentResponse(
                UUID.randomUUID().toString(),
                sourceAccountId: "CASH",
                request.accountId(),
                request.amount(),
                status: "COMPLETED",
                Instant.now());
        kafkaTemplate.send(TopicNames.TRANSACTIONS_COMPLETED, response.transactionId(), response);
        return response;
    }

    public record PaymentRequest(String sourceAccountId, String targetAccountId, BigDecimal amount) {
    }

    public record DepositRequest(String accountId, BigDecimal amount) {
    }

    public record PaymentResponse(
            String transactionId,
            String sourceAccountId,
            String targetAccountId,
            BigDecimal amount,
            String status,
            Instant completedAt) {
    }
}

```

Para mostrar el funcionamiento se debe hacer lo siguiente, desde la terminal debemos escribir docker exec -it banco-xyz-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 –list. Esto me debe de devolver lo siguiente:

bank.withdrawal.requested

bank.withdrawal.completed

bank.withdrawal.rejected

```
PS C:\Users\rzuni\Desktop\09 - EFT\EFT_S9_RaulZuniga\proyecto> docker exec -it banco-xyz-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
__consumer_offsets
bank.withdrawal.completed
bank.withdrawal.rejected
bank.withdrawal.requested

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug banco-xyz-kafka
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\rzuni\Desktop\09 - EFT\EFT_S9_RaulZuniga\proyecto>
```

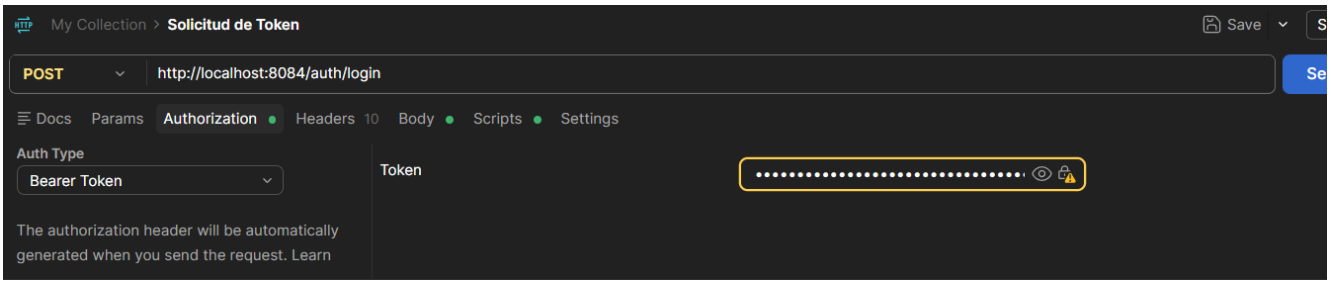
Ahora nos vamos a POSTMAN y vamos a obtener el Token ATM, para ello seleccionamos el método POST y escribimos la siguiente URL http://localhost:8084/auth/login después nos vamos donde dice Body, seleccionamos raw y escribimos el siguiente JSON:

```
{
  "username": "atmuser",
  "password": "atm123"
}
```

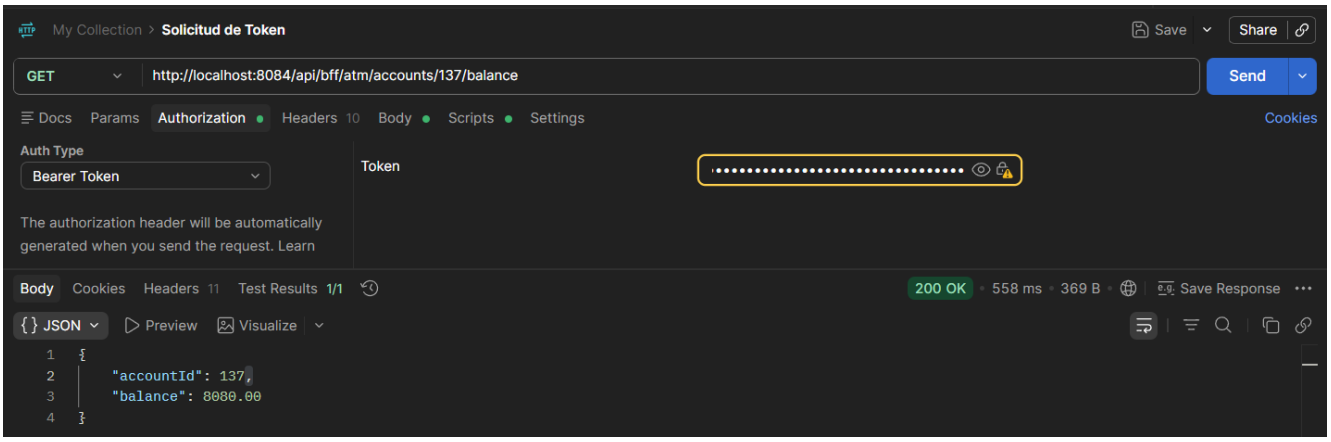
Ahora le damos al botón Send y para obtener el AccessToken



Y dicho Token lo agregamos a Authorization, Barrer Token y lo pegamos en el espacio para colocar el Token

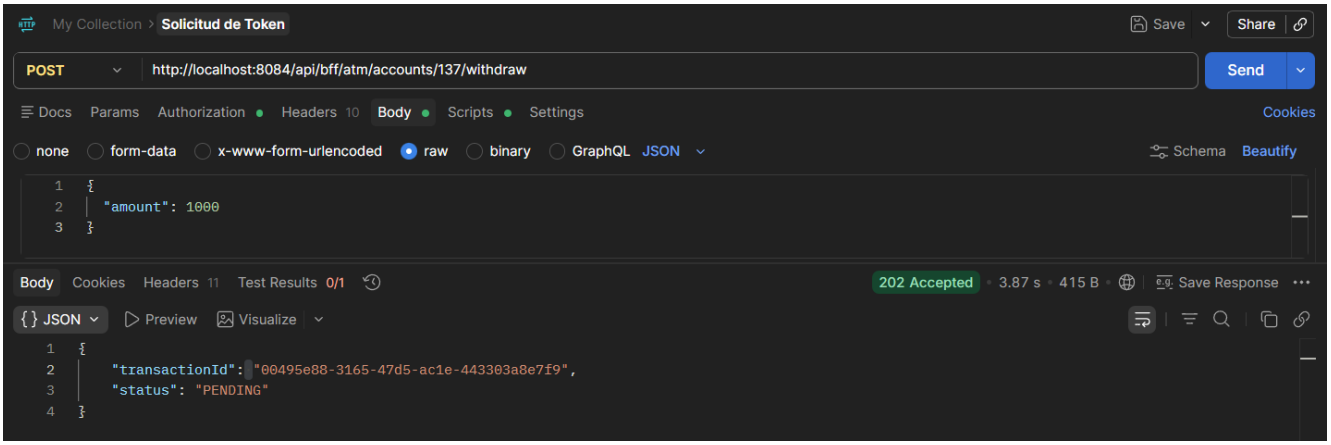


Ahora vamos a consultar el saldo, para ello vamos al método GET y escribimos la siguiente URL http://localhost:8084/api/bff/atm/accounts/137/balance y vemos cuanto dinero tiene.



Ahora vamos a solicitar un retiro de dinero, para ello cambiamos al método POST y escribimos la siguiente URL http://localhost:8084/api/bff/atm/accounts/137/withdraw y nos vamos donde dice Body, raw y escribimos el siguiente JSON:

{
  "amount": 1000
}



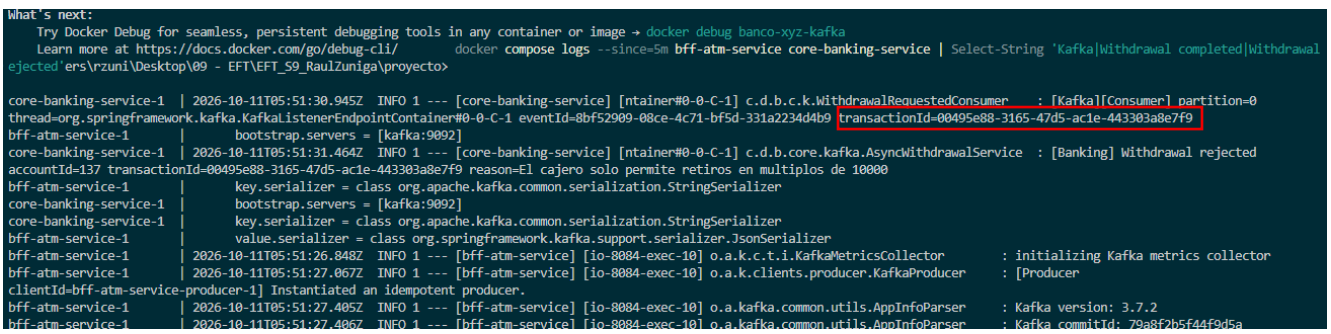
Esto nos devolvió un 202 Accepted con el siguiente JSON

{
  "transactionId": "00495e88-3165-47d5-ac1e-443303a8e7f9",
  "status": "PENDING"
}

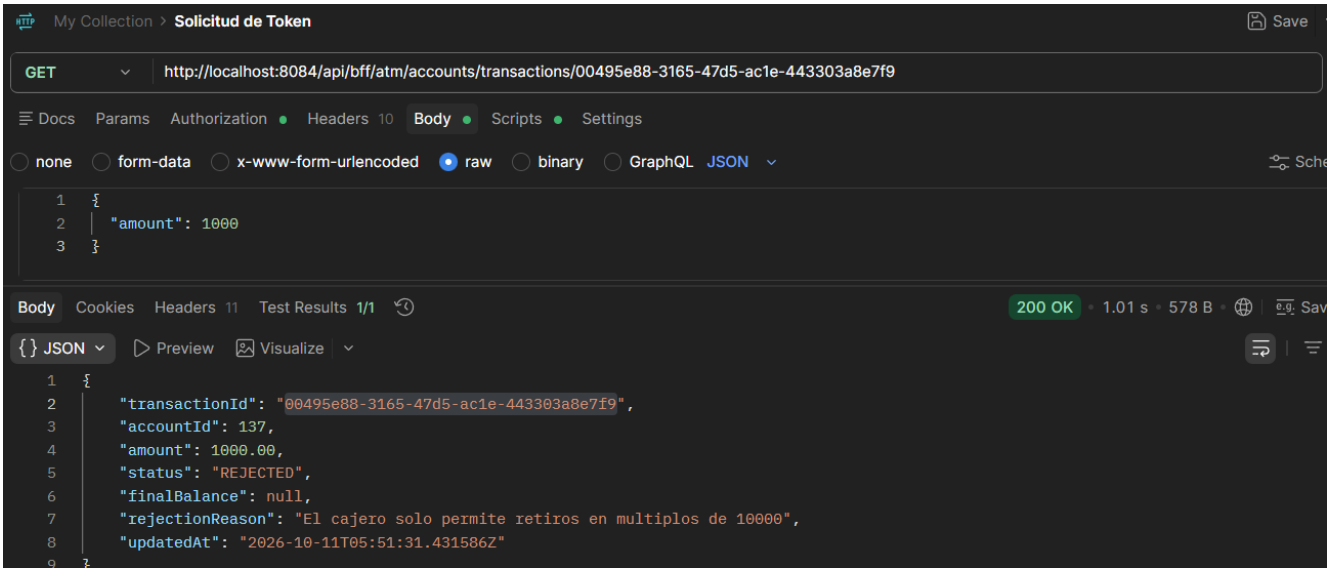
Esto muestra que se generó una ID de transacción y el estatus de que se encuentra pendiente. Ahora vamos a demostrar que Kafka entregó el evento, para ello desde la carpeta del proyecto vamos a ejecutar en la terminal lo siguiente:

docker compose logs --since=5m bff-atm-service core-banking-service | Select-String 'Kafka|Withdrawal completed|Withdrawal rejected'

y buscaremos donde dice sale la transactionid que se generó



Ahora para consultar el resultado final del proceso, me debo ir a POSTMAN y debemos seleccionar el método GET con la siguiente URL http://localhost:8084/api/bff/atm/accounts/transactions/<transactionId>, reemplazando el transactiuonId con el que nos entregó POSTMAN en el paso anterior



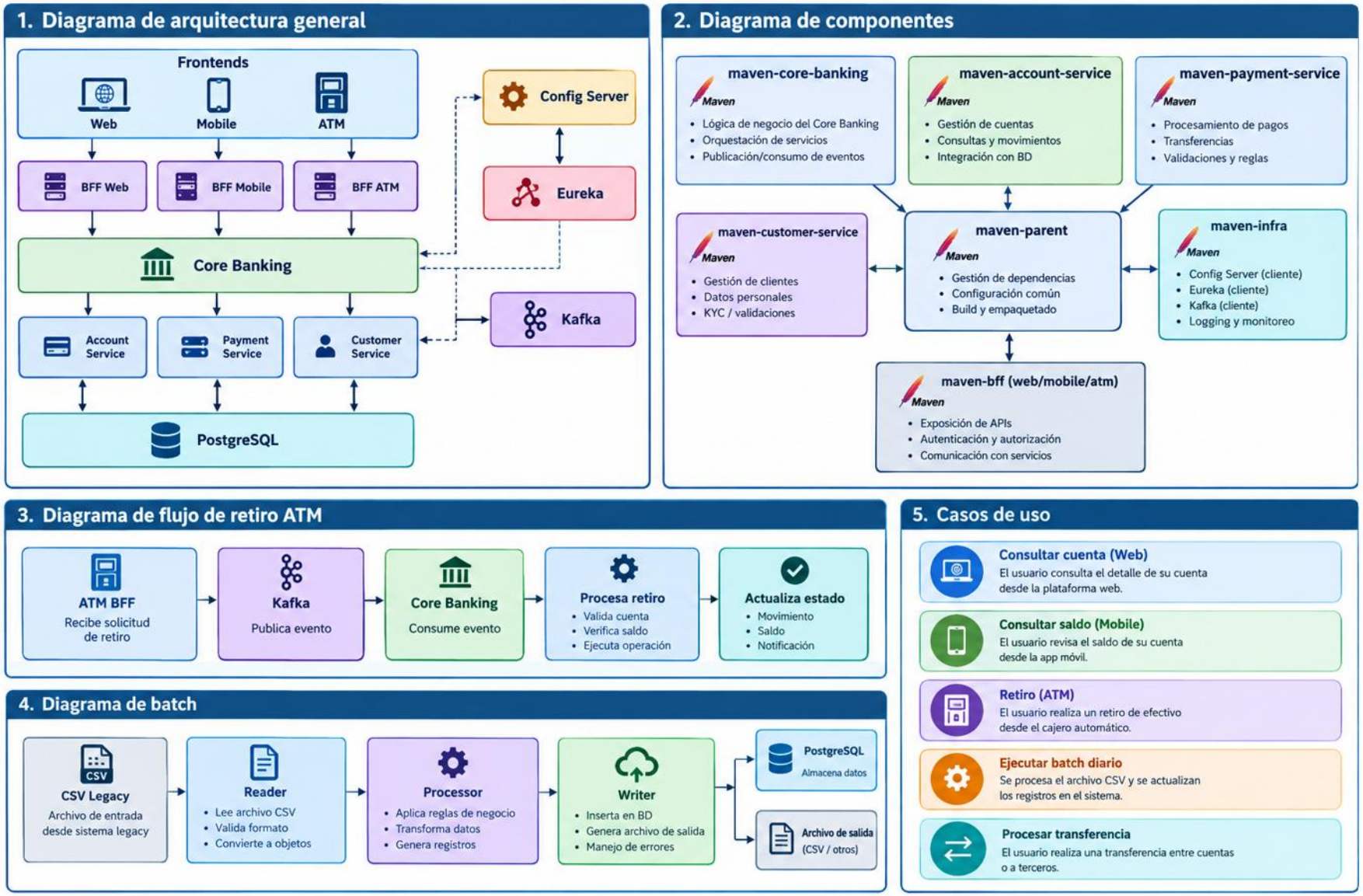
- Evidencia de Resilience4j

Es la biblioteca que nos ayudará a controlar los fallos cuando un microservicio llama a otro. En mi proyecto va a proteger las consultas de los BFF hacia Core Banking mediante un circuit breaker o un interruptor del circuito. Esto quiere decir que si las llamadaqs fallan repetidamente, el circuito se abre y rechaza temporalmente nuevas llamadas al Core. Depués permitirá algunas consultas de prueba para comprobar si el servicio se recuperó. Para ello vamos a usar los archivos yml de la siguiente forma

config-repo/bff-mobile-service.yml	El puerto **8082** y el circuito coreWeb, que protege las consultas del canal Web al Core.
config-repo/bff-atm-service.yml	El puerto **8083** y los parámetros de Resilience4j para las llamadas del canal Mobile al Core.
config-repo/account-service.yml	El puerto **8084** y los parámetros de Resilience4j para las consultas del canal ATM al Core.
config-repo/payment-service.yml	El puerto **8085**, el nombre del servicio, la clave de validación JWT y el circuito accountService.

config-repo/customer-service.yml	El puerto **8086**, el nombre del servicio, la validación JWT, la publicación de mensajes en Kafka y el circuito paymentService.
config-repo/bff-mobile-service.yml	El puerto **8087**, el nombre del servicio, la clave de validación JWT y el circuito customerService.

Diagramas recomendados para tu PDF

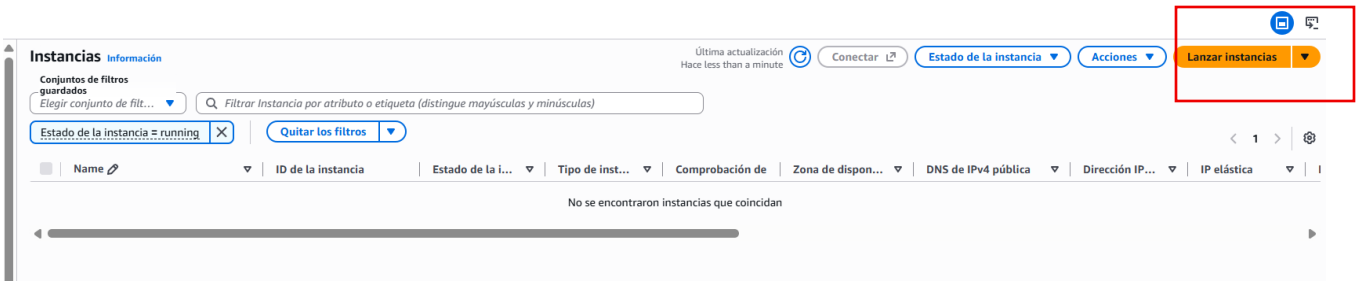


- **Pasos para desplegar el sistema en un entorno de nube:**

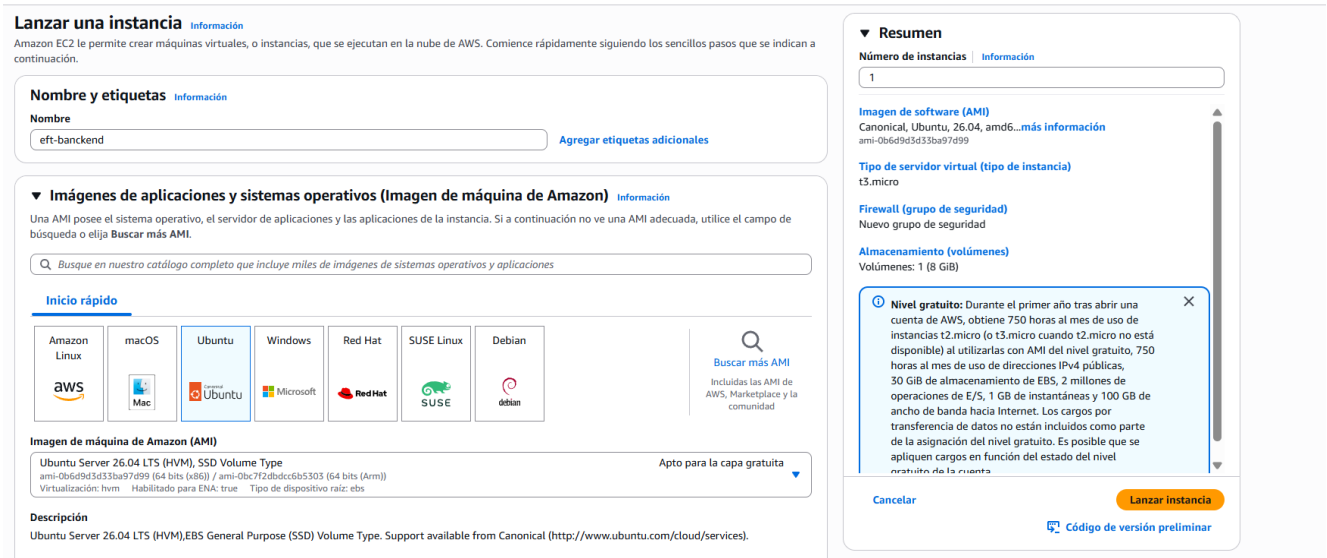
Para subir el proyecto a la nube, vamos a utilizar AWS, aquí vamos a hacer los siguientes pasos:

A- Creación de la instancia EC2

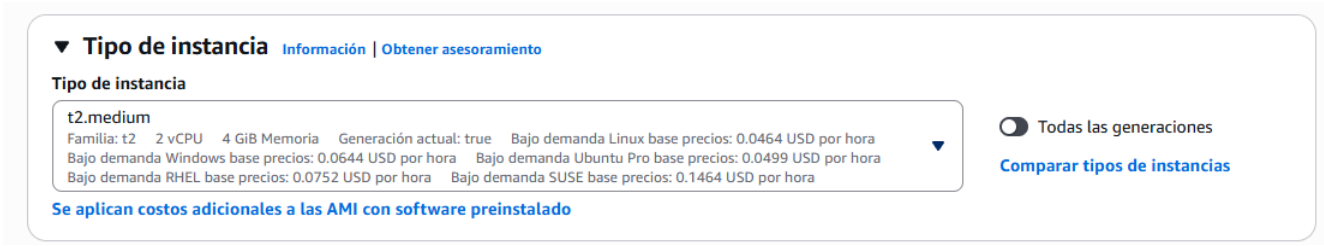
Seleccionamos EC2 en AWS y nos vamos donde dice lanzar instancia



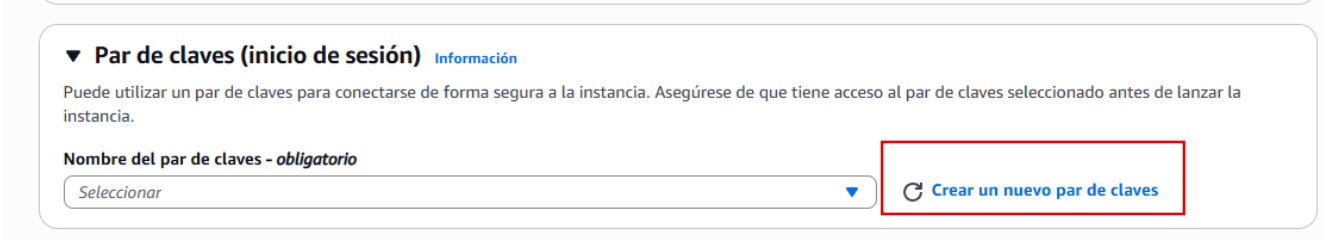
Aquí debemos colocarle un nombre a la instancia, en mi caso le coloqué eft-banckend y seleccioné el sistema Ubuntu y mantenemos la arquitectura



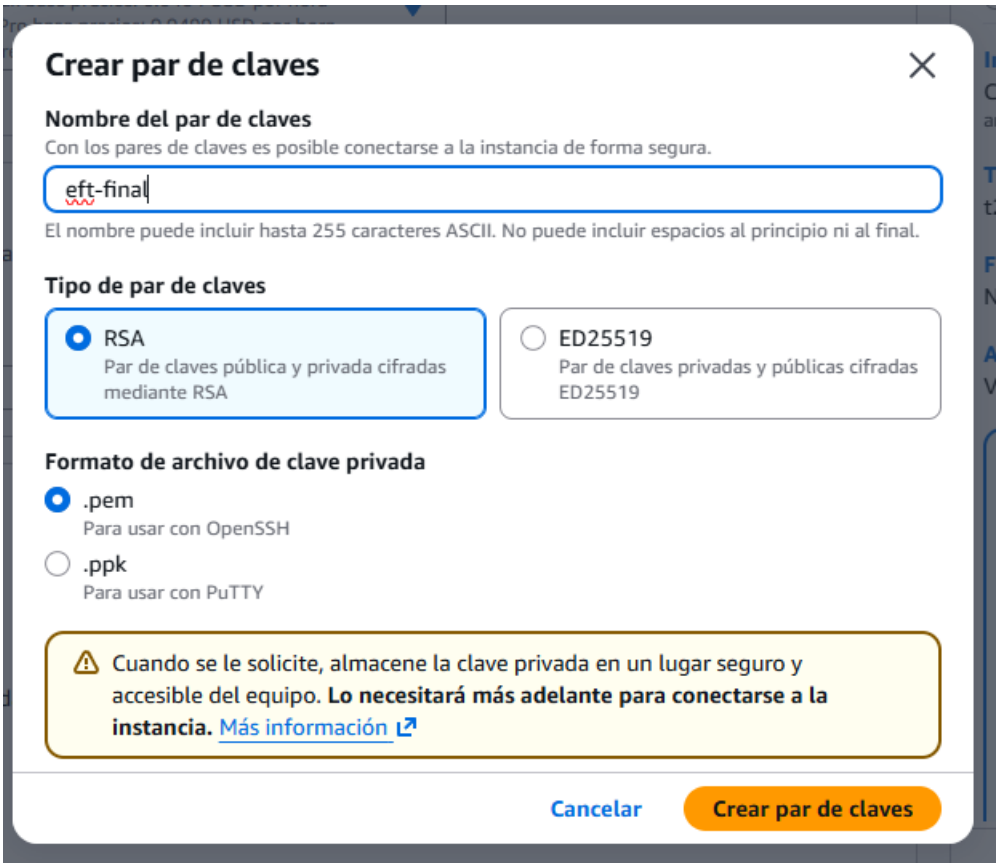
Como tipo de instancia vamos a usar t2.medium ya que vamos a trabajar con múltiples servicios y necesitamos mas potencia.



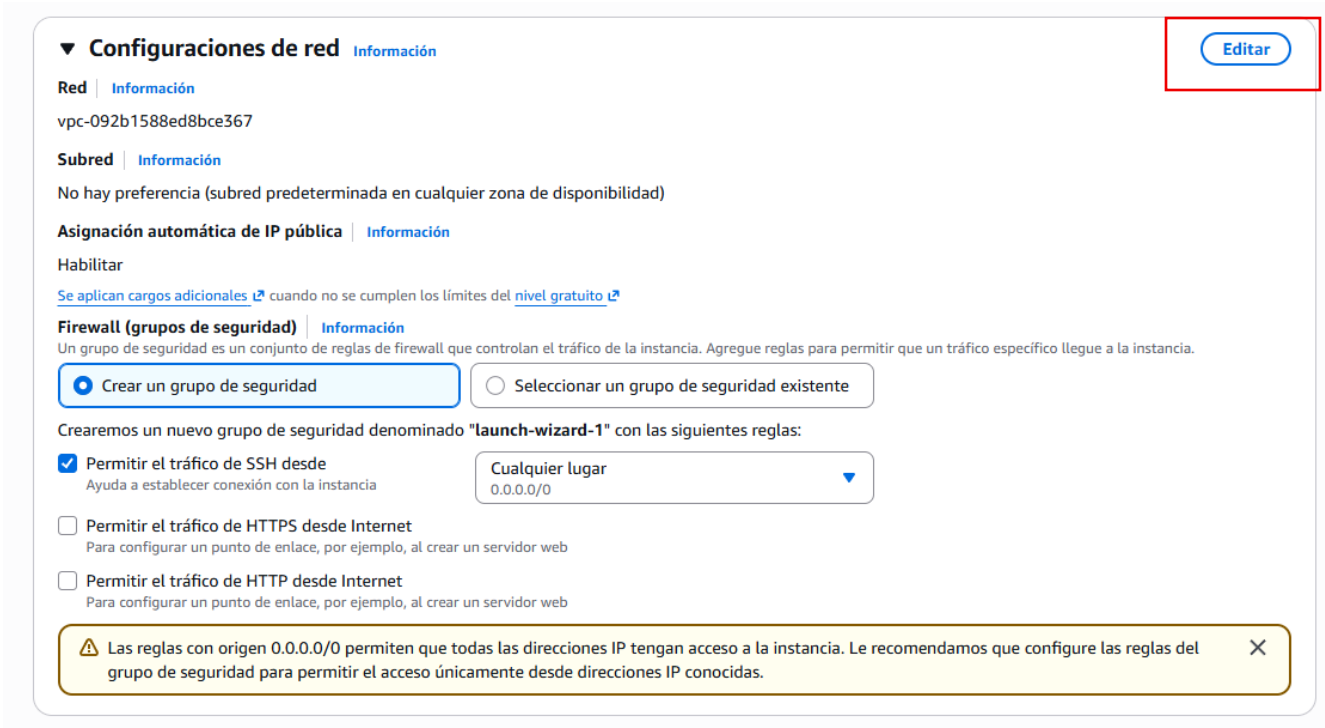
Ahora vamos a crear un par de claves



Ahora le colocamos un nombre al par de claves que en mi caso mantuve el mismo nombre que el que colocó el profesor que es eft-final y descargamos el par de claves .pem



Ahora vamos a configuración de red y vamos a habilitar dos puertos que van a ser los que vamos a ocupar para ello le damos al botón Editar



Aquí es importante tener abiertos todos los puertos donde se van a ejecutar los microservicios, para ello voy abrir los siguientes puertos

Puerto TCP	Componente	Cuándo abrirlo
22	SSH	Para conectarte a Ubuntu
8082	BFF Web	Login y consultas Web
8083	BFF Mobile	Login y consultas Mobile
8084	BFF ATM	Login, saldo y retiros
8085	Account Service	Pruebas directas de cuentas
8086	Payment Service	Pruebas directas de pagos
8087	Customer Service	Pruebas directas de clientes
8081	Core Banking	Si quieres consultar directamente el Core

Puerto TCP	Componente	Cuándo abrirlo
8761	Eureka	Para ver el panel desde tu navegador
8888	Config Server	Para consultar su configuración desde tu equipo

▼ Regla del grupo de seguridad 2 (TCP, 8082, 0.0.0.0/0, Login y consulta web) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8082

Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Login y consulta web
	0.0.0.0/0	

▼ Regla del grupo de seguridad 3 (TCP, 8083, 0.0.0.0/0, Login y consulta Mobil) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8083

Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Login y consulta Mobil
	0.0.0.0/0	

▼ Regla del grupo de seguridad 4 (TCP, 8084, 0.0.0.0/0, Login, saldo y retiros) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8084

Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Login, saldo y retiros
	0.0.0.0/0	

▼ Regla del grupo de seguridad 5 (TCP, 8085, 0.0.0.0/0, Pruebas directas de cuentas) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8085

Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Pruebas directas de cuentas
	0.0.0.0/0	

▼ Regla del grupo de seguridad 6 (TCP, 8086, 0.0.0.0/0, Pruebas directas de pagos) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8086

Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Pruebas directas de pagos
	0.0.0.0/0	

▼ Regla del grupo de seguridad 7 (TCP, 8087, 0.0.0.0/0, Pruebas directas de clientes) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8087

Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Pruebas directas de clientes
	0.0.0.0/0	

▼ Regla del grupo de seguridad 8 (TCP, 8081, 0.0.0.0/0, Si quieres consultar directamente ...) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8081

Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Si quieres consultar directamente el c
	0.0.0.0/0	

▼ Regla del grupo de seguridad 9 (TCP, 8761, 0.0.0.0/0, Para ver el panel desde tu navegad...) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8761

Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Para ver el panel desde tu navegador
	0.0.0.0/0	

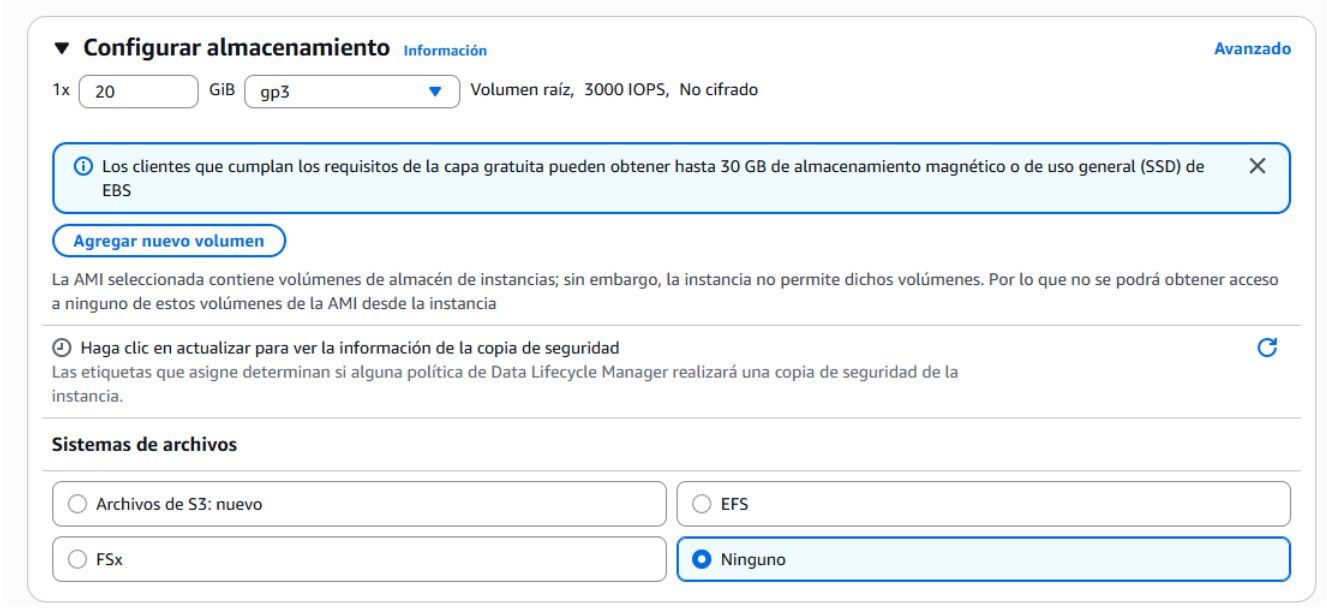
▼ Regla del grupo de seguridad 10 (TCP, 8888, 0.0.0.0/0, Para consultar su configracion des...) — Eliminar

Tipo	Protocolo	Rango de puertos
TCP personalizado	TCP	8888

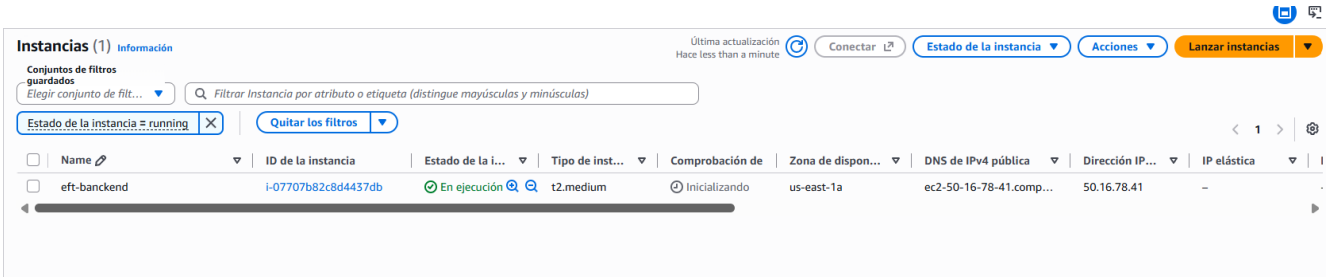
Tipo de origen	Origen	Descripción - *opcional*
Personalizado	Agregue CIDR, lista de prefijos o gr	Para consultar su configracion desde
	0.0.0.0/0	

⚠ Las reglas con origen 0.0.0.0/0 permiten que todas las direcciones IP tengan acceso a la instancia. Le recomendamos que configure las reglas del grupo de seguridad para permitir el acceso únicamente desde direcciones IP conocidas.

Para configuración de almacenamiento vamos a colocar 20 GB



Finalizamos con lanzar instancia y esperamos a que nos salga que la instancia está en ejecución



B- Ingresar a la instancia e instalar Dockers

Ya dentro de la instancia procedo a instalar Docker para ello escribo en la terminal de Linux los siguientes comandos:

Add Docker's official GPG key:

sudo apt update

sudo apt install ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc

Add the repository to Apt sources:

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF

Types: deb

URIs: https://download.docker.com/linux/ubuntu

Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")

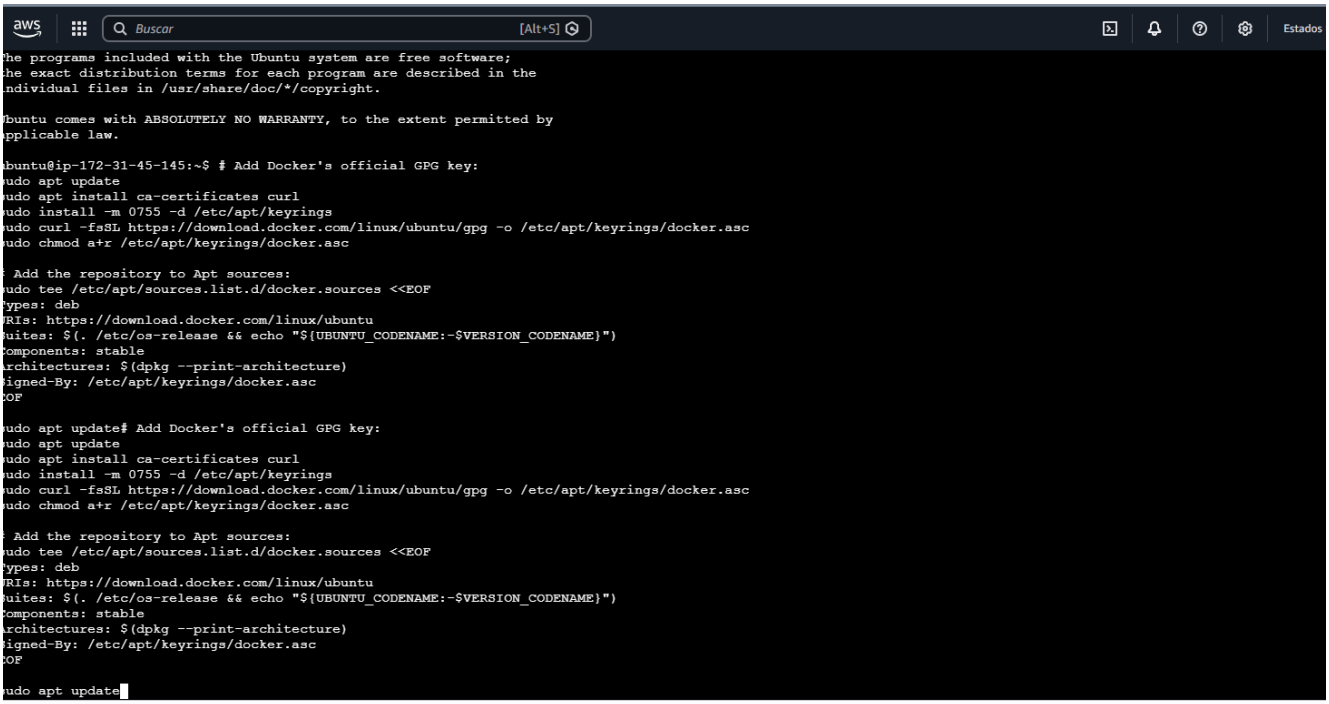
Components: stable

Architectures: $(dpkg --print-architecture)

Signed-By: /etc/apt/keyrings/docker.asc

EOF

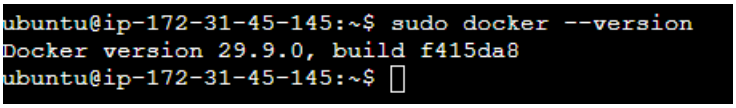
sudo apt update



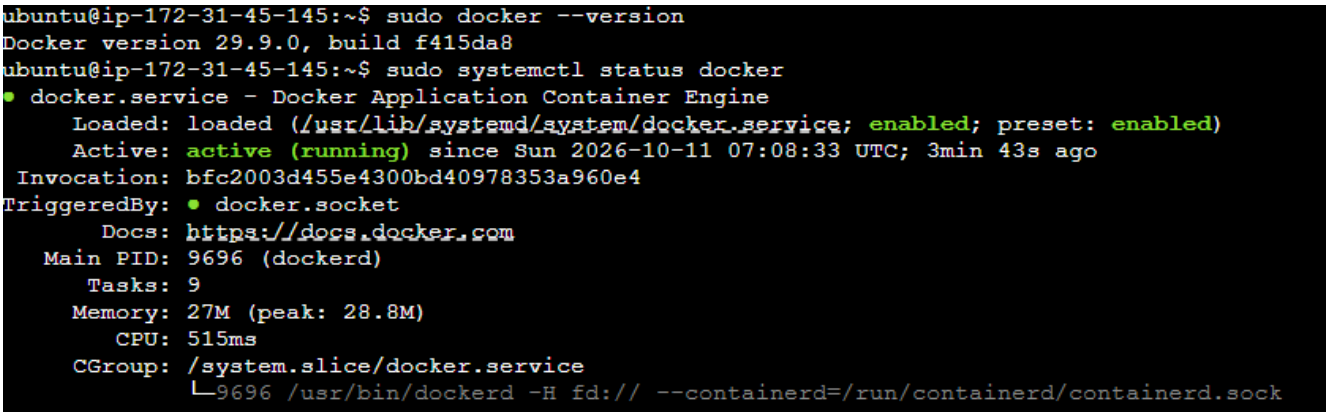
Una vez que termine de instalar el paquete de Docker para ello escribo el siguiente comando

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

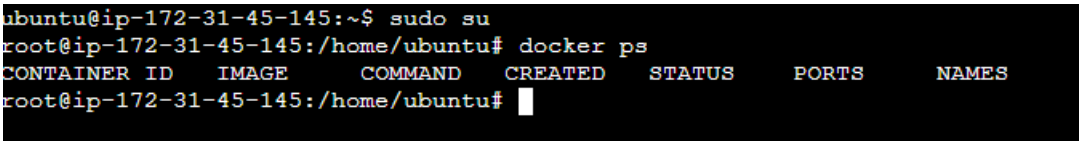
Una vez que se instaló Dockers debo ver la versión de Docker que se está ejecutando en mi equipo para ello voy a escribir el comando sudo docker –version y nos devolverá la versión instalada.



Ahora mediante el comando sudo systemctl status docker. Voy a ver si se está levantado o no.

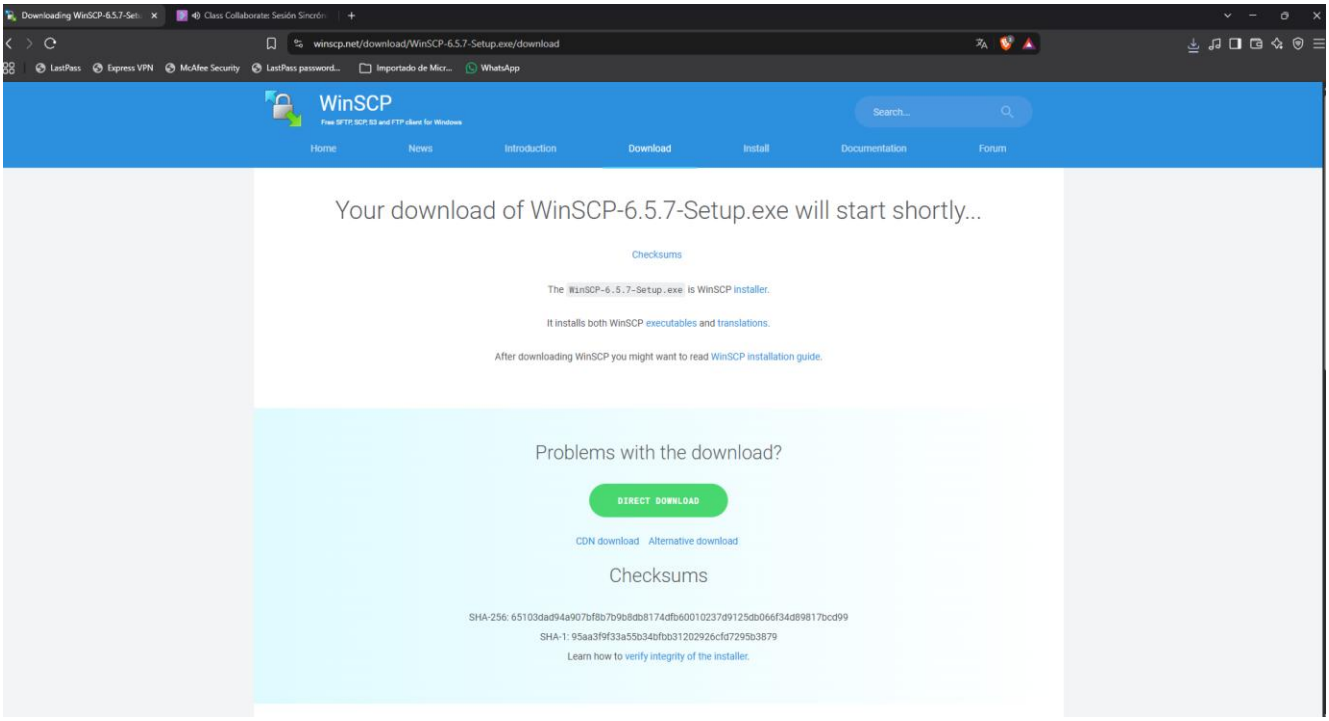


Ahora, debo darme permisos de super usuario, para ello escribo sudo su y después docker ps, esto me devolverá si hay algún contenedor levantado

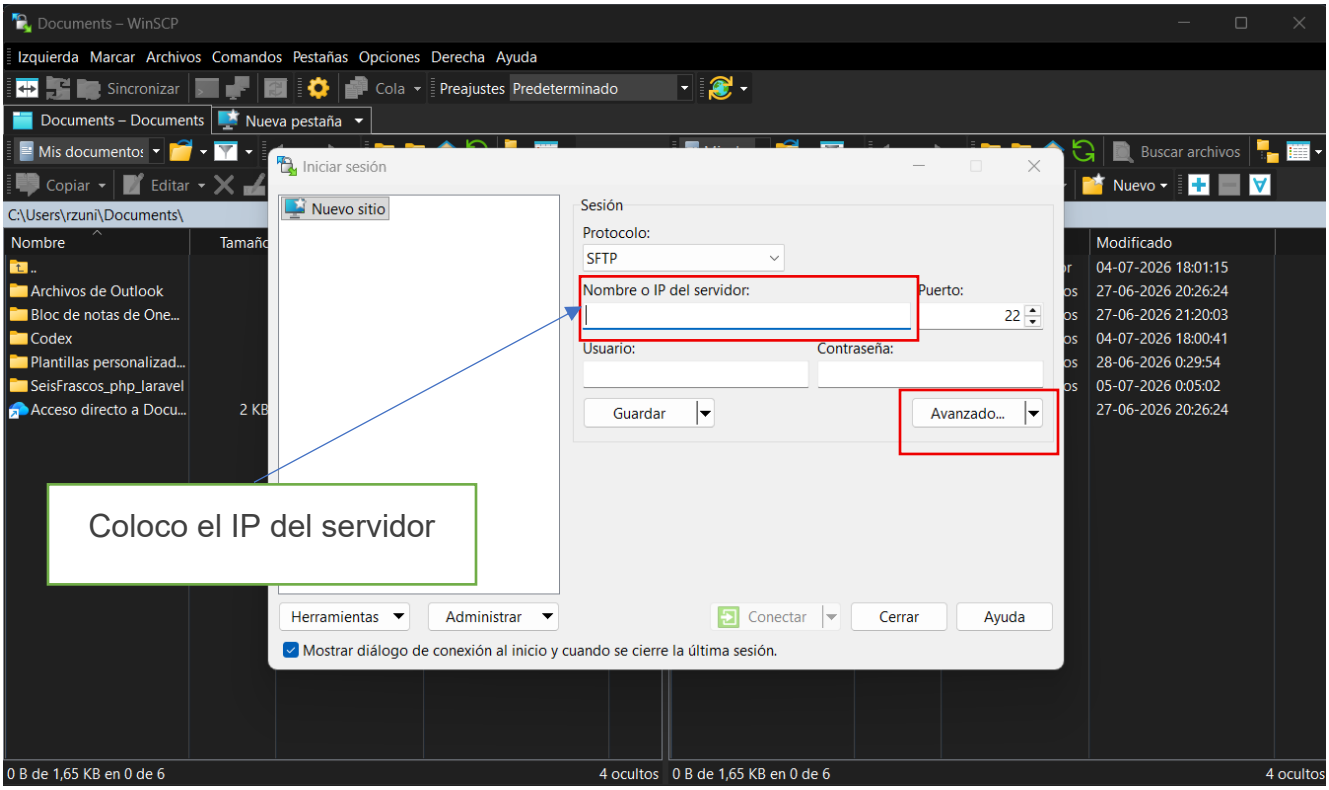


Aquí se observa que no tengo ningún contenedor levantado de momento.

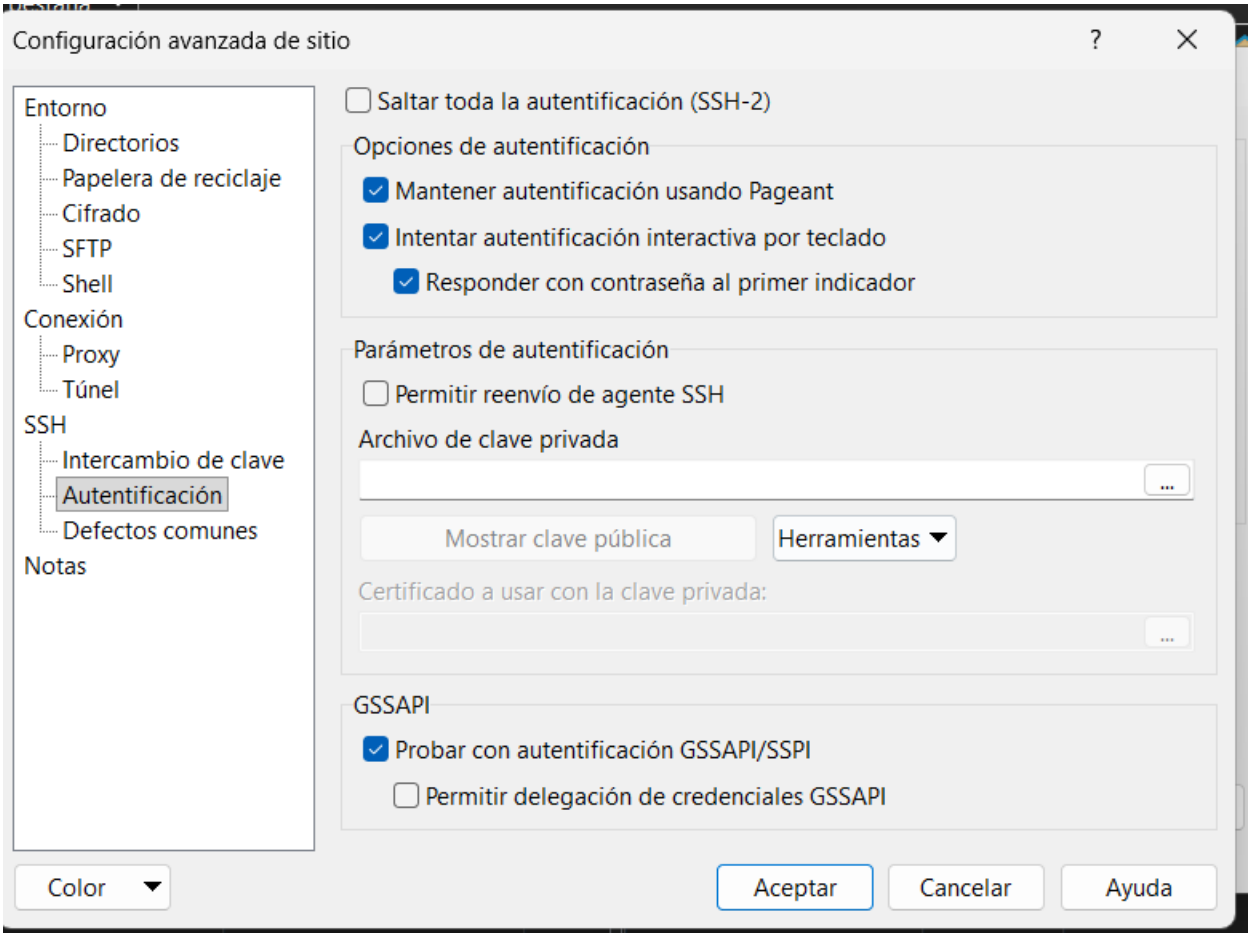
Ahora voy a instalar WinSCP que me permitirá mover archivos desde una maquina Windows hacia un servidor.



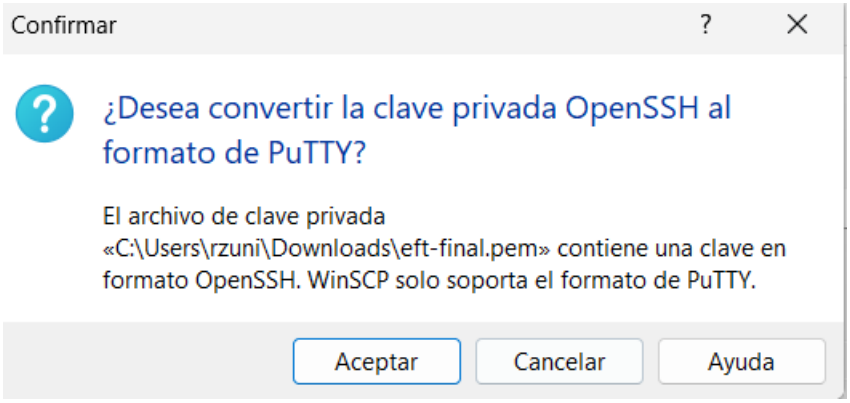
Una vez que se instaló nos abrirá una ventana aquí voy a colocar el protocolo SFTP y donde dice Nombre o IP del servidor, voy a colocar la IP de mi servidor, después me voy donde dice avanzado para cargar mi llave .pem que descargue



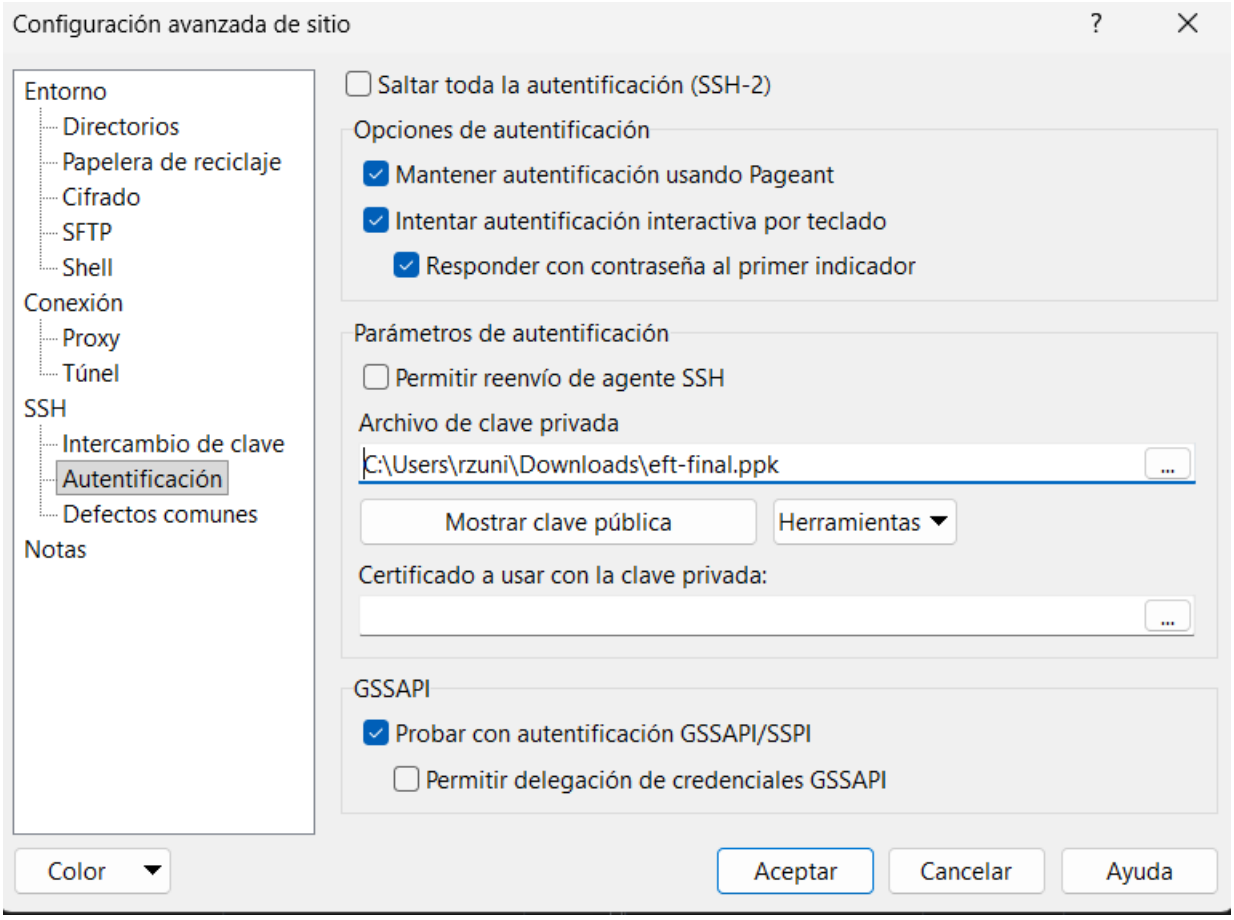
Ahora me voy donde dice Autentificación y donde dice Archivo de clave privada, le doy a los tres puntos y cargo mi clave .pem



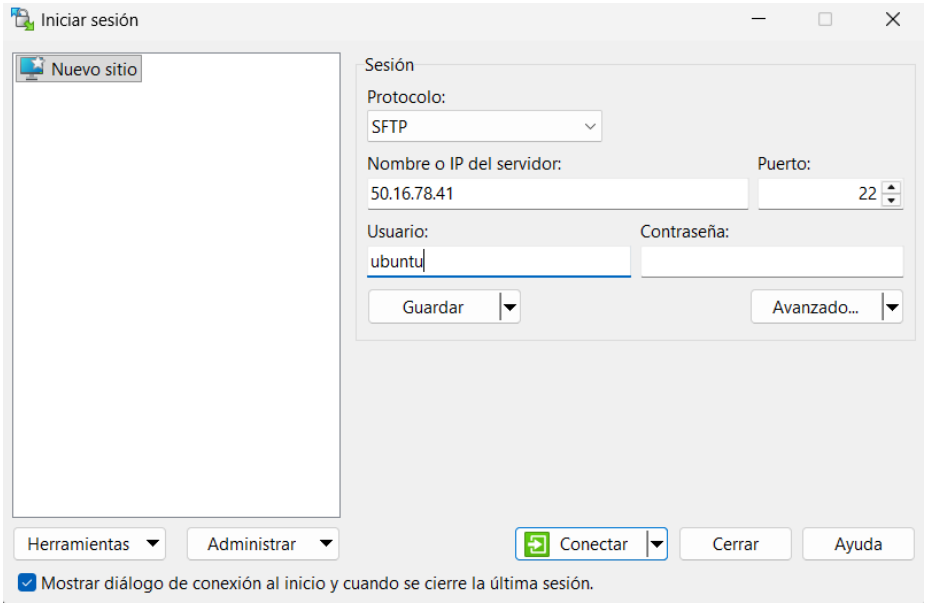
Cuando haga eso me va a lanzar un mensaje para convertir la clave .pem a un archivo ppk, le doy a aceptar



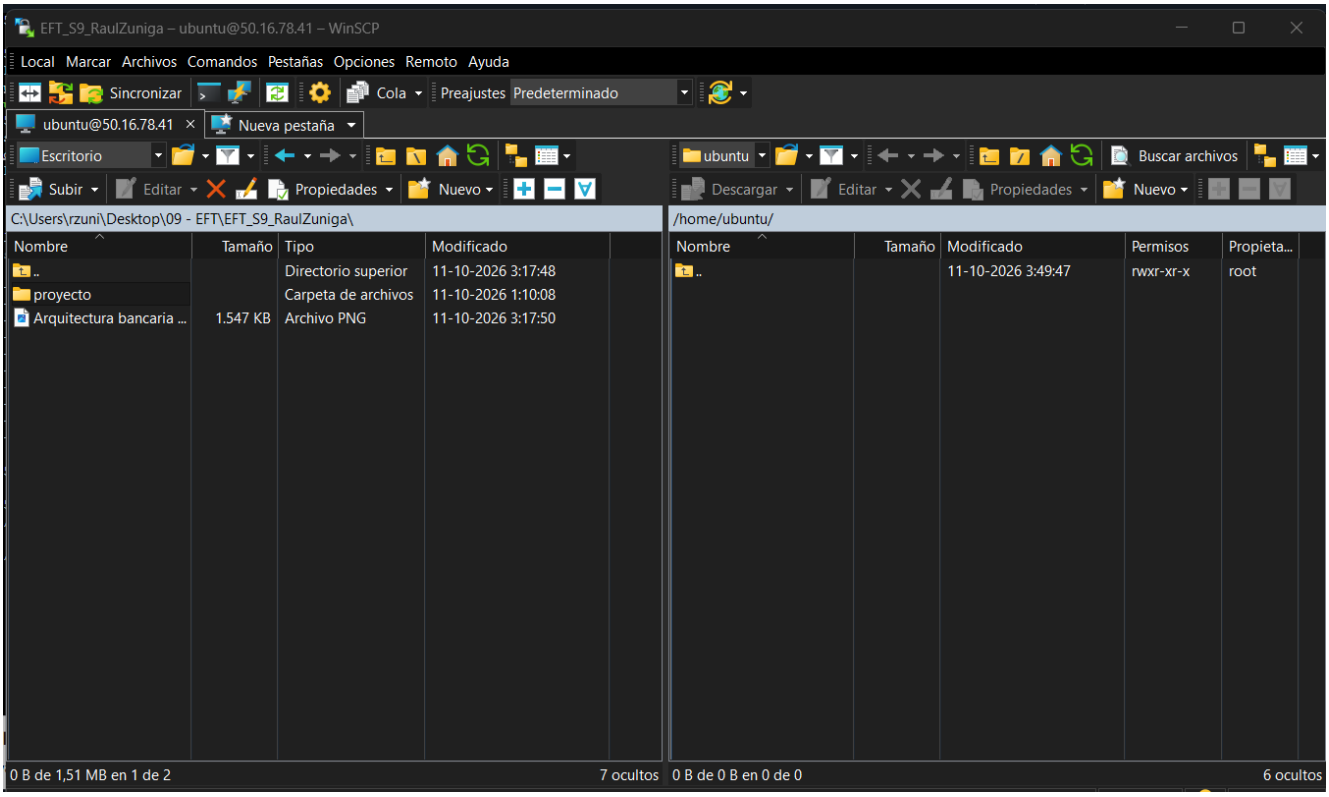
Lo anterior hará que mi llave ahora sea de tipo PPK y que lo reconozca el programa y le doy a aceptar.



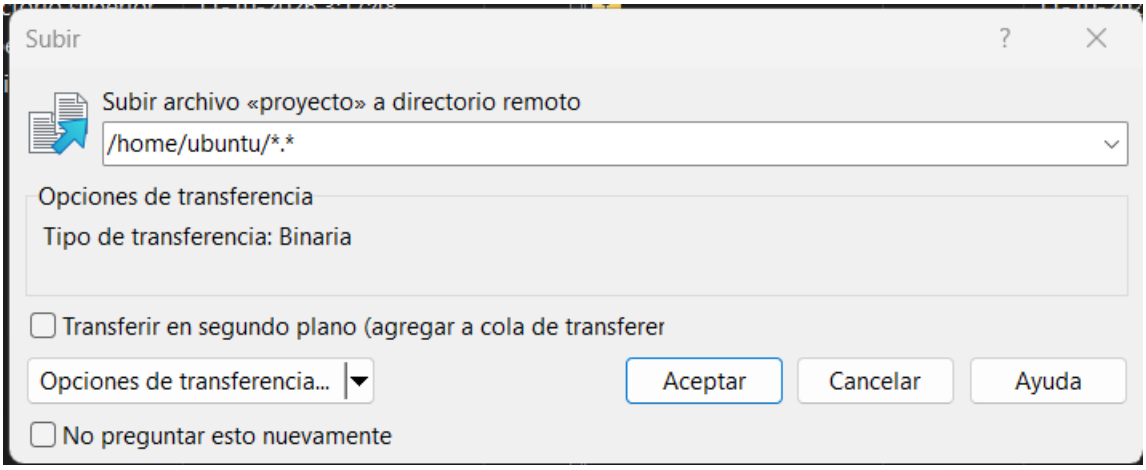
Finalizo con el botón conectar



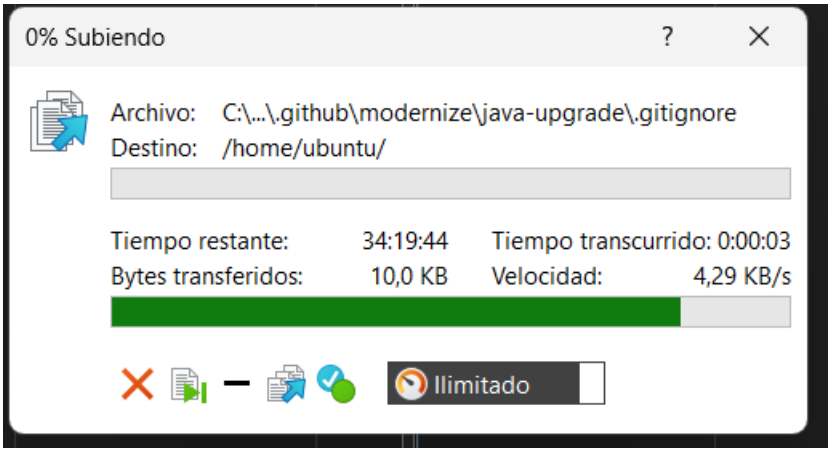
Ahora busco el archivo de mi proyecto y cuando lo encuentro le doy clic derecho y le doy a la opción subir



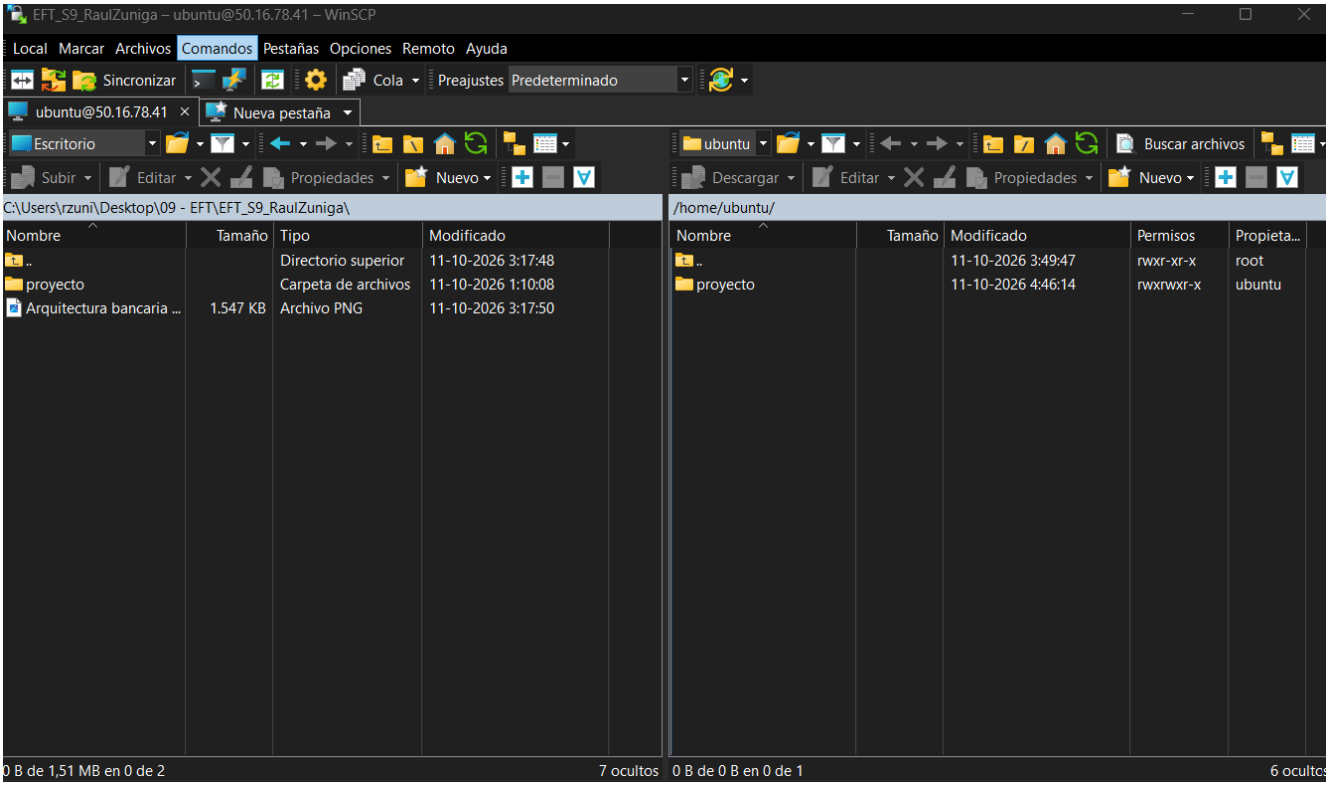
Esto me mostrará la dirección donde quiero que se instale y le doy a aceptar



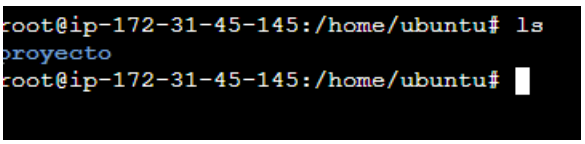
Con esto se comenzará a subir el archivo a mi instancia EC2



El proceso se va a demorar ya que tiene que subir todos los archivos.



Ahora como yo dejé configurado la carpeta en nuestra carpeta principal HOME yo puedo escribir ls y me va a mostrar la carpeta de mi proyecto



Ahora debo ingresar a la carpeta, para ello voy a escribir el comando cd proyecto y una vez dentro voy a volver a escribir el comando ls

```
root@ip-172-31-45-145:/home/ubuntu/proyecto# ls
README.md        bank_legacy_data-main  bff-mobile-service  config-repo    core-banking-service  docker
account-service  bff-atm-service         bff-web-service     config-server  customer-service       docker-compose.yml
```

Como puedo darme cuenta de que existe un docker compose, lo que debo hacer ahora es levantar mi aplicación, para ello utilizo el comando docker compose up -d y espero a que el servicio se comienza a instalar

```
root@ip-172-31-45-145:/home/ubuntu/proyecto# docker compose up -d
```

```
account-service  bff-atm-service         bff-web-service     config-ser
root@ip-172-31-45-145:/home/ubuntu/proyecto# docker compose up -d
[+] up 24/29
 ⠧ Image apache/kafka:3.9.1 [⣿⣿⣿
```



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.